



SATISTICS

Engineering Handbook

Shiv Vyas and contributors · Revision A

SATistics is SAT and GRE practice that plays like an arcade. Real-format exam questions ride inside six games, every topic has a lesson with a worked example and recommended videos, and a dashboard points each student at the topic they miss most. It is a Next.js frontend and a FastAPI backend on one Supabase project, deployed on Vercel, with an LLM and web search behind the question engine. This handbook is the single reference for how it is built, why it is built that way, and how to change it. It is written for contributors: pick the part of the app you want to work on, read its chapter, and every chapter ends with a reading order through the code and questions to check yourself against.

REVISION A 2 OCTOBER 2026 NEXT.JS 14 · FASTAPI · SUPABASE AUDIENCE: CONTRIBUTORS

Table of contents

Every line is a link as well as a page number. Part two is the features, one chapter per feature, each in the same shape.

PART ONE: FOUNDATIONS

1	What SATistics is	6
1.1	The product in one paragraph	6
1.2	Who it is for	7
1.3	The core loop: learn, play, review	7
1.4	Exams, sections and courses	8
1.5	A tour of the screens	9
1.6	What happens on first visit	10
1.7	What is out of scope	10
1.8	Study notes	12
2	Architecture	13
2.1	The three pieces	13
2.2	Outside services	14
2.3	The repository	15
2.4	A request, end to end	16
2.5	How the login token travels	17
2.6	Where state lives	18
2.7	Deployment shape	19
2.8	Code that nothing runs	19
2.9	Study notes	20
3	Running it locally	21
3.1	What runs where	21
3.2	Prerequisites	21
3.3	The database	22
3.4	Environment variables	23
3.5	Getting each key	25
3.6	What works with no optional keys	25
3.7	Running the two servers	26
3.8	Verifying the setup	27
3.9	Common setup failures	28
3.10	Study notes	29
4	The design system	30
4.1	Two ideas: brutal and glass	30
4.2	Files and folders	30
4.3	Tokens	31
4.4	Component classes	31
4.5	Type	32
4.6	Brand components	33
4.7	The app shell	34
4.8	Inside the games	35
4.9	Words	35
4.10	Building a new screen	36
4.11	Accessibility rules	36
4.12	Study notes	38

PART TWO: THE FEATURES

5	Accounts and profiles	40
5.1	Intent	40
5.2	Files and folders	41
5.3	Model	41
5.4	The server side	42
5.5	The client	43
5.6	How it works	43
5.7	Edge cases and tests	44
5.8	Study notes	46
6	Exams, courses and practice settings	47
6.1	Intent	47
6.2	Files and folders	47
6.3	Model	48
6.4	The server side	51
6.5	The client	51
6.6	How it works	52
6.7	Adding an exam or a section	52
6.8	Edge cases and tests	53
6.9	Study notes	54
7	The question engine	55
7.1	Intent	55
7.2	Files and folders	56
7.3	Model	56
7.4	The server side	57
7.5	The client	59
7.6	How it works	60
7.7	Performance and cost	61
7.8	Edge cases and tests	61
7.9	Study notes	62
8	The game framework	63
8.1	Intent	63
8.2	Files and folders	63
8.3	Model	64
8.4	Routes	65
8.5	How it works	66
8.6	Shared exam UI	68
8.7	Input and assets	69
8.8	Performance	70
8.9	Edge cases	70
8.10	Study notes	71

9	The games, and adding one	72		
9.1	The six at a glance	72		
9.2	Zombie Apocalypse	72		
9.3	Whack-A-Mole	73		
9.4	Balloon Pop	73		
9.5	Subway Surfers	74		
9.6	Squid Game	74		
9.7	Pac-Man	75		
9.8	Mario, the one that is not there	75		
9.9	Adding a new game	75		
9.10	Study notes	78		
10	Learn: lessons and videos	79		
10.1	Intent	79		
10.2	Files and folders	79		
10.3	Model	79		
10.4	The server side	81		
10.5	The client	82		
10.6	How it works	83		
10.7	How to add or edit a lesson	83		
10.8	Edge cases and tests	83		
10.9	Study notes	85		
11	Dashboard, statistics and sessions	86		
11.1	Intent	86		
11.2	Files and folders	86		
11.3	Model	87		
11.4	The server side	88		
11.5	The client	89		
11.6	How it works	89		
11.7	Edge cases and tests	90		
11.8	Study notes	92		
12	Mock exams and your own material	93		
12.1	Intent	93		
12.2	Files and folders	93		
12.3	Model	94		
12.4	The server side	95		
12.5	The client	97		
12.6	How it works	98		
12.7	Edge cases and tests	99		
12.8	Study notes	100		
			PART THREE : UNDER THE HOOD	
13	Data model and security	102		
13.1	Where the schema lives	102		
13.2	The tables	103		
13.3	Indexes, triggers and functions	108		
13.4	How the tables relate	109		
13.5	Row Level Security, and why it does not apply	109		
13.6	Who writes each table	111		
13.7	How a token becomes a user	111		
13.8	What lives in the browser	113		
13.9	CORS	113		
13.10	Input validation	114		
13.11	HTML sanitizing	114		
13.12	A security checklist for contributors	115		
13.13	Study notes	116		
14	API reference	117		
14.1	Conventions	117		
14.2	Root and health	118		
14.3	Auth	119		
14.4	Profile	120		
14.5	Games and stats	121		
14.6	Questions	122		
14.7	Learn	123		
14.8	Materials	123		
14.9	The frontend client	125		
14.10	Adding a route	125		
14.11	Study notes	126		
15	Deploying and operating	127		
15.1	Two projects, one repository	127		
15.2	The backend on Vercel	128		
15.3	The frontend on Vercel	128		
15.4	Environment variables per project	129		
15.5	Domains, CORS and deployment protection	130		
15.6	Changing the database	131		
15.7	Search engines	131		
15.8	Rolling back	132		
15.9	The other deploy files	132		
15.10	Operations checklist	132		
15.11	Study notes	134		
16	Contributing	135		
16.1	Picking something to work on	135		
16.2	Branches and commits	137		
16.3	Checks before a pull request	137		
16.4	There are no tests yet	138		
16.5	The pull request	138		
16.6	Common changes, at a glance	139		
16.7	Code style	140		
16.8	Updating this handbook	141		
16.9	Study notes	142		

APPENDICES: REFERENCE

A	Glossary and reading order	144
A.1	Exams and study	145
A.2	Project terms	147
A.3	Infrastructure terms	149
A.4	Reading order: frontend contributor	149
A.5	Reading order: game developer	150
A.6	Reading order: backend contributor	150
A.7	Reading order: questions and AI	150
A.8	Study notes	151

B	Decisions	152
B.1	A separate FastAPI backend	152
B.2	Supabase for auth and data, behind the backend	152
B.3	A bearer token in localStorage and a cookie	152
B.4	Official questions first, then the LLM, then an offline bank	153
B.5	Question length matched to the game	153
B.6	Weak topics decided per game, by fixed thresholds	153
B.7	Lessons as static content	153
B.8	A video provider chain with a one-day cache	154
B.9	Profiles in a table, with a pending-profile flow	154
B.10	Lesson progress in localStorage	154
B.11	Mock exams saved as game sessions	154
B.12	Uploaded questions need approval	155
B.13	A neo-brutalist and glass design system	155
B.14	Game covers drawn as inline SVG	155
B.15	Schema as hand-run SQL files	155
B.16	The backend as one Vercel Python function	155

PART ONE

Foundations

The shape of the system before any feature: what SATistics is for, how the frontend, backend and database fit together, how to get it running on your machine with every integration it uses, and the design system every screen is built from.

- 1. What SATistics is
- 2. Architecture
- 3. Running it locally
- 4. The design system

1 What SATistics is

SATistics is a practice site for the SAT and the GRE that hides real-format exam questions inside arcade games. You shoot the zombie wearing the right answer, run through the right gate, pop the right balloon. Around the games sit short concept lessons with videos, a timed mock exam, a profile with a target score and a test date, and a page for turning your own study material into questions. The one idea that shapes it is that every answer you give is recorded by topic, and the topics you miss are the ones that come back.

This chapter describes the product as the code on `main` builds it. It is the map a new contributor needs before reading any code: who the site is for, what a session looks like, which exams and sections it covers, which screen lives at which route, and what it deliberately does not do. Chapter 2 takes the same product apart into its frontend, backend and database. The repository still carries an earlier name in places: the FastAPI title in `backend/src/main.py` is "NYU Hacks Arcade API", and the root `README.md` is headed "NYU Hacks Arcade". The product name, set in `frontend/lib/site.ts` as `SITE_NAME`, is SATistics, served at `https://www.satistic.tech`.

1.1 The product in one paragraph

`SITE_DESCRIPTION` in `frontend/lib/site.ts` says it plainly: "Train for the SAT and GRE with arcade games built on real-format questions, concept lessons and videos." Each part of that sentence is a feature with its own code.

- **Arcade games.** Six playable games, listed in `frontend/lib/games.ts`: Zombie Apocalypse, Whack-A-Mole, Balloon Pop (id `carnival`), Subway Surfers, Squid Game and Pac-Man. Each one asks exam questions as part of play, and the way you answer is the game's own mechanic. `GAME_META` in `frontend/lib/gameMeta.ts` records that mechanic for each game as `answerBy`, for example "Switch lanes and run through the right gate".
- **Real-format questions.** SAT questions come first from the College Board Educator Question Bank, fetched live by `CollegeBoardSource` in `backend/src/services/question_sources.py`. Anything missing, and all GRE questions, come from an LLM that is handed web pages of practice questions to extract from and writes the rest in the official style. If the backend cannot answer at all, the browser falls back to an offline set of 36 questions in `frontend/lib/questionBank.ts`.
- **Concept lessons and videos.** Fifteen lessons, one per course topic, written as static content in `frontend/lib/lessons.ts`. Each has key ideas, a worked example, the traps that cost points and a one-question quick check. Beside each lesson the backend finds YouTube videos for the topic.
- **Profiles with goals.** A display name, a username, the exam, a target score, a test date, daily minutes and an avatar colour, stored in the `profiles` table.
- **A dashboard** that greets you, counts down to your test date, suggests a game and a focus topic, and shows your recent sessions.
- **Mock exams.** A timed section in test-day format at `/mock`, with no feedback until the end and a score report afterwards.

- **Your own material.** Upload a PDF or text file, or paste text, and the backend pulls the multiple-choice questions out of it (and can write new ones from it). Questions you approve are mixed into your games.

1.2 Who it is for

The person the code is written for is a student preparing for one of two exams, who finds a question bank dull and will play a game for longer than they would sit through a worksheet. The landing page,

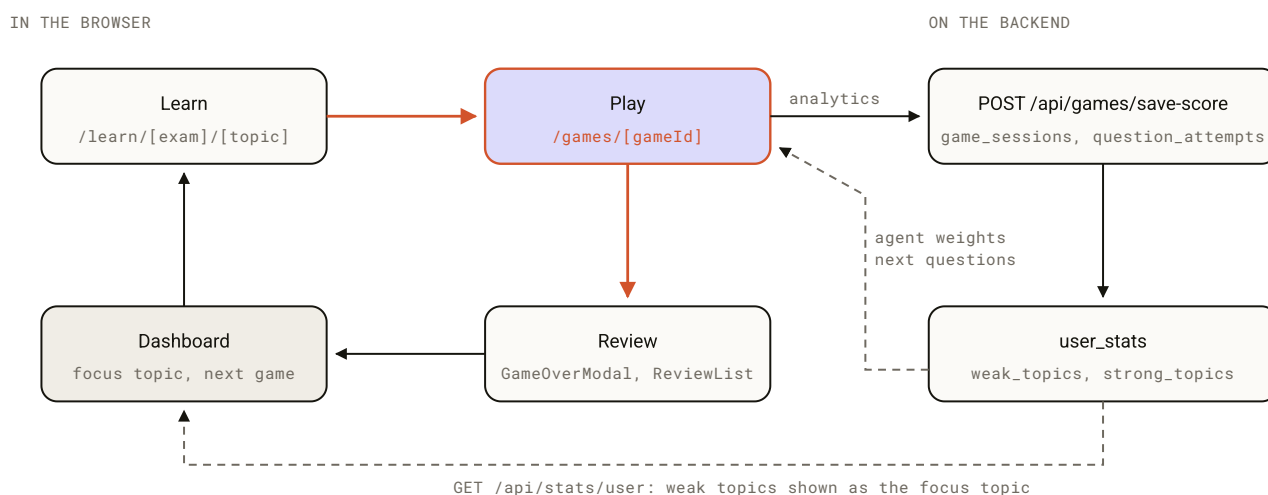
`frontend/app/page.tsx`, makes the pitch: "Every zombie, mole and balloon carries a real-format exam question. Answer right to win, and your weak topics come back until they aren't."

Two assumptions follow from that. First, the student is studying for a specific exam on a specific date, so signup asks for both and the dashboard counts down to the date. Second, the student will not configure much. Exam, section and question count are remembered per browser in the `exam_prefs` localStorage key, defaulting to SAT, quant and ten questions (`DEFAULT_PREFS` in `frontend/lib/exam.ts`), and every route into a game (the dashboard's course cards, a lesson's practice button) sets those preferences for you before it navigates.

The site is free. The structured data in `frontend/app/layout.tsx` declares an `Offer` with price `0`, and there is no payment code anywhere in the repo.

1.3 The core loop: learn, play, review

A session goes round a loop. You read a lesson on a topic. You play a game whose questions lean toward the topics you are weak in. At the end of the game you review every question with your answer and the explanation. The game's analytics go to the backend, which updates your weak and strong topics, and those topics decide what the dashboard suggests you learn next and which questions the next game leans on.



The learning loop. The red path is what the student does; the black paths are what the code does with it. Weak topics are written once per game and read twice: by the dashboard and by the question agent.

Each arrow is real code. A lesson's practice button calls `setExamPrefs` with the lesson's exam and section and pushes `/games`. Each game container fetches questions through `fetchAIQuestions` or `fetchQuestionsWithCache` in `frontend/lib/api/questions.ts`, and when the game ends it calls

`apiClient.saveScore` . `GameService._update_user_stats` in `backend/src/services/game_service.py` marks a topic weak below 50 percent accuracy in that game and strong at 80 percent or above, and moves topics between the two lists as the student improves or slips. The dashboard reads `weak_topics` , keeps the ones in the current exam's courses, and offers the first as the focus. The question agent reads the same table and asks for about half of each difficulty band from weak topics.

Review happens inside the game screen. `components/exam/ReviewList.tsx` lists every question with the player's answer; wrong answers keep the right one hidden until the player asks, "so the review can be used to retry". During play, `frontend/lib/hints.ts` offers a strategy hint matched to the question's skill or wording, and can rule out a wrong choice without naming the right one.

1.4 Exams, sections and courses

Two exams, two sections each, one course per section. The types are in `frontend/lib/exam.ts` : `ExamId` is `'sat' | 'gre'` and `SectionId` is `'quant' | 'verbal'` . The section names and pacing come from the `EXAMS` table; the courses and their topics come from `COURSES` in `frontend/lib/courses.ts` .

COURSE	IDS	REAL PACING	SECONDS PER QUESTION	TOPICS
SAT Math	<code>sat , quant</code>	22 questions in 35 min	95	Algebra; Advanced Math; Problem-Solving and Data Analysis; Geometry and Trigonometry
SAT Reading & Writing	<code>sat , verbal</code>	27 questions in 32 min	71	Information and Ideas; Craft and Structure; Expression of Ideas; Standard English Conventions
GRE Quant	<code>gre , quant</code>	12 questions in 21 min	105	Arithmetic; Algebra; Geometry; Data Analysis
GRE Verbal	<code>gre , verbal</code>	12 questions in 18 min	90	Text Completion; Sentence Equivalence; Reading Comprehension

Fifteen topics in all, and each has exactly one lesson. The topic names are not decoration. The comment on `Course.topics` says they match "the topic names the question bank uses", which is what lets a weak topic recorded from a College Board question be matched to a course and to a lesson.

Games also have a pace. `gamePace` returns `'deep'` for `squid-game` and `'quick'` for everything else. Quick games ask for short questions (the backend drops long ones with `is_quick_question` , which caps the text at 220 characters for quant and 420 for verbal), and the offline bank skips its hard questions for them. Squid Game, where the player freezes on red light and has time to think, takes passages and multi-step problems, with a calculator for math.

Profile score targets use the real scales: `SCORE_RANGES` in `frontend/lib/profile.ts` sets 400 to 1600 in steps of 10 for the SAT and 260 to 340 for the GRE (verbal plus quant).

1.5 A tour of the screens

All pages live under `frontend/app/` in the Next.js App Router. Signed-in pages share `components/DashboardLayout.tsx`, whose sidebar `NAV` lists Home, Learn, Games, Mock exam, My material, Statistics and Profile, and shows the avatar, the username and the days left to the test.

ROUTE	FILE	WHAT IT DOES
<code>/</code>	<code>app/page.tsx</code>	Public landing page: hero, a sample question from the offline bank (<code>HeroQuestion</code>), courses, how it works, the game covers.
<code>/signup</code>	<code>app/signup/page.tsx</code>	Multi-step signup: email and password, then the profile. Accepts <code>?exam=gre</code> .
<code>/login</code>	<code>app/login/page.tsx</code>	Login; saves a pending profile on first login; honours <code>?redirect=</code> .
<code>/dashboard</code>	<code>app/dashboard/page.tsx</code>	Greeting, countdown, the exam's courses, a suggested game, a focus topic with a video, recent sessions.
<code>/learn</code>	<code>app/learn/page.tsx</code>	Every course and its lessons, with learned ticks and your weak and strong topics.
<code>/learn/[exam]/[topic]</code>	<code>app/learn/[exam]/[topic]/page.tsx</code>	One lesson: key ideas, worked example, traps, quick check, videos, "mark as learned".
<code>/games</code>	<code>app/games/page.tsx</code>	"Explore": search and filter by genre, with your plays per game.
<code>/games/[gameId]/select</code>	<code>app/games/[gameId]/select/page.tsx</code>	Choose exam, section and question count (6, 10 or 20) before playing.
<code>/games/[gameId]</code>	<code>app/games/[gameId]/page.tsx</code>	The game itself, full screen, through <code>GameContainer</code> .
<code>/mock</code>	<code>app/mock/page.tsx</code>	Timed mock exam: quick check (10), one module, or a full section (two modules).
<code>/materials</code>	<code>app/materials/page.tsx</code>	Upload material, review the extracted questions, approve them for games.
<code>/stats</code>	<code>app/stats/page.tsx</code>	Accuracy over recent sessions, totals, results by game, recent sessions.
<code>/profile</code>	<code>app/profile/page.tsx</code>	Edit the profile; with <code>?setup=1</code> it is the "finish your profile" step.
<code>/developer</code>	<code>app/developer/page.tsx</code>	Public page about the site's author and what was built.

`frontend/middleware.ts` sends anyone without an `auth_token` cookie from the seven app prefixes (`/dashboard`, `/games`, `/learn`, `/materials`, `/mock`, `/profile`, `/stats`) to `/login?redirect=...`, and sends a signed-in visitor away from `/login` and `/signup` to `/dashboard`. So the public surface is the landing page, the developer page, signup and login, which is exactly what `app/sitemap.ts` lists.

THE MOCK EXAM

The mock exam is the one place the product drops the game. `MODULE_QUESTIONS` in `app/mock/page.tsx` holds the real module sizes (SAT 22 and 27, GRE 12 and 12). The clock is the question count times the section's `secondsPerQuestion`. Questions come from the same `fetchAIQuestions` the games use. When it finishes, the result is saved through the same `saveScore` call with the game id `mock-exam`, "so it feeds statistics and weak topics". The results page gives a rough scaled estimate using `SCORE_RANGE`: 200 to 800 per SAT section, 130 to 170 per GRE section.

YOUR OWN MATERIAL

The materials page posts a file (4 MB at most, enforced on both sides as `MAX_FILE_BYTES` and `MAX_UPLOAD_BYTES`) or pasted text to `POST /api/materials`. The backend parses numbered questions with lettered choices and an answer line or answer key. If the parser finds nothing, or the student ticked the option to write questions, the LLM reads the material too. Every question starts as `pending`; a question with no answer shows "No answer was found. Pick the correct one". Only `approved` questions reach games, where they can make up to half of a set.

1.6 What happens on first visit

A new visitor lands on `/`, clicks "Start free" and goes to `/signup`. The form asks for an email and a password of at least six characters, then the profile. `createAccount` calls `apiClient.signup` and, before anything else, copies the chosen exam into the saved preferences, so the first game is on the right exam.

What happens next depends on a Supabase setting. If email confirmation is off, signup returns an access token, the profile is saved at once with `PUT /api/profile`, and the student lands on `/dashboard`. If it is on, there is no session yet, so the profile cannot be saved. `savePendingProfile` parks it in `localStorage` under `pending_profile`, keyed to the lower-cased email, and the page tells the student to confirm their email. On their first login, `takePendingProfile` finds it, the login page saves it, and the key is removed. If saving fails either way (a taken username returns 409), the student is sent to `/profile?setup=1` with the error as a notice, rather than failing the signup.

1.7 What is out of scope

Knowing what the product does not do saves a contributor from looking for it.

- **No other exams.** `ExamId` is two values and the backend's queries validate `^(sat|gre)$`. There is no ACT, LSAT or GMAT.
- **No free-response or figure questions.** `CollegeBoardSource._normalize` keeps only `mcq` items and drops any whose stem, stimulus or options contain a figure. The LLM prompt says "Skip anything that needs a figure, chart, or image". GRE numeric entry and multi-select types are not produced.
- **No official scoring.** The mock exam's scaled score is a linear estimate and the page calls it rough. There is no adaptive second module.
- **No social features.** Usernames are unique, but nothing shows another user's profile, scores or a leaderboard. High scores are per browser, in `arcade_hi_<gameId>` `localStorage` keys.
- **No lesson authoring.** Lessons are TypeScript constants. There is no editor, no CMS and no LLM in the lesson path.

- **No synced lesson progress.** "Mark as learned" is stored in the `learned_lessons` `localStorage` key. It does not follow the student to another browser.
- **No payments, no admin console, no email of its own.** Confirmation emails, if enabled, are sent by Supabase Auth.
- **No mobile app.** The site supports touch in the games, but there is only the web client.

A GAME THAT EXISTS BUT CANNOT BE PLAYED

`frontend/games/mario/MarioGame.ts` and the `GameRenderer` that constructs it are still in the tree, and the README still lists Mario. But `mario` is not in `games` in `frontend/lib/games.ts`, so `/games/mario` returns `notFound()`. Treat it as dead code. Chapter 2 lists the rest.

1.8 Study notes

READ IN THIS ORDER

1. `frontend/lib/site.ts` , the product's name and one-line description.
2. `frontend/lib/exam.ts` and `frontend/lib/courses.ts` , the exams, sections, pacing and topics.
3. `frontend/lib/games.ts` and `frontend/lib/gameMeta.ts` , the six games and how each one answers.
4. `frontend/app/page.tsx` , the pitch as a student sees it.
5. `frontend/components/DashboardLayout.tsx` , the navigation that ties the screens together.
6. `frontend/app/dashboard/page.tsx` , how weak topics become a suggestion.
7. `frontend/app/mock/page.tsx` , the one screen without a game.
8. `frontend/middleware.ts` , which routes are public.

CHECK YOURSELF

A student finishes a mock exam with poor results on Geometry. Where will that show up?

The mock page calls `saveScore('mock-exam', ...)` , so the attempt lands in `game_sessions` and `question_attempts` , and `_update_user_stats` adds Geometry to `weak_topics` if accuracy was under 50 percent. The dashboard will offer it as the focus topic (if it is in the current exam's courses), and the question agent will lean on it in the next game.

Why must course topic names match the question bank's topic names?

Weak topics are recorded from each question's `topic` . The dashboard keeps only weak topics that appear in `COURSES` , and lessons are found by topic. A mismatch would record weak topics that no course or lesson can ever point to.

A student signs up with email confirmation on, then logs in from a different browser. What happens to the profile they filled in?

It is lost. The pending profile is in the first browser's `localStorage` under `pending_profile` . The second browser has no profile, so the sidebar shows "Profile not set up" and the student fills it in again at `/profile` .

Which game asks long reading passages, and why only that one?

Squid Game. `gamePace` marks it `deep` because the player stops moving on red light and has time to read. Every other game is `quick` , and long questions are filtered out for them.

2 Architecture

SATistics is three pieces: a Next.js 14 frontend that runs the pages and the games in the browser, a FastAPI backend that owns every decision touching data or an outside service, and a Supabase project that holds identity and every table. The browser never talks to Supabase. It talks to the backend with a bearer token, and the backend talks to Supabase with the service-role key. Everything else in this chapter follows from that one boundary.

This chapter is the map. It names the pieces and the outside services, lays out the repository, follows one request from a click to a database row and back, shows how the login token travels, lists where each kind of state lives, and marks the code in the tree that nothing runs, so you can ignore it. Where the code and this handbook disagree, the code is right.

2.1 The three pieces

The frontend is `frontend/`: Next.js 14 with the App Router, React 18, TypeScript, Tailwind CSS, and Three.js for the 3D games. Almost every page is a client component (`'use client'`), and all data comes from the backend through one class, `ApiClient` in `frontend/lib/api/client.ts`, exported as the singleton `apiClient`. The only server-side code the frontend runs is `frontend/middleware.ts`, which checks for a cookie, plus metadata routes (`app/robots.ts`, `app/sitemap.ts`). There are no Next.js API routes.

The backend is `backend/`: FastAPI on Python, built in `backend/src/main.py`. It mounts eight routers, each in `backend/src/api/`, and most of them hand off to a service in `backend/src/services/`.

PREFIX	ROUTER	WHAT IT SERVES	MAIN SERVICE
<code>/api/health</code>	<code>health.py</code>	Liveness, and <code>/supabase</code> to test the database	none
<code>/api/auth</code>	<code>auth.py</code>	signup, login, logout, me	<code>auth_service.py</code>
<code>/api/games</code>	<code>games.py</code>	POST <code>/save-score</code>	<code>game_service.py</code>
<code>/api/stats</code>	<code>stats.py</code>	<code>/user</code> , <code>/sessions</code>	queries inline
<code>/api/questions</code>	<code>questions.py</code>	the question set for a game, <code>/topics</code>	<code>agent.py</code> , <code>custom_questions.py</code>
<code>/api/profile</code>	<code>profile.py</code>	GET and PUT the caller's profile	queries inline
<code>/api/learn</code>	<code>learn.py</code>	<code>/videos</code> for a topic	<code>videos.py</code>
<code>/api/materials</code>	<code>materials.py</code>	upload, list, review, approve, delete	<code>material_parser.py</code>

Supabase provides two things: Auth (email and password users in `auth.users`) and Postgres. The schema is plain SQL in `backend/database/`, pasted into the Supabase SQL editor by hand; there is no migration runner. `schema.sql` creates `game_sessions`, `question_attempts` and `user_stats`; `add_profiles.sql`

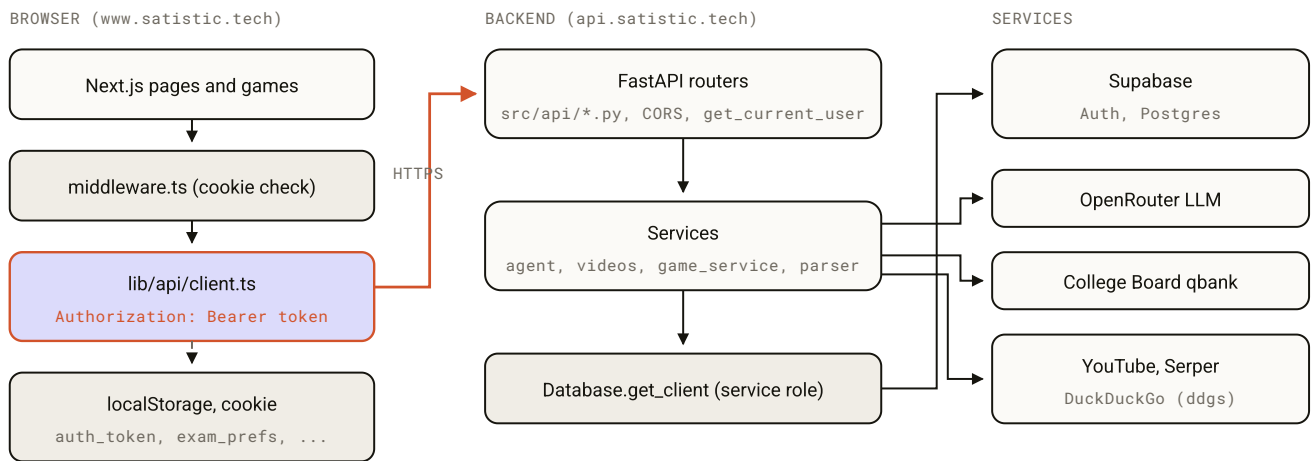
adds `profiles` ; `add_materials.sql` adds `materials` and `custom_questions` ; `reset.sql` drops and recreates the first four. Every table has row level security enabled with owner-only policies, but the backend connects with the service-role key, which bypasses those policies. The backend enforces ownership itself, by filtering every query on the caller's `user_id` .

2.2 Outside services

Only the backend talks to the outside world. Each service is optional except Supabase, and each has a fallback, so the site keeps working with nothing but a database configured.

SERVICE	USED BY	FOR	CONFIGURED BY	WITHOUT IT
Supabase	<code>utils/database.py</code> , <code>auth_service.py</code>	Auth and all tables	<code>SUPABASE_URL</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code>	The backend raises on first use.
College Board question bank	<code>CollegeBoardSource</code>	Official SAT questions	none (public API)	SAT falls through to the LLM.
OpenRouter	<code>agent.py</code> , <code>material_parser.py</code>	Writing and extracting questions	<code>OPENROUTER_API_KEY</code> , optional <code>OPENROUTER_MODEL</code>	No LLM questions; the browser uses the offline bank.
DuckDuckGo (<code>ddgs</code>)	<code>WebQuestionSource</code> , <code>videos.py</code>	Finding practice pages, last-resort video search	none	Rate limited; retried with backoff.
YouTube Data API	<code>videos.py</code>	Lesson videos, first choice	<code>YOUTUBE_API_KEY</code>	Skipped; Serper is tried.
Serper	<code>videos.py</code>	Lesson videos, second choice	<code>SERPER_API_KEY</code>	Skipped; DuckDuckGo is tried.
Vercel	both apps	Hosting	<code>backend/vercel.json</code> , <code>frontend/vercel.json</code>	Run locally with <code>start_local.sh</code> .

The LLM is reached through the `openai` Python client pointed at `https://openrouter.ai/api/v1` .
`LLM_MODEL` in `backend/src/services/agent.py` defaults to `anthropic/claude-haiku-4.5` .
`get_llm_client` returns `None` when no key is set, and every caller checks for that.



Vercel hosts both: the Next.js app, and the backend as a Python function (backend/api/index.py)

The system. The browser holds a token and nothing else secret. Every secret, the service-role key and the API keys, lives on the backend side of the red arrow.

2.3 The repository

README.md	what it is, quick start
CONTRIBUTING.md	how to send a change
CODE_OF_CONDUCT.md, SECURITY.md, CHANGELOG.md, LICENSE (MIT)	
start_local.sh	runs backend (port 8000) and frontend (port 3000)
package.json	root: only "three"; not a workspace
.github/	
workflows/ci.yml	frontend lint, typecheck, build; backend install and import
ISSUE_TEMPLATE/	bug report, feature request, new game
PULL_REQUEST_TEMPLATE.md	
docs/	
handbook/	this book
images/	screenshots
backend/	
.env.example	every backend env var, by name
requirements.txt	fastapi, supabase, openai, ddgs, httpx, pypdf, ...
vercel.json	rewrites every path to /api/index
api/index.py	Vercel entry point: imports src.main.app
run.sh, setup.sh, start.sh	local helpers
database/	
schema.sql	game_sessions, question_attempts, user_stats
add_profiles.sql	profiles
add_materials.sql	materials, custom_questions
add_user_id_to_attempts.sql	
reset.sql	drop and recreate (not materials)
src/	
main.py	the FastAPI app, CORS, routers
config.py	OPENROUTER_API_KEY and Supabase names
api/	one router per prefix
services/	agent, question_sources, videos, game_service,
	auth_service, material_parser, custom_questions,
	supabase_agent_ops

models/schemas.py	Pydantic request and response models
utils/database.py	the Supabase client singleton
utils/html_utils.py	html_to_text, sanitize_html, has_figure
frontend/	
.env.example	NEXT_PUBLIC_API_URL
middleware.ts	route protection
next.config.js, tailwind.config.js, vercel.json	
app/	one folder per route (chapter 1 has the list)
components/	
*GameContainer.tsx	one React wrapper per game
exam/	QuestionCard, HintButton, ReviewList, Calculator, ...
learn/	Videos, QuickCheck
brand/	Logo, Icons, GameCover (SVG covers)
DashboardLayout.tsx	the signed-in shell
games/	the game engines (Three.js), one folder each
lib/	
api/client.ts	ApiClient, the only door to the backend
api/questions.ts	fetchAIQuestions, fetchQuestionsWithCache
exam.ts, courses.ts	exams, sections, topics, saved prefs
lessons.ts	the fifteen lessons
questionBank.ts	the offline question bank
profile.ts, useAccount.ts, learnProgress.ts, sessions.ts, hints.ts	
public/	models, textures, audio for the games

The split to remember: `frontend/lib/` is shared logic and static content, `frontend/components/` is React, and `frontend/games/` is plain TypeScript classes that drive a canvas. On the backend, routers in `src/api/` do auth and shape checking, and services in `src/services/` do the work.

2.4 A request, end to end

Follow one click: a signed-in student on `/games/zombie/select` presses play.

- The page.** The select page has already written the student's choice of exam, section and question count with `setExamPrefs`. It navigates to `/games/zombie`. `middleware.ts` sees the `auth_token` cookie and lets it through.
- The game container.** `app/games/[gameId]/page.tsx` renders `GameContainer`, which hands `zombie` to `ZombieGameContainer`. That component calls `fetchAIQuestions(examPrefs.questionCount, undefined, gamePace(GAME_ID))`.
- The client.** `fetchAIQuestions` reads the prefs and calls `apiClient.getAIQuestions`, which builds `GET /api/questions/?use_agent=true&limit=10&use_web_search=true&exam=sat§ion=quant&pace=quick`. The private `request` method prefixes `NEXT_PUBLIC_API_URL`, adds `Authorization: Bearer <token>`, and turns any non-2xx into an `Error` carrying the response's `detail`.
- The router.** `get_questions` in `src/api/questions.py` resolves the caller with `get_current_user_optional`, so the route also works signed out (as a guest with an all-zero user id). It loads up to half the set from the student's approved material with `load_custom_questions`, then asks an `ExamLearningAgent` for the rest.
- The service.** `generate_questions` reads the student's weak topics from `user_stats` and `question_attempts` through `SupabaseAgentOps`. For the SAT it calls `CollegeBoardSource.fetch`; any

shortfall, and all GRE, goes to `_questions_from_llm`, which may first search the web for practice pages. Every question carries a `source` of `official`, `web` or `ai`.

6. **The response.** The router validates each question against the `Question` model, shuffles, and returns `{ questions, total }`. If the agent throws, the router returns just the custom questions.
7. **The fallback.** Back in the browser, an error or an empty list makes `fetchAIQuestions` return `getBankQuestions(exam, section, pace)` from the offline bank. The game always starts.

When the game ends, the reverse trip is shorter. The container builds a `GameAnalytics` object and calls `apiClient.saveScore`. `POST /api/games/save-score` requires a user (`get_current_user`), and `GameService.save_game_session` inserts one `game_sessions` row, one `question_attempts` row per question, and updates `user_stats`.

Two requests skip `request`. `uploadMaterial` calls `fetch` directly so the browser can set the multipart boundary, adding the bearer header itself. And the backend's `/api/auth/signup` and `/api/auth/login` have explicit `OPTIONS` handlers in addition to the CORS middleware, which allows the origins in `ALLOWED_ORIGINS` plus localhost on ports 3000 and 3001.

2.5 How the login token travels

There is one credential: the Supabase access token, a JWT. It is issued by Supabase, passed through the backend to the browser, kept in two places there, and sent back on every call.

1. **Issued.** `AuthService.login` calls `sign_in_with_password` on the backend's Supabase client and returns `session.access_token`. `AuthService.signup` returns one only when email confirmation is off. The refresh token is returned by the service but dropped by `TokenResponse`; nothing refreshes a session.
2. **Stored.** `ApiClient.setToken` writes the token to `localStorage` under `auth_token` and to a cookie of the same name with `path=/`, `max-age=604800` (seven days) and `SameSite=Lax`. On construction the client reads `localStorage` first and falls back to the cookie, syncing it back.
3. **Checked by Next.js.** `middleware.ts` only checks that the cookie exists. It does not decode or validate it. Its matcher skips `_next` assets, images and `api` paths.
4. **Sent to the backend.** Every call carries `Authorization: Bearer`. FastAPI's `HTTPBearer(auto_error=False)` extracts it, and `get_current_user` answers 401 with "Missing authorization token" or "Invalid or expired token".
5. **Decoded.** `AuthService.get_user` decodes the JWT with `jwt.decode(token, options={"verify_signature": False})`, rejects it if `exp` has passed, and returns `{ id: sub, email }`.
6. **Cleared.** `logout` posts to `/api/auth/logout`, which only acknowledges, and then always calls `setToken(null)`, removing the key and expiring the cookie. `DashboardLayout` also calls `clearAccountCache`.

THE SIGNATURE IS NOT CHECKED

`get_user` trusts the payload of any well-formed, unexpired JWT. Because the backend then queries with the service-role key, a forged token with another user's `sub` would read and write that user's rows. Verifying the token (with the project's JWT secret, or by calling Supabase Auth) is the fix; see Appendix B.

Expiry shows up as a 401 on the next call. The middleware cannot see it, because the cookie outlives the token. `useAccount` turns a failed `/api/auth/me` into a signed-out account, but no code sends the student back to `/login` on a 401.

2.6 Where state lives

IN SUPABASE

TABLE	WRITTEN BY	READ BY	ONE ROW IS
<code>profiles</code>	<code>PUT /api/profile</code>	<code>GET /api/profile</code>	One user's profile; <code>user_id</code> primary key, <code>username</code> unique
<code>game_sessions</code>	<code>save_game_session</code>	<code>/api/stats/sessions</code>	One finished game or mock exam
<code>question_attempts</code>	<code>save_game_session</code>	<code>SupabaseAgentOps.get_topic_performance</code>	One answered question
<code>user_stats</code>	<code>_update_user_stats</code>	<code>/api/stats/user</code> , the agent	Running totals and weak and strong topics
<code>materials</code>	<code>POST /api/materials</code>	<code>GET /api/materials</code>	One upload
<code>custom_questions</code>	upload, review routes	<code>load_custom_questions</code>	One question from an upload, <code>pending</code> or <code>approved</code>

IN THE BROWSER

KEY	STORE	OWNER	HOLDS
<code>auth_token</code>	localStorage and cookie	<code>lib/api/client.ts</code>	The Supabase access token
<code>exam_prefs</code>	localStorage	<code>lib/exam.ts</code>	Exam, section, question count
<code>pending_profile</code>	localStorage	<code>lib/profile.ts</code>	A profile waiting for email confirmation
<code>learned_lessons</code>	localStorage	<code>lib/learnProgress.ts</code>	Ids like <code>sat:Algebra</code>
<code>lesson_video_wide</code>	localStorage	lesson page	Whether the video column is wide
<code>arcade_hi_<gameId></code>	localStorage	<code>lib/arcade.ts</code>	The high score per game
<code>ai_questions_<game>_<exam>_<section></code>	sessionStorage	<code>lib/api/questions.ts</code>	Questions for five minutes (Whack-A-Mole, Balloon Pop)

Every localStorage access sits in a `try`, and each one says what happens when storage is unavailable: progress lasts for the visit, prefs fall back to the defaults.

IN MEMORY

Three caches live in process memory and vanish on restart: on the backend,

`CollegeBoardSource._index_cache` (the question index, six hours) and `_cache` in `videos.py` (video results, one day); in the browser, the module-level `cached` account in `lib/useAccount.ts`, shared across pages so navigation does not refetch the profile. On Vercel each function instance has its own backend caches.

2.7 Deployment shape

Both apps deploy to Vercel as separate projects. The backend's `vercel.json` rewrites every path to `/api/index`, and `backend/api/index.py` adds the backend folder to `sys.path` and imports `app` from `src.main`, which Vercel's Python runtime serves as ASGI. `backend/.vercelignore` keeps `.env`, the shell scripts, `database/` and the other hosts' configs out of the upload. The frontend learns where the backend is from `NEXT_PUBLIC_API_URL`, baked in at build time by `next.config.js`, defaulting to `http://localhost:8000`. The backend must list the frontend's origin in `ALLOWED_ORIGINS`. Locally, `start_local.sh` runs both.

2.8 Code that nothing runs

The repo grew out of a hackathon project and still carries pieces of earlier designs. None of the following is reached from a page or a route. Do not extend them, and do not be misled by them when reading.

- **TypeScript under** `backend/src/`, such as `index.ts`, `api/games.ts`, `config/supabase.ts` and `services/gameScoreService.ts`. They describe a Node backend that was never built; there is no `package.json` in `backend/` to compile them. The live equivalents are the `.py` files beside them.
- **Other hosts' configs:** `backend/Dockerfile`, `fly.toml`, `railway.json`, `render.yaml`. Production is Vercel.
- **The rewrite in** `frontend/vercel.json` to a placeholder Railway URL. The client calls the backend by absolute URL, so `/api/*` on the frontend is never requested.
- `frontend/games/GameRenderer.ts` **and** `frontend/games/mario/`. `GameContainer` only uses the renderer for games without their own container, and the only such game, `mario`, is not in `lib/games.ts`.
- **Unused components:** `components/AuthButton.tsx` and `components/GameCard.tsx` are imported nowhere.
- **Unused dependencies:** `@supabase/ssr` and `@supabase/supabase-js` in `frontend/package.json`. The frontend never talks to Supabase.
- **Unused backend methods:** `ExamLearningAgent.update_performance` and `get_learning_insights`, and the `save_*` and `update_user_stats` methods of `SupabaseAgentOps` they call. Scores are saved by `GameService`. `GET /api/questions/topics` and `apiClient.getQuestions` and `getTopics` have no caller, and `GET /api/questions/` without `use_agent=true` returns an empty list.

The older architecture notes and test script that once sat at the repo root have been removed; this handbook replaces them.

2.9 Study notes

READ IN THIS ORDER

1. `backend/src/main.py` , the app, CORS and the eight routers.
2. `frontend/lib/api/client.ts` , every call the frontend can make.
3. `frontend/middleware.ts` , what the cookie protects.
4. `backend/src/api/auth.py` and `backend/src/services/auth_service.py` , the token's life.
5. `backend/src/utils/database.py` , which key the backend uses and why RLS does not apply to it.
6. `backend/src/api/questions.py` , then `backend/src/services/agent.py` , the main request path.
7. `frontend/lib/api/questions.ts` , the fallback to the offline bank.
8. `backend/database/schema.sql` , `add_profiles.sql` , `add_materials.sql` .
9. `backend/vercel.json` and `backend/api/index.py` , how it is hosted.

CHECK YOURSELF

The RLS policies say users can only see their own rows. Why does that not protect the backend's queries?

`Database.get_client` prefers `SUPABASE_SERVICE_ROLE_KEY` , and the service role bypasses RLS. The backend's own `.eq("user_id", ...)` filters are what keep users apart, so a new query that forgets the filter leaks data.

A token expires while the student is on the dashboard. What do they see?

The middleware still passes them, since the cookie lasts seven days. Backend calls return 401; `getUserStats` and `getRecentSessions` swallow the error and return empty values, so the dashboard looks empty rather than signed out.

OpenRouter is down. Does a GRE game still start?

Yes. `_questions_from_llm` returns an empty list, the router returns only custom questions (possibly none), and `fetchAIQuestions` falls back to `getBankQuestions` , which has nine GRE questions per section.

Why does `uploadMaterial` not use the shared `request` method?

`request` always sets `Content-Type: application/json` . A multipart upload needs the browser to set the content type with its boundary, so it calls `fetch` directly with only the bearer header.

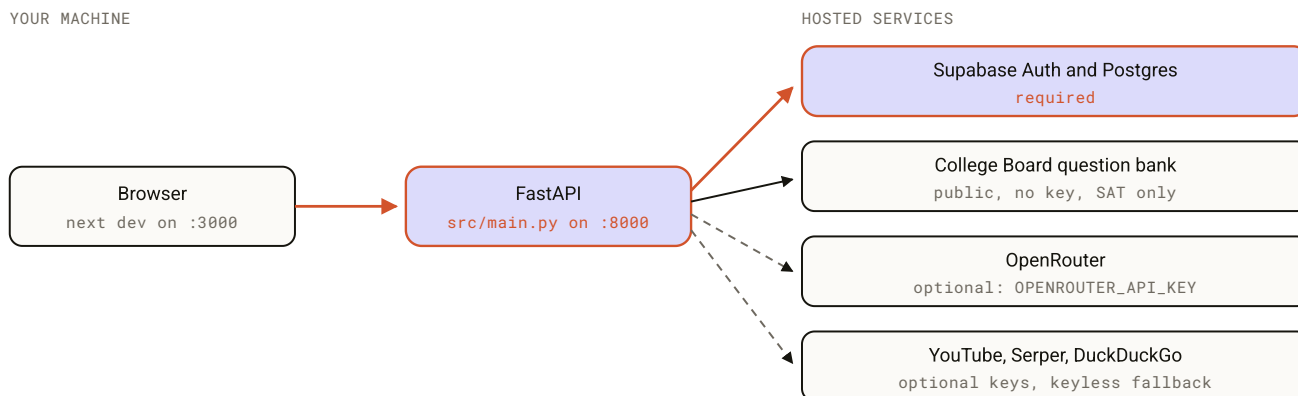
3 Running it locally

SATistics runs on a laptop as two processes and one hosted database: the Next.js dev server on port 3000, the FastAPI server on port 8000, and a Supabase project you create for yourself. Supabase is the only integration you must set up. Everything else (AI questions, lesson video search) has a keyless fallback, so you can play every game and read every lesson before you sign up for anything else.

This chapter walks from an empty machine to a working sign-up. It covers the tool versions the code actually needs, the SQL that builds the database, every environment variable either app reads, where to get each key, what still works when a key is missing, and the failures people hit most often on the way. Two files are the short form of this chapter: `backend/.env.example` and `frontend/.env.example`. Copy them, fill them in, and come back here when something does not start.

3.1 What runs where

The browser only ever talks to the FastAPI server. The frontend does not hold any Supabase key and never calls Supabase directly: sign-up, login, scores, profiles and uploaded material all go through `frontend/lib/api/client.ts` to the backend, which talks to Supabase with the service-role key. That is why the frontend needs exactly one variable and the backend needs the rest.



The browser holds only `NEXT_PUBLIC_API_URL`. Every key lives in `backend/.env`.

A local run. Solid lines are always used; dashed lines are used only when their key is set, except DuckDuckGo, which needs none.

3.2 Prerequisites

You need Node.js, Python 3, Git and a free Supabase account. The versions come from the code, not from preference:

TOOL	MINIMUM	WHY
Node.js	18.17	<code>frontend/package.json</code> pins <code>next</code> at <code>^14.2.5</code> , and Next 14.2 declares <code>"node": ">=18.17.0"</code> in its <code>engines</code> field. CI builds on Node 22, and any current LTS release works.
Python	3.9	The backend code uses <code>typing</code> generics (<code>List</code> , <code>Dict</code> , <code>Optional</code>) and no 3.10 syntax, and runs on 3.9. The repository's <code>README.md</code> says 3.9 or newer. <code>backend/Dockerfile</code> builds on <code>python:3.11-slim</code> , which is a safe choice for a new virtual environment. CI installs the requirements on 3.12.
npm	bundled with Node	The frontend has a <code>package-lock.json</code> ; use <code>npm install</code> , not another package manager.

The shell scripts at the root and in `backend/` mention "Python 3.8 or higher". Trust the table above; the scripts are older than the current dependency list. Section 3.8 says which of those scripts still work.

3.3 The database

Create a new project at supabase.com. Any region and the free tier are fine. When it is ready, open the SQL Editor and run the files in `backend/database/`. There are five, and they are not a numbered migration series. Two of them are the ones you run on a fresh project:

<code>backend/database/reset.sql</code>	drops and recreates <code>game_sessions</code> , <code>question_attempts</code> , <code>user_stats</code> , <code>profiles</code> , the <code>set_updated_at()</code> trigger function, indexes and RLS policies; deletes every auth user
<code>add_materials.sql</code>	materials and custom_questions, their indexes and RLS
<code>add_profiles.sql</code>	profiles only (already included in <code>reset.sql</code>)
<code>add_user_id_to_attempts.sql</code>	adds <code>question_attempts.user_id</code> to an old <code>schema.sql</code> database
<code>schema.sql</code>	the original three tables, without <code>user_id</code> or <code>profiles</code>

For a new project, run `reset.sql` and then `add_materials.sql`, in that order. `reset.sql` is the complete current schema for the core tables, but it does not create the two tables behind "My material" uploads, and the materials pages fail with a missing table error until `add_materials.sql` has run. The other three files exist for databases created before `reset.sql` did; you do not need them on a fresh project. Chapter 13 describes every table and policy.

RESET.SQL DELETES ALL USERS

The file starts with `DROP TABLE IF EXISTS ... CASCADE` for every core table and then runs `DELETE FROM auth.users;`. Every account in the project, and through the `ON DELETE CASCADE` foreign keys every row that belongs to one, is gone. Run it only on a new project or a development project you are happy to wipe. Never run it against production, and never paste it into a SQL Editor tab without checking which project is selected.

Unlike `reset.sql`, `add_materials.sql` is additive and safe to run again: it uses `CREATE TABLE IF NOT EXISTS` and `CREATE INDEX IF NOT EXISTS`, and drops each policy before creating it.

One Auth setting matters for local work. Supabase enables "Confirm email" by default. With it on, `POST /api/auth/signup` creates the user but returns no session, and the backend answers "Please check your email to verify your account". For a development project, turn it off under Authentication, Sign In / Providers, Email, so a sign-up logs you straight in.

3.4 Environment variables

The backend loads `backend/.env` with `python-dotenv`. Three modules do it independently: `src/main.py`, `src/config.py` and `src/utils/database.py` each look for `.env` one level above `src/`. The frontend reads `frontend/.env.local`, which Next.js loads automatically. Both real files are ignored by the root `.gitignore`; the `.env.example` files are explicitly un-ignored so they stay in git.

```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env.local
```

This is every variable the code reads, found by searching for `os.getenv` and `process.env`.

BACKEND

VARIABLE	REQUIRED	READ IN	WHAT IT DOES
<code>SUPABASE_URL</code>	yes	<code>src/utils/database.py</code> , <code>src/config.py</code>	The project URL. <code>NEXT_PUBLIC_SUPABASE_URL</code> is read as a fallback.
<code>SUPABASE_SERVICE_ROLE_KEY</code>	yes	<code>src/utils/database.py</code> , <code>src/config.py</code>	The key the shared Supabase client uses for every query and auth call. It bypasses Row Level Security, which the backend relies on because it filters by user id itself. <code>SUPABASE_SERVICE_KEY</code> is read as a fallback.
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	no	<code>src/utils/database.py</code> , <code>src/config.py</code>	Last fallback for the client key if neither service-role name is set. With only the anon key, RLS blocks the backend's inserts and reads. See 3.9.
<code>OPENROUTER_API_KEY</code>	no	<code>src/config.py</code> , used by <code>src/services/agent.py</code> and <code>src/services/material_parser.py</code>	Turns on AI-written and web-extracted questions, and AI reading of uploaded material.
<code>OPENROUTER_MODEL</code>	no	<code>src/services/agent.py</code>	Any OpenRouter model id. Defaults to <code>anthropic/claude-haiku-4.5</code> .
<code>YOUTUBE_API_KEY</code>	no	<code>src/services/videos.py</code>	First provider for lesson video search.
<code>SERPER_API_KEY</code>	no	<code>src/services/videos.py</code>	Second provider for lesson video search.
<code>ALLOWED_ORIGINS</code>	no locally, yes deployed	<code>src/main.py</code>	Comma-separated origins added to the CORS allow list. The four localhost origins on ports 3000 and 3001 are always allowed.

Two more names are read but change nothing. `src/config.py` reads `SUPABASE_ANON_KEY` into a constant no module imports, and its `SUPABASE_SERVICE_KEY` constant is likewise unused; only the lookups in `src/utils/database.py` decide which key the client gets. The lookup order there is:

```
# backend/src/utils/database.py, Database.get_client()
supabase_url = os.getenv("SUPABASE_URL") or os.getenv("NEXT_PUBLIC_SUPABASE_URL")
supabase_key = (
    os.getenv("SUPABASE_SERVICE_ROLE_KEY")
    or os.getenv("SUPABASE_SERVICE_KEY")
    or os.getenv("NEXT_PUBLIC_SUPABASE_ANON_KEY")
)
```

The client is created lazily on the first request that needs it, not at import time. A backend with no Supabase variables starts, answers `GET /api/health/`, and raises `ValueError: Supabase URL and Key are required` on the first route that touches the database.

FRONTEND

VARIABLE	REQUIRED	READ IN	WHAT IT DOES
<code>NEXT_PUBLIC_API_URL</code>	no locally, yes deployed	<code>lib/api/client.ts</code> , <code>next.config.js</code>	Base URL of the FastAPI server. Both files default to <code>http://localhost:8000</code> .

That is the whole list. Older READMEs, `start_local.sh` and `frontend/deploy.sh` mention `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` for the frontend. No file under `frontend/app` , `components` , `lib` or `games` reads them, and the `@supabase/ssr` and `@supabase/supabase-js` packages in `package.json` are not imported anywhere. Leave them out. Because the variable starts with `NEXT_PUBLIC_` , Next.js inlines its value into the client bundle when it compiles, so after changing it you restart `npm run dev` .

3.5 Getting each key

Supabase. In the dashboard, open Project Settings, then the API keys page. Copy the project URL into `SUPABASE_URL` , the `service_role` key into `SUPABASE_SERVICE_ROLE_KEY` , and the `anon` key into `NEXT_PUBLIC_SUPABASE_ANON_KEY` . Newer dashboards list these JWT-style keys under a "legacy" tab beside the newer publishable and secret keys; the backend was written and tested against the `anon` and `service_role` pair. The service-role key is a root credential for your project. It goes in `backend/.env` and nowhere else.

OpenRouter. Create an account at openrouter.ai, add a small amount of credit, and create a key under Settings, Keys. Put it in `OPENROUTER_API_KEY` . The backend talks to `https://openrouter.ai/api/v1` through the `openai` Python package, so any model id OpenRouter lists can go in `OPENROUTER_MODEL` . A cheap fast model is the right default; each question request is one model call of up to 25 questions (`MAX_LLM_QUESTIONS` in `src/services/agent.py`).

YouTube Data API. In Google Cloud Console, create a project, enable "YouTube Data API v3", and create an API key. Put it in `YOUTUBE_API_KEY` .

Serper. Sign up at serper.dev and copy the API key into `SERPER_API_KEY` .

Video search tries the providers in a fixed order and the first one that returns results wins: YouTube if its key is set, then Serper if its key is set, then DuckDuckGo through the `ddgs` package, which needs no key. Results are cached in memory for 24 hours per exam, section and topic (`CACHE_TTL_SECONDS` in `src/services/videos.py`), and only non-empty results are cached. Chapter 10 covers the ranking.

CORS. Locally you can leave `ALLOWED_ORIGINS` unset, because `http://localhost:3000` , `http://localhost:3001` and the `127.0.0.1` forms of both are hard-coded in `src/main.py` . Set it only if you serve the frontend from another origin, such as a LAN address to test on a phone.

3.6 What works with no optional keys

With only the Supabase variables set, the app is complete enough to develop almost anything:

- **Accounts, profiles, scores and stats** work. They need Supabase and nothing else.

- **SAT questions** still come from the real College Board question bank. `ExamLearningAgent.generate_questions` calls `CollegeBoardSource.fetch` for SAT before it considers the model, and that public API needs no key. Only the shortfall goes to `_questions_from_llm`, which logs "OPENROUTER_API_KEY not set" and returns nothing.
- **GRE questions** come from the offline bank. When the API returns no questions, `fetchAIQuestions` in `frontend/lib/api/questions.ts` falls back to `getBankQuestions` from `frontend/lib/questionBank.ts`.
- **Games** run entirely in the browser from `frontend/games/` and only need questions and a place to save the score.
- **Lessons** are static content in `frontend/lib/lessons.ts` and `frontend/lib/courses.ts`.
- **Lesson videos** use DuckDuckGo, which rate limits after a burst of searches. An empty video list during development is usually that, not a bug.
- **Material uploads** still parse PDFs and text with the rule-based parser in `src/services/material_parser.py`; the AI pass that finds more questions is skipped.

Without Supabase you can load the landing page and the backend health check, and nothing more.

`frontend/middleware.ts` redirects `/dashboard`, `/games`, `/learn`, `/materials`, `/mock`, `/profile` and `/stats` to `/login` unless an `auth_token` cookie is present, and only the backend can issue one.

3.7 Running the two servers

BACKEND

```
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn src.main:app --reload --port 8000
```

Run uvicorn from `backend/`, because every import is absolute from the `src` package (`from src.api import auth`). `python src/main.py` also works: its `__main__` block puts `backend/` on `sys.path` and starts uvicorn on `0.0.0.0:8000` with reload. `backend/run.sh` wraps that second form and installs the requirements if `fastapi` is missing. Interactive API docs are at `http://localhost:8000/docs`.

FRONTEND

```
cd frontend
npm install
npm run dev
```

The dev server listens on `http://localhost:3000`. Open it, sign up, and the three-step sign-up creates your profile and drops you on the dashboard.

BOTH AT ONCE

`start_local.sh` at the repository root starts both in the background, writes `backend.log` and `frontend.log`, and stops both on Ctrl+C. If `backend/.env` or `frontend/.env.local` is missing it writes a stub, so copy the `.env.example` files first; its stub uses underscore placeholders and omits the optional keys.

3.8 Verifying the setup

```
curl http://localhost:8000/api/health/  
# {"status":"healthy","service":"NYU Hacks Arcade API"}  
  
curl http://localhost:8000/api/health/supabase  
# {"status":"healthy","supabase":"connected","database":"accessible"}
```

The first route proves the process is up. The second creates the Supabase client and selects one row from `game_sessions`, so it proves the URL, the key and the schema in one call. It always answers 200; read the `status` field, and on failure the `error` field carries the exception text. Note the trailing slash on `/api/health/`: the router declares `"/"`, so the bare path answers with a redirect.

Then do the real check: sign up in the browser, play one round of any game, and open `/stats`. A row in `game_sessions` in the Supabase table editor means the whole path works.

Two scripts in `backend/` no longer work: `setup.sh` and `start.sh` import `config` and run `main.py` from the backend root, and neither file exists there any more (they moved under `src/`). `check_server.sh` still works; it checks port 8000 and a CORS preflight.

3.9 Common setup failures

SYMPTOM	CAUSE	FIX
<code>ModuleNotFoundError: No module named 'src'</code>	uvicorn started from inside <code>backend/src</code> or from the root	Run it from <code>backend/</code> .
<code>/api/health/supabase</code> says <code>relation "public.game_sessions" does not exist</code>	The SQL was not run, or was run in another project	Run <code>reset.sql</code> in the project whose URL is in <code>.env</code> .
Uploads or the materials page fail on <code>materials</code> or <code>custom_questions</code>	<code>add_materials.sql</code> was skipped	Run it.
Saving a score fails with a row-level security error	The client fell back to the anon key because neither service-role variable is set	Set <code>SUPABASE_SERVICE_ROLE_KEY</code> and restart.
Sign-up succeeds but you stay on the sign-up page with "check your email"	"Confirm email" is on	Turn it off for the dev project, or confirm from the email.
"Too many signup attempts"	Supabase's built-in email rate limit	Wait, or turn off email confirmation.
Browser console shows a CORS error	The page is served from an origin not on the list (for example a LAN IP or port 3002)	Add it to <code>ALLOWED_ORIGINS</code> and restart the backend.
Every API call fails with "Failed to fetch"	Backend not running, or <code>NEXT_PUBLIC_API_URL</code> points somewhere else	Check <code>/api/health/</code> ; restart <code>npm run dev</code> after editing <code>.env.local</code> .
Every page redirects to <code>/login</code>	No <code>auth_token</code> cookie, often after clearing site data	Log in again.
Games show the same small set of GRE questions	No <code>OPENROUTER_API_KEY</code> , so the offline bank is in use	Expected. Add a key for variety.
Lessons show no videos	DuckDuckGo rate limiting	Wait a minute, or set <code>YOUTUBE_API_KEY</code> .
<code>start_local.sh</code> asks "Port 8000 is already in use"	An old backend is still running	<code>lsof -ti:8000</code> shows its PID; stop it.

3.10 Study notes

READ IN THIS ORDER

1. `backend/.env.example` and `frontend/.env.example`, the short form of this chapter.
2. `backend/src/utils/database.py`, for the key lookup order and the lazy client.
3. `backend/src/main.py`, for `.env` loading, the CORS list and the router prefixes.
4. `backend/database/reset.sql`, then `add_materials.sql`, the schema you just built.
5. `backend/src/api/health.py`, the two checks you ran.
6. `frontend/lib/api/client.ts`, the only place the frontend reaches the network.
7. `frontend/middleware.ts`, for why signed-out pages redirect.

CHECK YOURSELF

The backend starts and `/api/health/` is healthy, but `.env` is empty. When does it fail?

On the first request that calls `Database.get_client()`, which raises `ValueError` because the client is built lazily, not at import.

You ran `reset.sql` on a new project. What else must you run?

`add_materials.sql`, because `reset.sql` does not create `materials` or `custom_questions`.

Why does the frontend not need any Supabase key?

It never talks to Supabase. All auth and data calls go through the FastAPI server, which holds the service-role key.

With no `OPENROUTER_API_KEY`, where do SAT questions come from?

The public College Board question bank, through `CollegeBoardSource.fetch`. Only GRE falls back to the offline bank in `frontend/lib/questionBank.ts`.

You changed `NEXT_PUBLIC_API_URL` and the browser still calls the old URL. Why?

`NEXT_PUBLIC_` values are inlined when Next.js compiles. Restart the dev server (or rebuild and redeploy in production).

4 The design system

SATistics looks like a filled-in answer sheet that learned to play games: thick ink outlines, hard offset shadows, flat blocks of cobalt and lime, and the occasional frosted glass panel floating over soft paper. The whole system is small. It is seven color tokens and four shadows in the Tailwind config, a dozen component classes in one stylesheet, two fonts, a handful of SVG brand pieces and one app shell. This chapter walks through each of them and ends with the rules for building a screen that fits.

There is no component library and no design token package. Pages are client components that compose Tailwind utilities with the classes in `frontend/app/globals.css`. That keeps the system easy to read and easy to break, so most of this chapter is about the conventions the existing screens follow rather than about APIs.

4.1 Two ideas: brutal and glass

The comment above the component layer in `globals.css` names the identity in one line: "brutalist outlines with frosted glass panels". The two halves do different jobs.

Neo-brutalism carries anything you can press or anything that holds content. It is a 2 px `ink` border, a solid offset shadow with no blur, generous radii (16 to 32 px), and flat fills from the palette. Buttons move toward their shadow when hovered and sink into it when pressed, which makes the shadow read as physical depth. Cards, form fields, chips, the shell's outer frame and the game covers all use the same outline.

Glass is for decoration and for secondary panels that sit over something busy: the landing page's section nav, the "how it works" steps, the key idea cards on a lesson, the caption on the developer page and the "Today's focus" card on the auth screens. It is a translucent white fill, a thin white border, a soft blurred shadow and `backdrop-blur-xl`. Glass only works over something with texture, which is why the app's background is the `.paper-blobs` gradient rather than a flat color.

The brand motif is the exam answer sheet. The logo mark is four answer bubbles laid out like a gamepad's face buttons, a decorative `BubbleRow` shows A to D with one bubble filled, and a `Pencil` draws a No. 2 test pencil in coral, paper and lime.

4.2 Files and folders

<code>frontend/tailwind.config.js</code>	color tokens, font families, shadows, 4xl radius
<code>frontend/app/globals.css</code>	component classes, focus ring, reduced motion, game skins
<code>frontend/app/layout.tsx</code>	loads Bricolage Grotesque and Instrument Serif
<code>frontend/lib/arcade.ts</code>	loads Press Start 2P for the arcade games, high scores
<code>frontend/components/brand/Logo.tsx</code>	Logo, LogoMark, BubbleRow, Pencil
<code>frontend/components/brand/Icons.tsx</code>	17 stroke icons that inherit <code>currentColor</code>
<code>frontend/components/brand/GameCover.tsx</code>	SVG cover art for each game

```
frontend/components/DashboardLayout.tsx the signed-in app shell
frontend/components/auth/AuthShell.tsx  the signed-out shell, Field, FormError
frontend/components/arcade/ArcadeFrame.tsx pixel cabinet: ArcadeFrame, ArcadeTopBar, ArcadeStar
frontend/components/arcade/GameHud.tsx   site-styled HUD: HUD_PANEL, GameTopBar, Pips
```

The Tailwind `content` globs cover `pages/`, `components/` and `app/`. A class written anywhere else, for example in a string inside `frontend/games/` or `frontend/lib/`, is not scanned and will be purged from the build. Keep class names in those three folders.

4.3 Tokens

All tokens live under `theme.extend` in `frontend/tailwind.config.js`, so the default Tailwind palette is still available. The house rule is to use the tokens and nothing else for color; the only other hues in the codebase are the pink `#F45D9C` that appears as an avatar swatch and in cover art, and the neon colors inside the arcade games.

TOKEN	VALUE	SOFT VARIANT	USED FOR
<code>paper</code>	<code>#ECEAE4</code>	none	Page background, the html element, <code>.theme-paper</code>
<code>ink</code>	<code>#17171C</code>	none	Text, every outline, every hard shadow, the active nav item
<code>cobalt</code>	<code>#3A33E8</code>	<code>#DCDBFB</code>	Primary accent, links, focus ring, hero cards, filled bubbles
<code>lime</code>	<code>#D4F34A</code>	<code>#F1FBC9</code>	Call to action, success, "learned", strong topics
<code>coral</code>	<code>#FF6B4A</code>	<code>#FFE1D9</code>	Errors, wrong answers, weak topics, traps
<code>mist</code>	<code>#F7F6F2</code>	none	The shell's inner surface, <code>.btn-paper</code> , light HUD chips

`cobalt`, `lime` and `coral` are objects with a `DEFAULT` and a `soft` key, so both `bg-lime` and `bg-lime-soft` exist. The soft variants are tints for large areas (a lesson's traps section is `bg-coral-soft`, a quick check is `bg-cobalt-soft`); the full colors are for small, loud things.

Four shadows are defined. `shadow-brutal` is `4px 4px 0 0` in ink, `shadow-brutal-sm` is `2px` and `shadow-brutal-lg` is `8px` (used once, on the shell's outer frame). `shadow-glass` is a large soft drop shadow plus a one-pixel white inset highlight. Hover states that need a `5px` or a cobalt shadow write an arbitrary value instead, for example `shadow-[3px_3px_0_0_#3A33E8]` on the active nav link and on a focused field. The config also adds `rounded-4x1` (`2rem`), though most screens write radii such as `rounded-[24px]` directly.

Two font families are declared: `font-display` reads the CSS variable `--font-display` and `font-serif` reads `--font-serif`, each with a system fallback. Section 4.5 covers where those variables come from.

4.4 Component classes

`globals.css` defines its reusable pieces inside `@layer components` with `@apply`, so they sort before utilities and any utility on the same element wins. That is deliberate: `className="btn btn-ink w-full py-3.5"` overrides the button's padding without a specificity fight.

CLASS	WHAT IT APPLIES
<code>.theme-paper</code>	<code>min-h-screen bg-paper text-ink</code> . Put it on the root of every non-game screen.
<code>.brutal</code>	2 px ink border and <code>shadow-brutal</code> . The outline on its own.
<code>.glass</code>	<code>border-white/70 bg-white/55 shadow-glass backdrop-blur-xl</code> .
<code>.btn</code>	Inline flex, <code>rounded-2xl</code> , ink border, bold 15 px text, <code>shadow-brutal</code> . Hover lifts by one pixel and grows the shadow to 5 px; active shifts 3 px and drops the shadow; disabled is half opacity with no pointer events.
<code>.btn-ink</code> , <code>.btn-lime</code> , <code>.btn-cobalt</code> , <code>.btn-paper</code>	Fill and text color only. Always combine with <code>.btn</code> .
<code>.arrow-btn</code>	A 44 px round outlined button for a single icon, usually an arrow that rotates 45 degrees on group hover.
<code>.field</code>	Full width, <code>rounded-2xl</code> , ink border, white fill, <code>shadow-brutal-sm</code> , no outline. On focus the shadow turns cobalt.
<code>.chip</code>	Small pill with an ink border and semibold text, for tags, toggles and topic lists.
<code>.paper-blobs</code>	Three radial gradients (two white, one cobalt-soft) over paper. Outside the layer, plain CSS.

Below the component layer the file sets a global `:focus-visible` ring (3 px solid cobalt, 3 px offset) and a `prefers-reduced-motion` rule that cuts every animation and transition to 0.01 ms. It also holds styles that belong to other features: `.question-content` for exam questions rendered from HTML and MathML, `.dark-scroll` , `.game-hud` and `.game-reading` for text over game scenes, the `.arcade-*` classes and the `.hud-panel` skins described in section 4.8.

TWO LEFTOVERS.

The `body` rule sets `background: #0a0a0a` and `color: #fff` , from when every screen was dark. The `html` element is paper, and every current screen paints its own background, so you only see it if a page forgets `.theme-paper` . About 400 lines of `.retro-*` Game Boy classes are still in the file; their only user is `frontend/components/GameCard.tsx` , which nothing imports.

4.5 Type

`frontend/app/layout.tsx` loads two Google fonts with `next/font/google` , which self-hosts them at build time. `Bricolage_Grotesque` becomes the CSS variable `--font-display` ; `Instrument_Serif` , at weight 400 in normal and italic, becomes `--font-serif` . Both use `display: 'swap'` . The root `<html>` carries both variable classes and `<body>` carries `font-display` , so the grotesque is the default everywhere.

Headings are heavy and tight: `font-extrabold tracking-tight` at `text-4xl` or `text-5xl` for a page title, `text-2xl` or `text-xl` for a section. The serif is an accent and only ever appears in italic, as the second line of a big headline: "Your score, *leveled up.*" on signup, "Back for *another round?*" on login, "one game at a time." on the landing page. The pattern in JSX is a `<span className="block font-serif font-normal italic">` inside the `<h1>` . Do not set body copy in the serif.

A third font, `Press_Start_2P`, is loaded in `frontend/lib/arcade.ts` as `--font-arcade`. It is exported as `arcadeFontVariable` and only applied inside `ArcadeFrame`, so the pixel font is downloaded only on pages that use it.

4.6 Brand components

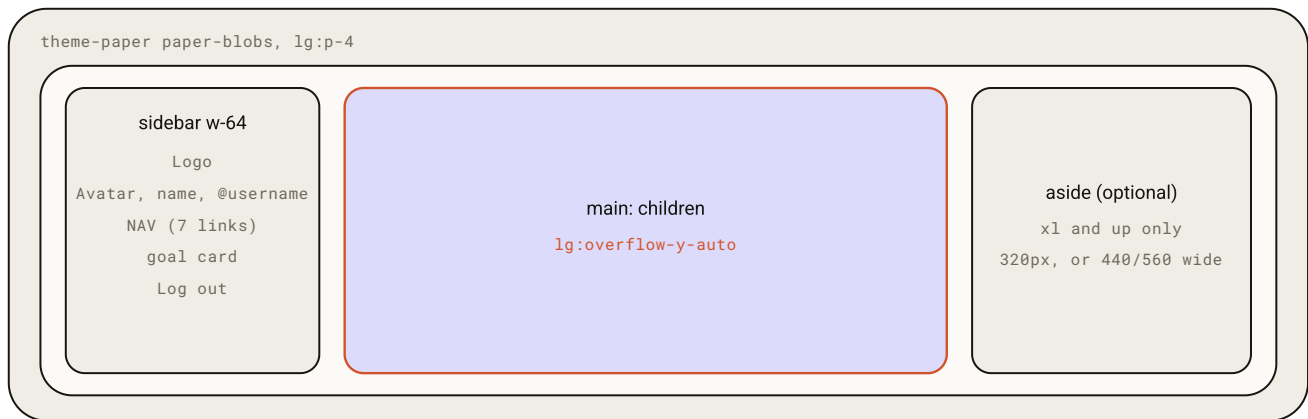
`frontend/components/brand/Logo.tsx` exports four pieces. `Logo` is the wordmark: the text "SATistics" in 22 px extrabold inside a `Link`, with an `inverted` prop for white on cobalt and an `href` that defaults to `/` (the shell passes `/dashboard`). `LogoMark` is the square icon: an ink tile with rounded corners, a lime bubble on top and three outlined bubbles, inverted to a white tile with cobalt rings when asked. `BubbleRow` draws four lettered bubbles and fills the one at index `filled` (default 2, so C) in cobalt. `Pencil` draws the pencil. All three SVGs are `aria-hidden`; they are decoration.

`Icons.tsx` holds 17 stroke icons on a 24 by 24 grid: `ArrowUpRight`, `ArrowLeft`, `HomeIcon`, `GamepadIcon`, `ChartIcon`, `UserIcon`, `SearchIcon`, `LogoutIcon`, `CheckIcon`, `PlayIcon`, `MenuIcon`, `CloseIcon`, `BookIcon`, `UploadIcon`, `ExpandIcon`, `ShrinkIcon` and `ClockIcon`. A shared `base()` helper gives each one `stroke="currentColor"`, a 2.2 stroke width, round caps and joins, `aria-hidden`, and a default size of `h-5 w-5`. Color an icon by setting the text color of its parent; size it with a `className`. Add new icons here in the same shape rather than pulling in an icon package.

`GameCover.tsx` draws a cover for each game as SVG on a 300 by 400 view box, using constants that mirror the tokens (`INK`, `PAPER`, `LIME`, `COBALT`, `CORAL`, plus `PINK`). The `COVERS` record has an entry, a background color and an `art` fragment, for `zombie`, `whackamole`, `carnival`, `subway-surfers`, `squid-game` and `pac-man`. The zombie cover is a lime crosshair over a zombie hand in front of two answer tombstones; Whack-A-Mole is a cobalt mole holding a "B" card under a coral mallet. A small `letter()` helper places an answer letter in the display font. Unknown ids get `FALLBACK`, a lime circle on ink.

The SVG uses `preserveAspectRatio="xMidYMid slice"` and fills its parent absolutely, so the parent decides the shape: `aspect-[3/4]` on the auth screens, a square `w-16` thumbnail in session lists, a circle on the dashboard. With `showTitle` on (the default) the game's title from `GAME_META` sits in a white outlined label at the bottom left. A new game needs an entry in `COVERS` or it will show the fallback everywhere.

4.7 The app shell



`DashboardLayout` at desktop width. Below `lg` the frame drops away, the sidebar becomes a menu under a top bar, and the aside is not rendered at all.

Every signed-in page renders inside `DashboardLayout` from `frontend/components/DashboardLayout.tsx`. It takes `children`, an optional `aside` and an `asideWide` flag. The outer div is `theme-paper paper-blobs`; inside it a `bg-mist/80` panel becomes, from `lg` up, a rounded 32 px frame with an ink border and `shadow-brutal-1g` that fills the viewport, with its own scrolling main column.

The sidebar shows the `Logo` , then the signed-in person's `Avatar` (76 px, from `components/profile/ProfileFields.tsx`), name and `@username` , read through `useAccount` . While the account loads, a pulsing outlined circle stands in. The `NAV` array lists seven links: Home, Learn, Games, Mock exam, My material, Statistics and Profile. A link is active when the path matches exactly, except Games and Learn, which match by prefix so a lesson or a game's select page keeps its tab lit. The active link is an ink pill with white text and a cobalt offset shadow, and carries `aria-current="page"` .

At the bottom, a dashed goal card shows the exam's target score and either a test-day countdown (from `daysUntil`) or the daily minutes. If the account has no profile, a lime "Set up your profile" link takes its place. "Log out" calls `apiClient.logout()` , clears the account cache and pushes to `/login` whether or not the request succeeded.

Below `lg` a top bar holds the logo and an `.arrow-btn` menu toggle with `aria-expanded` and a label that flips between "Open menu" and "Close menu". The menu renders the same sidebar markup inline. The `aside` column is `hidden xl:block`, so anything a page puts there is invisible below 1280 px unless the page repeats it in the main column. The lesson page does this (its video panel has an `xl:hidden` copy); the dashboard does not, so its courses, focus topics and "Haven't tried yet" covers do not appear on tablets or phones.

Signed-out screens use `AuthShell` instead: a cobalt showcase panel on the left from `lg` up (logo, a headline, a caption, three tilted `GameCover` s behind a glass card) and the form column on the right, capped at 440 px. It also exports `Field` , which pairs a bold label with an optional hint, and `FormError` , a coral-soft box with `role="alert"` .

4.8 Inside the games

Games run full screen over 3D or canvas scenes, so they do not use the shell. There are three looks, chosen per game.

Site-styled HUD. `frontend/components/arcade/GameHud.tsx` brings the brutal outline into the game. `HUD_PANEL` is a class string for an ink card with a lime offset shadow, used for question panels; question cards are styled for a dark background, so the panel stays ink even on a light scene. `GameTopBar` puts a round pause button on the left and a row of stat chips in the middle, each chip toned `paper`, `lime`, `cobalt` (soft) or `coral` (soft) through a `TONES` map. An empty 44 px span on the right balances the pause button so the chips stay centered. `Pips` draws filled and empty dots for lives or shots and labels itself "3 of 5" for screen readers. Whack-A-Mole, Balloon Pop and Pac-Man use this set.

Arcade cabinet. `ArcadeFrame.tsx` wraps a game in a fixed black screen that sets `--arcade-color` from its `color` prop, applies the pixel font variable and lays the `.arcade-crt` scanline and vignette overlay on top. `ArcadeTopBar` is a marquee of red labels over glowing values, with pause and mute buttons; `ArcadeStartScreen` is the attract screen with a zero-padded "HI-SCORE", a START button, a blinking "INSERT COIN" and a plain "Back to games" link. High scores come from `getHighScore` and `recordHighScore` in `lib/arcade.ts`, stored in `localStorage` under `arcade_hi_<gameId>`. Only the zombie game uses this frame today.

Themed skins. Subway Surfers and Squid Game use `.hud-panel`, whose background, accent, radius and shadow come from CSS variables. `.skin-subway` sets yellow on deep navy with squared corners, like station signage; `.skin-squid` sets tracksuit green with a guard-pink offset shadow. `.hud-banner` paints a strip in the accent color across the top of a panel. A new skin is four variables in a new `.skin-*` class.

4.9 Words

The copy is part of the system and follows a few rules you can check by reading any screen.

- **Sentence case everywhere.** "Mark as learned", "Set up your profile", "Recent sessions", "Haven't tried yet". Never Title Case on buttons or headings. The arcade attract screen is the one place that shouts in capitals, on purpose.
- **No emoji in the interface.** Icons come from `Icons.tsx`. The arcade top bar's menu and music glyphs are text symbols inside labelled buttons. The `name` fields in `frontend/lib/games.ts` still carry older titles with emoji; show titles through `gameTitle` from `frontend/lib/gameMeta.ts` instead.
- **Plain verbs on buttons.** "Log in", "Continue", "Create profile", "Open lesson", "Practice in a game", "Play Whack-A-Mole". Say what will happen.
- **Talk to one person, briefly.** "Your account is ready. Finish your profile to start." Empty states say what will fill them: "Finish a game and your score, accuracy and streak show up here."
- **Errors say what to do.** "Usernames are 3 to 24 lowercase letters, numbers or underscores." "The username @maya is taken. Try another one." End sentences with a period.
- **Loading text ends in an ellipsis character:** "Logging in...", "Finding a video lesson..."

4.10 Building a new screen

A new signed-in page is a client component in `frontend/app/<name>/page.tsx`. Follow the shape of `frontend/app/stats/page.tsx`, which is the simplest complete example.

1. Wrap the page in `<DashboardLayout>`. Add the route to the `NAV` array if it deserves a tab, and to `protectedPrefixes` in `frontend/middleware.ts` if it needs a login.
2. Inside, use `<div className="mx-auto max-w-4xl p-5 sm:p-8">` (or `max-w-3xl` for reading, `max-w-5xl` for grids).
3. Open with an `h1` in `text-5xl font-extrabold tracking-tight` and a one-sentence `text-ink/65` description under it.
4. Group content in cards: `rounded-[24px] border-2 border-ink bg-white p-5`. Promote the one most important card with a fill (`bg-cobalt text-white` or `bg-lime`) and `shadow-brutal`. Use soft tints for status, never for decoration.
5. Use `.btn` with one variant for actions. One `btn-ink` or `btn-lime` per view is the primary action; others are `btn-paper`.
6. Show skeletons while loading (`animate-pulse rounded-[22px] bg-white` at the size of what is coming), and a dashed outlined box with a next step when empty.
7. If you use `aside`, decide what small screens see instead.

A minimal page looks like this:

```
'use client'
import { DashboardLayout } from '@components/DashboardLayout'

export default function ExamplePage() {
  return (
    <DashboardLayout>
      <div className="mx-auto max-w-4xl p-5 sm:p-8">
        <h1 className="text-5xl font-extrabold tracking-tight">Example</h1>
        <p className="mt-2 max-w-xl text-ink/65">One sentence on what this page is for.</p>
        <section className="mt-8 rounded-[24px] border-2 border-ink bg-white p-5">
          <h2 className="text-xl font-extrabold">A section</h2>
          <button className="btn btn-ink mt-4">Do the thing</button>
        </section>
      </div>
    </DashboardLayout>
  )
}
```

4.11 Accessibility rules

The codebase already follows these; keep doing so.

- **Never remove the focus ring.** `:focus-visible` draws a cobalt outline on everything. `.field` sets `outline-none` but replaces it with a cobalt shadow; if you suppress an outline, replace it.

- **Respect reduced motion.** The global rule already zeroes transitions and animations. Do not drive essential state changes with animation alone, and add a `prefers-reduced-motion` override for any keyframe animation you add, as `.arcade-blink` does.
- **Label icon-only buttons.** The menu toggle, the pause button, the back arrow on signup, the play overlay on a video thumbnail and each avatar swatch all carry an `aria-label`.
- **Expose state.** Toggles use `aria-pressed` (exam choice, daily minutes, avatar color, "Mark as learned"), nav uses `aria-current="page"`, the signup steps use `aria-current="step"`.
- **Announce results.** Errors render in `FormError` with `role="alert"`; quick check feedback and "Profile saved" sit in `aria-live="polite"` regions.
- **Decorative SVG is hidden.** Logo marks, bubbles, the pencil, covers and icons are all `aria-hidden`. Do not put meaning only in a cover; put the game's title in text beside it.
- **Keep contrast.** Ink on paper, mist, white, lime and every soft tint is high contrast. White text goes on ink, cobalt and coral only. Muted text uses `text-ink/55` to `text-ink/70`; do not go lighter for anything a person needs to read. `avatarTextColor` switches to ink on the lime swatch for the same reason.
- **Label every field.** Use `Field` with a matching `htmlFor` and `id`, and group choice buttons in a `fieldset` with a `legend`.

4.12 Study notes

READ IN THIS ORDER

1. `frontend/tailwind.config.js` , the whole token set in 40 lines.
2. `frontend/app/globals.css` , the first 90 lines, then the game sections from `.game-hud` down.
3. `frontend/app/layout.tsx` , how the fonts become CSS variables.
4. `frontend/components/DashboardLayout.tsx` , the shell and its breakpoints.
5. `frontend/components/auth/AuthShell.tsx` , glass over cobalt, and `Field` .
6. `frontend/components/brand/Logo.tsx` and `GameCover.tsx` .
7. `frontend/components/arcade/GameHud.tsx` , then `ArcadeFrame.tsx` .
8. `frontend/app/stats/page.tsx` , a whole screen built from these parts.

CHECK YOURSELF

Why can a page write `btn btn-ink py-3.5` and get the larger padding?

Because `.btn` is declared in `@layer components` , which Tailwind emits before the utilities layer. The `py-3.5` utility comes later in the cascade with the same specificity, so it wins.

Where does `font-serif` get its typeface, and what happens if the font fails to load?

`layout.tsx` loads Instrument Serif through `next/font/google` as the variable `--font-serif` on `<html>` ; Tailwind's `font-serif` reads that variable and falls back to Georgia, then the generic serif.

You add a seventh game and its card shows a lime circle on black. Why?

`GameCover` found no entry for the new id in `COVERS` and used `FALLBACK` . Add cover art keyed by the game's id.

A page passes a helpful panel as `aside` . A tester on an iPad says it is missing. What is going on?

The aside column is `hidden xl:block` , so it only renders at 1280 px and wider. Render the same content in the main column with `xl:hidden` , as the lesson page does with its videos.

Why is the app's background a gradient rather than flat paper?

Glass panels blur what is behind them. Over a flat color the blur has nothing to work on and the panel looks like a grey box; the `.paper-blobs` gradients give it soft texture to frost.

PART TWO

The features

One chapter per feature area. Each follows the same shape: intent, files, model, the server side, the client side, how it works end to end, edge cases, and study notes, so you can pick the part of the app you want to work on and start there.

- 5. Accounts and profiles
- 6. Exams, courses and practice settings
- 7. The question engine
- 8. The game framework
- 9. The games, and adding one
- 10. Learn: lessons and videos
- 11. Dashboard, statistics and sessions
- 12. Mock exams and your own material

5 Accounts and profiles

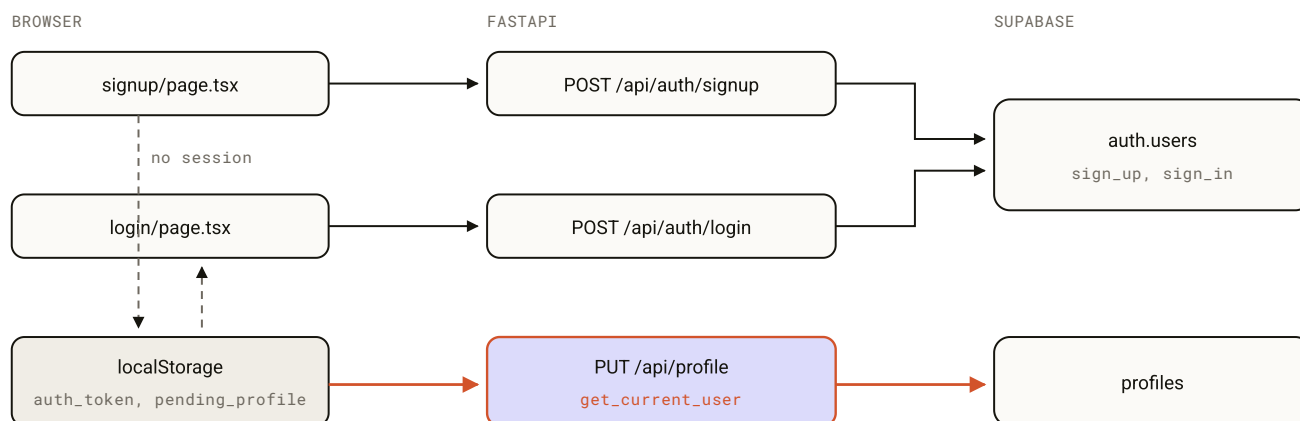
An account in SATistics is two things: a Supabase Auth user, which owns the email and password, and a row in `profiles`, which holds the name, the exam and the goal that every other screen reads. The browser never talks to Supabase directly. It talks to the FastAPI backend, which signs people up and in through Supabase, hands back the access token, and later reads the user id out of that token on every request. This chapter follows that path end to end and is candid about where it is thinner than it looks.

5.1 Intent

Signup should feel like setting up a game, not filling in a form. So it is three short steps (account, profile, goal) and the profile is created in the same flow, which means the dashboard can greet someone by name and aim their first game at their exam the moment they arrive.

The second intent is that a broken step never loses work. If the account is created but the profile save fails, signup does not fail; it sends the person to the profile page with a notice. If Supabase requires email confirmation, there is no session yet to save the profile with, so the profile waits in `localStorage` and is saved on the first login.

The third is that validation is the same on both sides. The form checks the profile before it sends it, using rules that mirror the backend's Pydantic model, so most errors show inline without a round trip, and the backend still enforces them.



Signup and login both go through FastAPI to Supabase Auth. The profile is a separate authenticated call, made right after signup when a session exists, or after the first login when it does not.

5.2 Files and folders

frontend/app/signup/page.tsx	three-step signup, awaiting-confirmation screen
frontend/app/login/page.tsx	login, saves a pending profile, honors ?redirect=
frontend/app/profile/page.tsx	edit profile, first-run setup (?setup=1¬ice=)
frontend/components/auth/AuthShell.tsx	signed-out layout, Field, FormError
frontend/components/profile/ProfileFields.tsx	Avatar, IdentityFields, GoalFields, validateProfi
frontend/lib/profile.ts	Profile type, constants, pending-profile storage
frontend/lib/useAccount.ts	shared account cache and hook
frontend/lib/api/client.ts	ApiClient: token storage, auth and profile calls
frontend/middleware.ts	redirects based on the auth_token cookie
backend/src/api/auth.py	/api/auth routes and get_current_user
backend/src/services/auth_service.py	AuthService: Supabase sign_up, sign_in, token decode
backend/src/api/profile.py	GET and PUT /api/profile
backend/src/models/schemas.py	UserSignup, UserLogin, TokenResponse, Profile
backend/src/utils/database.py	the shared Supabase client
backend/database/add_profiles.sql	the profiles table and its policies

5.3 Model

`backend/database/add_profiles.sql` creates `profiles` with `user_id` as the primary key, referencing `auth.users(id)` with `ON DELETE CASCADE`. One user has at most one profile, and deleting the auth user removes it.

COLUMN	TYPE	RULE
<code>display_name</code>	text, not null	1 to 60 characters (Pydantic)
<code>username</code>	text, not null, unique	<code>^[a-z0-9_]{3,24}\$</code>
<code>exam</code>	text, default <code>'sat'</code>	check constraint: <code>sat</code> or <code>gre</code>
<code>target_score</code>	integer, nullable	400 to 1600 for SAT, 260 to 340 for GRE
<code>test_date</code>	date, nullable	ISO date
<code>daily_minutes</code>	integer, default 20	5 to 240 on the server; the UI offers 10, 20, 30, 45
<code>avatar_color</code>	text, default <code>'#3A33E8'</code>	any six-digit hex on the server; five swatches in the UI
<code>created_at</code> , <code>updated_at</code>	timestampz	the API sets <code>updated_at</code> on each save

Row level security is on, with select, insert and update policies keyed on `auth.uid() = user_id`. There is no delete policy. In practice the backend usually connects with the service role key (section 5.4), which bypasses these policies, so the `.eq("user_id", ...)` filters in `profile.py` are what actually scope the queries.

On the client, `frontend/lib/profile.ts` declares the matching `Profile` interface, `EMPTY_PROFILE` (SAT, 20 minutes, cobalt avatar), `AVATAR_COLORS` (cobalt, lime, coral, pink, ink), `DAILY_MINUTES` and `SCORE_RANGES`, which carries each exam's min, max, step and label. It also has small helpers: `initials`, `daysUntil` (a whole-day count to a `yyyy-mm-dd` date at local midnight, or `null`) and `avatarTextColor`.

5.4 The server side

Routes. `backend/src/api/auth.py` is mounted at `/api/auth` and defines `POST /signup`, `POST /login`, `POST /logout` and `GET /me`, plus explicit `OPTIONS` handlers for signup and login. `backend/src/api/profile.py` is mounted at `/api/profile` with `GET` and `PUT` on the empty path.

Signup. The body is `UserSignup` (an `EmailStr` and a password string). `AuthService.signup` calls `db.auth.sign_up`. If Supabase returns a user, the result has `success`, the user's id and email, and, only when Supabase also returned a session (email confirmation off), an `access_token`. Known Supabase errors are rewritten into friendlier text ("An account with this email already exists", "Too many signup attempts. Please try again later.") and the route turns an unsuccessful result into a 400. Any other exception becomes a 500 whose detail includes the exception text.

Login. `AuthService.login` calls `sign_in_with_password` and returns the access token, a refresh token and the user. The route wraps this in `TokenResponse`, which has `access_token`, `token_type` and `user`, so the refresh token is dropped. Failures are 401 with "Invalid email or password" or the raw message.

Logout. `AuthService.logout` does nothing but return success. Its comments are explicit: the token is not revoked, it simply expires, and the client is trusted to forget it.

Who is calling. Every protected route depends on `get_current_user`. It reads the bearer token through `HTTPBearer(auto_error=False)`, returns 401 "Missing authorization token" if there is none, and otherwise calls `AuthService.get_user`. That method decodes the JWT with `jwt.decode(token, options={"verify_signature": False})`, rejects it if `exp` is in the past, and returns `{"id": sub, "email": email}`. A `None` result becomes 401 "Invalid or expired token". `GET /me` returns exactly that dictionary.

SECURITY CAVEAT: TOKENS ARE NOT VERIFIED.

Because the signature is never checked, anyone can construct a JWT with any `sub` and a future `exp`, and the backend will treat them as that user. Every route that uses `get_current_user` (profiles, sessions, stats, materials, videos) inherits this. The fix is to verify the token, either with the project's JWT secret (`jwt.decode` with the key, `algorithms=["HS256"]` and `audience="authenticated"`) or by calling `db.auth.get_user(token)`, which asks Supabase. Note that `PyJWT`, which provides the `jwt` module here, is not listed in `backend/requirements.txt`; it arrives as a dependency of the Supabase package. `python-jose` is listed but not used by this code.

The database client. `get_db` returns one process-wide client from `Database.get_client()`, built with `SUPABASE_SERVICE_ROLE_KEY` if set, then `SUPABASE_SERVICE_KEY`, then `NEXT_PUBLIC_SUPABASE_ANON_KEY`. The same client is used for `auth.sign_up` and `auth.sign_in_with_password`. The supabase-py 2.x client listens for its own auth events, and on `SIGNED_IN` it replaces its `Authorization` header with the new session's access token. On a shared server-side client that means table queries after a login run as the last person who logged in, not with the service key, until the process restarts. If you see RLS errors or empty results on a long-lived backend process, look here first; the safe pattern is a separate client for auth calls. `Database.get_auth_client()`, which builds a separate client with the anon key, exists but nothing calls it.

Profiles. `GET /api/profile` selects the seven profile fields for the caller and returns `{"profile": ...}` or `{"profile": null}`. `PUT` takes a `Profile` body, which Pydantic validates: the field constraints in the table above plus `check_target_score`, a model validator that applies the per-exam score range. The handler first looks for another user with the same username and returns 409 "The username @name is taken. Try another one." if there is one. Then it adds `user_id` and `updated_at` and upserts on `user_id`, returning the saved row.

5.5 The client

Token storage. `ApiClient` in `frontend/lib/api/client.ts` is a singleton, `apiClient`. On construction in the browser it reads `auth_token` from `localStorage`, or, failing that, from `document.cookie` and copies it back. `setToken(token)` writes both: `localStorage`, and a cookie `auth_token=...; path=/; max-age=604800; SameSite=Lax`, which is seven days. `setToken(null)` removes both. The cookie exists only so the Next.js middleware can see it; API calls send the token as an `Authorization: Bearer` header from memory. `signup` and `login` call `setToken` when a token comes back, and `logout` clears it in a `finally` so a failed request still logs you out locally.

Errors from `request` become `Error(detail)`, using the JSON `detail` field from FastAPI. That is why the 409 username message can be shown verbatim in the form.

Route protection. `frontend/middleware.ts` runs on every path except static assets, image files and anything under `api`. If the path starts with one of `/dashboard`, `/games`, `/learn`, `/materials`, `/mock`, `/profile` or `/stats` and there is no `auth_token` cookie (or `Authorization` header), it redirects to `/login?redirect=<path>`. If the path is `/login` or `/signup` and there is a token, it redirects to `/dashboard`. It checks only that a token is present, not that it is valid.

The account cache. `frontend/lib/useAccount.ts` keeps one module-level `cached` account, one `inflight` promise and a set of listeners. `useAccount()` subscribes the component, starts `loadAccount()` if nothing is cached (sharing the in-flight promise if one exists), and returns `{ account, loading, updateProfile }`. `loadAccount` calls `getCurrentUser()` and, if that succeeds, `getProfile()`; either failing yields `null` rather than an error. `setCachedProfile` (exposed as `updateProfile`) patches the cached profile and notifies every listener, so the sidebar updates the moment the profile page saves. `clearAccountCache()` is called on `signup`, `login` and `logout` so the next page loads fresh.

Shared fields. `frontend/components/profile/ProfileFields.tsx` is used by both `signup` and the profile page. `IdentityFields` renders the `Avatar` preview, five color swatches, the name and the username. While the username still equals `slugifyUsername` of the current name, typing a name keeps suggesting a matching username; once the person edits the username themselves, it stops following. The username input lowercases as you type. `GoalFields` renders the exam choice (switching exam clears the target score, since the ranges differ), a number input whose min, max and step come from `SCORE_RANGES`, an optional date, and the daily minutes chips. `validateProfile` returns the first problem as a sentence, or `null`.

5.6 How it works

Signup. `frontend/app/signup/page.tsx` holds `step` (0, 1 or 2 over `STEPS = ['Account', 'Profile', 'Goal']`), the email, two passwords and a `Profile` draft seeded from `EMPTY_PROFILE`, with the exam taken from `?exam=gre` if present. A progress bar of three segments marks the current one with `aria-current="step"`. Each submit runs `checkStep`: step 0 needs a password of at least 6 characters and a

matching confirmation; steps 1 and 2 run `validateProfile` on the whole draft. Browser validation applies on step 0 (so the email must look like one) and is turned off with `noValidate` after that. The back button clears the error and steps back.

On the last step, `createAccount` calls `apiClient.signup`, clears the account cache and stores the chosen exam in the practice preferences with `setExamPrefs`. Then it branches:

- **No access token** (confirmation is on): `savePendingProfile(email, profile)` and switch to the "Check your inbox" screen, which links to login.
- **Token, profile saves**: push to `/dashboard`.
- **Token, profile fails** (for example a taken username): push to `/profile?setup=1¬ice=<message>`. The account exists; only the profile is left to do.

The pending profile. `savePendingProfile` writes `{ email, profile }` to `localStorage` under `pending_profile`, with the email lowercased. `takePendingProfile(email)` returns it only if the emails match, deletes it as it reads, and merges it over `EMPTY_PROFILE` so a profile saved by an older version still has every field. Both swallow storage errors. Because the data lives in one browser, someone who confirms on their phone and logs in on a laptop starts without it and is sent to set up a profile.

Login. `frontend/app/login/page.tsx` calls `apiClient.login`, throws if no token came back, clears the cache, and then calls `takePendingProfile`. If a pending profile exists it is saved; a failure there goes to the setup page with a notice, as signup does. Finally it pushes to `?redirect` if it is a same-site path (it must start with one `/` and not `//` or `/\`), else `/dashboard`, and refreshes the router so server components and middleware see the new cookie.

The profile page. `frontend/app/profile/page.tsx` renders inside `DashboardLayout`. When the account arrives it seeds a draft from the saved profile or, if there is none, from the email's local part (as the name and, slugified, the username). A cobalt header card previews the avatar, name, exam, target and test-day countdown live as you edit. Saving validates locally, calls `saveProfile`, pushes the result into the account cache with `updateProfile`, and syncs the exam preference. In setup mode (`?setup=1`) the page shows a lime banner with any `notice`, the button reads "Save and start", and success goes to the dashboard; otherwise "Profile saved" appears in a polite live region. Below the form, the strongest and weakest topics from `getUserStats()` are listed as chips.

Logout. The sidebar's "Log out" in `DashboardLayout` calls `apiClient.logout()`, then clears the cache and pushes to `/login` in a `finally`.

5.7 Edge cases and tests

There are no automated tests for this area, in either the frontend or the backend. These are the behaviors worth knowing before you change anything.

- **Expired sessions.** Supabase access tokens are short-lived, but the cookie lasts seven days and no refresh token is kept. After expiry the middleware still lets the person in, every API call returns 401, and nothing in the client redirects on 401. `useAccount` resolves to no user, so the sidebar shows an empty name; pages that read stats show empty states. Logging out and in again fixes it.
- **Validation drift.** The backend accepts a `display_name` of a single space (only `min_length=1`), any `daily_minutes` from 5 to 240 and any hex color. The client is stricter: it trims the name and offers only

the listed values. If you add a client option, check the server range.

- **Pydantic errors read badly.** A 422 from FastAPI has a list in `detail`, and `request` passes it to `new Error`, which stringifies it. The client-side checks catch the common cases first, which is why this is rarely seen.
- **Username race.** The 409 check and the upsert are two queries. Two people claiming the same name at once both pass the check; the unique constraint rejects the second, which surfaces as a 500 with the database message.
- **Redirect parameter.** Login follows `?redirect` only when it is a same-site path, so a crafted link cannot send someone to another site after they sign in.
- **Token exposure.** The token is in `localStorage` and in a cookie set from JavaScript, so it is not `HttpOnly`; any script running on the page can read it.
- **Preflight handlers.** The explicit `OPTIONS` routes for signup and login echo back whatever `Origin` they receive with credentials allowed, which is looser than the `ALLOWED_ORIGINS` list given to `CORSMiddleware` in `backend/src/main.py`.

To test by hand, run Supabase with email confirmation both on and off. With it off, signup should land on the dashboard with the name in the sidebar. With it on, signup should show "Check your inbox", `pending_profile` should appear in `localStorage`, and the first login should remove it and land on the dashboard with the profile in place. Try a taken username in both modes; you should arrive at `/profile?setup=1` with the 409 message in the banner.

5.8 Study notes

READ IN THIS ORDER

1. `backend/database/add_profiles.sql` , the table and its policies.
2. `backend/src/models/schemas.py` , the `Profile` model and its validator.
3. `backend/src/services/auth_service.py` , especially `get_user` .
4. `backend/src/api/auth.py` , then `backend/src/api/profile.py` .
5. `frontend/lib/api/client.ts` , the constructor, `setToken` and `request` .
6. `frontend/middleware.ts` .
7. `frontend/lib/profile.ts` and `frontend/lib/useAccount.ts` .
8. `frontend/app/signup/page.tsx` , then `login/page.tsx` , then `profile/page.tsx` .
9. `frontend/components/profile/ProfileFields.tsx` .

CHECK YOURSELF

Why does the token live in both `localStorage` and a cookie?

The API client sends it as a bearer header, which needs it in JavaScript, and the middleware runs on the edge before any JavaScript, where only cookies are visible. The cookie is there purely so `middleware.ts` can decide whether to redirect.

Signup succeeds but no token comes back. What happens to the profile the person just filled in?

It is written to `localStorage` under `pending_profile` with the lowercased email. On the next login with the same email, `takePendingProfile` returns and deletes it, and the login page saves it with `PUT /api/profile` .

How does the backend know who is calling, and what does it not check?

`get_current_user` decodes the bearer JWT and takes `sub` as the user id after checking `exp` . It does not verify the signature, so a forged token is accepted.

Why does the sidebar show the new name immediately after saving the profile?

The profile page calls `updateProfile` , which is `setCachedProfile` : it replaces the profile in the module-level cache and calls every subscribed `useAccount` listener, including the one in `DashboardLayout` .

Why does changing the exam clear the target score?

The ranges do not overlap: 400 to 1600 for the SAT and 260 to 340 for the GRE. Keeping the old number would make the draft invalid on both sides.

Signup with a taken username: does the account get created?

Yes. The auth user is created first; the profile save then fails with 409, and signup routes to `/profile?setup=1` with the message as the notice instead of failing.

6 Exams, courses and practice settings

Every question in SATistics belongs to one exam and one section. That pair, plus a question count, is the only setting a player chooses before a game. This chapter covers where the pair is defined, how it is stored, how it reaches the games and the backend, and what a question looks like once it arrives. It ends with a checklist for adding an exam or a section.

6.1 Intent

The product trains for two tests, the Digital SAT and the GRE, and each has a quantitative and a verbal section. Rather than build a separate game per test, the app keeps one small model of "what am I practising" and threads it through everything: the game setup screen, the question loader, the timer, the offline bank, the learning hub's courses and the mock exam. A game never knows which exam it is running; it receives questions and a number of seconds per question.

The model is deliberately thin. It is a pair of string ids, a question count and a table of real-exam pacing. There is no database row for it: the choice lives in the browser's `localStorage`, under one key, and the backend is told the exam and section as query parameters on every question request. The profile's `exam` field, covered with profiles, is a separate thing: it sets the target score scale and is not read by the question loader.

6.2 Files and folders

<code>frontend/lib/exam.ts</code>	<code>ExamId, SectionId, Pace, ExamPrefs, EXAMS, QUESTION_COUNTS, QUICK_SECONDS_PER_QUESTION, gamePace, getExamPrefs, setExamPrefs, sectionLabel</code>
<code>frontend/lib/courses.ts</code>	<code>Course, COURSES (official topic domains), courseFor</code>
<code>frontend/lib/api/questions.ts</code>	<code>SATQuestion, fetchAIQuestions, fetchQuestionsWithCache</code>
<code>frontend/lib/questionBank.ts</code>	<code>offline bank, keyed by exam and section (chapter 7)</code>
<code>frontend/app/games/[gameId]/select/page.tsx</code>	<code>the game setup screen</code>
<code>frontend/app/mock/page.tsx</code>	<code>MODULE_QUESTIONS, SCORE_RANGE (chapter 12)</code>
<code>backend/src/services/question_sources.py</code>	<code>EXAMS (names, option_count), section_label</code>
<code>backend/src/models/schemas.py</code>	<code>Question, QuestionResponse</code>
<code>backend/src/api/questions.py</code>	<code>exam and section as Literal query params</code>
<code>backend/src/api/materials.py</code>	<code>exam and section as Literal form fields</code>
<code>backend/database/add_materials.sql</code>	<code>CHECK (exam IN ('sat', 'gre'))</code>

The frontend and backend each keep their own `EXAMS` table. They are not generated from one another and they hold different facts: the frontend's has display names and pacing, the backend's has the names used in prompts and search queries and the number of answer choices.

6.3 Model

EXAM, SECTION AND PACE

```
// frontend/lib/exam.ts
export type ExamId = 'sat' | 'gre'
export type SectionId = 'quant' | 'verbal'
export type Pace = 'quick' | 'deep'

export interface ExamPrefs {
  exam: ExamId
  section: SectionId
  questionCount: number
}
```

`SectionId` uses the GRE's vocabulary for both tests. On the SAT, `quant` means Math and `verbal` means Reading and Writing. The display name comes from `EXAMS`, never from the id.

`Pace` is not a player setting. It is a property of the game: `gamePace` returns `'deep'` for any id in `DEEP_GAMES`, which today holds only `'squid-game'`, and `'quick'` for everything else. Quick games want short questions that can be read while something is moving on screen; the deep game wants passages and multi-step problems. Chapter 7 shows how each question source honours it.

THE PACING TABLE

EXAM	SECTION ID	NAME	PACING	SECONDS PER QUESTION
SAT	<code>quant</code>	Math	22 questions in 35 min	95
SAT	<code>verbal</code>	Reading & Writing	27 questions in 32 min	71
GRE	<code>quant</code>	Quantitative	12 questions in 21 min	105
GRE	<code>verbal</code>	Verbal	12 questions in 18 min	90

`secondsPerQuestion` is the official section length divided by its question count, rounded down (35 minutes over 22 questions is 95.45 seconds). It is used where the app wants the real exam's pace: the deep game's per-question timer and the mock exam's single clock. Quick games ignore it and use `QUICK_SECONDS_PER_QUESTION`, which is 35.

`QUESTION_COUNTS` is `[6, 10, 20]`, the three choices on the setup screen. The default preferences are `{ exam: 'sat', section: 'quant', questionCount: 10 }`.

STORAGE

`getExamPrefs` reads the JSON under the `localStorage` key `exam_prefs` and validates each field on the way out: an exam that is not a key of `EXAMS`, a section that is neither `'quant'` nor `'verbal'`, or a count not in `QUESTION_COUNTS` falls back to the default for that field alone. On the server (`typeof window === 'undefined'`) or on a parse error it returns the defaults. `setExamPrefs` writes the whole object and

swallows storage errors. Neither function touches the backend, so preferences are per browser, not per account.

COURSES AND OFFICIAL DOMAINS

```
// frontend/lib/courses.ts
export interface Course {
  exam: ExamId
  section: SectionId
  title: string
  summary: string
  topics: string[] // official content domains
}
```

COURSE	TOPICS
SAT Math	Algebra, Advanced Math, Problem-Solving and Data Analysis, Geometry and Trigonometry
SAT Reading & Writing	Information and Ideas, Craft and Structure, Expression of Ideas, Standard English Conventions
GRE Quant	Arithmetic, Algebra, Geometry, Data Analysis
GRE Verbal	Text Completion, Sentence Equivalence, Reading Comprehension

The SAT topic strings are the College Board's own domain names. They matter because the official question bank reports a question's topic as its `primary_class_cd_desc`, which uses the same strings, so topic statistics from games line up with course topics without a mapping table. The offline bank uses the same names for its `topic` fields. `courseFor(exam, section)` finds the matching course and falls back to `COURSES[0]`, SAT Math, if none matches. Besides the setup screen, the landing page, the dashboard, the learning hub and `frontend/lib/lessons.ts` import this file.

A QUESTION

The question shape is defined twice, once per language, and the two are kept in step by hand.

FIELD	TYPE	MEANING
<code>id</code>	int	Stable per source, see chapter 7 for how each source makes it
<code>question</code>	str	Full plain text, passage first if there is one
<code>options</code>	list of str	Plain-text choices, 2 to 5
<code>correctAnswer</code>	int	0-based index into <code>options</code>
<code>topic</code> , <code>difficulty</code> , <code>explanation</code>	str	Difficulty is <code>easy</code> , <code>medium</code> or <code>hard</code>
<code>exam</code> , <code>section</code> , <code>skill</code>	optional str	Skill is the fine-grained College Board skill
<code>passage</code> , <code>stem</code>	optional str	The passage and the question without it
<code>questionHtml</code> , <code>passageHtml</code> , <code>optionsHtml</code> , <code>explanationHtml</code>	optional	Sanitized HTML and MathML, only on official questions
<code>source</code> , <code>sourceName</code> , <code>sourceUrl</code>	optional str	Where the question came from

The plain and rich forms exist side by side because the games need both. Canvas games draw text themselves and can only use `question` and `options`. The React overlays and the mock exam render HTML through `QuestionContent` and fall back to the plain text when no HTML is present. `question` always carries the passage too, so a game that shows only one string still shows everything; renderers that have room use `passage` and `stem` separately.

`source` takes five values in practice: `official` (College Board), `web` (copied from a practice page), `ai` (written by the model), `custom` (the user's own material, chapter 12) and `practice` (the offline bank, set only in the browser). The comment in `schemas.py` lists three of them and the comment in `SATQuestion` lists four; neither mentions `custom`. The field is a free string in both, so nothing breaks.

6.4 The server side

```
# backend/src/services/question_sources.py
EXAMS = {
    "sat": {
        "name": "SAT",
        "sections": {
            "quant": {"name": "Math", "option_count": 4},
            "verbal": {"name": "Reading and Writing", "option_count": 4},
        },
    },
    "gre": {
        "name": "GRE",
        "sections": {
            "quant": {"name": "Quantitative Reasoning", "option_count": 5},
            "verbal": {"name": "Verbal Reasoning", "option_count": 5},
        },
    },
}
```

`section_label(exam, section)` joins the two names, giving strings like `"GRE Quantitative Reasoning"`. That label goes into the LLM prompts, the DuckDuckGo queries and the material parser's prompt. `option_count` is only read by the agent's prompt ("Single-answer multiple choice with 5 options"). The lesson-video search in `backend/src/services/videos.py` also reads the names from this table.

The names differ from the frontend's on purpose: the backend uses the official long names because they make better search queries. Do not "fix" one to match the other without checking what reads it.

Validation happens at the edges. `GET /api/questions` declares `exam: Literal["sat", "gre"]` and `section: Literal["quant", "verbal"]`, so FastAPI rejects anything else with a 422 before any code runs. `POST /api/materials` does the same with form fields, and `GET /api/learn/videos` uses a regex pattern `^(sat|gre)$`. In the database, `materials.exam` and `profiles.exam` carry a `CHECK (exam IN ('sat', 'gre'))`. The response model is `QuestionResponse`: a list of `Question` and a `total`.

6.5 The client

The setup screen at `/games/[gameId]/select` is the only place a player edits the preferences for games. It is a client component. Because preferences live in `localStorage`, it renders a pulsing placeholder on first paint and loads them in a `useEffect`, which avoids a hydration mismatch between the server render (defaults) and the browser (saved values).

It shows three rows of toggle buttons: exam (from `Object.keys(EXAMS)`), section (from the chosen exam's `sections`) and question count (from `QUESTION_COUNTS`), each with `aria-pressed`. Under the counts it prints the real exam's `pacing` string, and under that the topic chips from `courseFor(prefs.exam, prefs.section).topics`. Every click goes through `updatePrefs`, which merges the change, sets state and calls `setExamPrefs` at once. There is no save button. "Start game" is a plain link to `/games/[gameId]`; it carries no parameters, because the game page reads the preferences itself.

An unknown `gameId` (not in `games` from `frontend/lib/games.ts`) renders a "That game doesn't exist" card with a link back to `/games`.

WHAT EACH GAME DOES WITH THE PREFERENCES

CONTAINER	ASKS FOR	PACE	TIMER
<code>ZombieGameContainer</code>	<code>questionCount</code>	quick	<code>QUICK_SECONDS_PER_QUESTION</code>
<code>SubwaySurfersGameContainer</code>	<code>questionCount * 2</code>	quick	<code>QUICK_SECONDS_PER_QUESTION</code>
<code>SquidGameContainer</code>	<code>questionCount * 2</code>	deep	section's <code>secondsPerQuestion</code>
<code>PacManGameContainer</code>	20	quick	<code>QUICK_SECONDS_PER_QUESTION</code>
<code>CarnivalGameContainer</code>	50, cached	quick	game's own
<code>WhackAMoleGameContainer</code>	50, cached	quick	<code>QUICK_SECONDS_PER_QUESTION</code> pop time

Subway Surfers and Squid Game fetch twice the count so their difficulty can adapt as the run goes, then cap the run at `questionCount`. Pac-Man, Carnival and Whack-a-Mole ignore the count setting. Every container reads `getExamPrefs()` on mount and the loader in `frontend/lib/api/questions.ts` reads it again to build the request, so the exam and section always come from storage, never from props.

6.6 How it works

1. The player opens `/games/squid-game/select`. The effect calls `getExamPrefs()`; say it returns GRE, verbal, 10.
2. The screen shows "Real exam pace: 12 questions in 18 min" and the chips Text Completion, Sentence Equivalence, Reading Comprehension.
3. The player taps 20. `setExamPrefs` writes `{"exam": "gre", "section": "verbal", "questionCount": 20}` to `exam_prefs`.
4. "Start game" navigates to `/games/squid-game`. `SquidGameContainer` reads the preferences again and calls `fetchAIQuestions(40, undefined, 'deep')`.
5. `fetchAIQuestions` reads the preferences a third time and requests `/api/questions/?use_agent=true&limit=40&use_web_search=true&exam=gre§ion=verbal&pace=deep`.
6. The questions come back (chapter 7). The container builds the game with `questionCount: 20` and `secondsPerQuestion: 90`, from `EXAMS.gre.sections.verbal`.

The mock exam (chapter 12) has its own exam and section picker, and before it loads questions it calls `setExamPrefs({ ...getExamPrefs(), exam, section })`, because the loader only reads the section from storage. A side effect is that choosing a mock exam also changes the section the next game will use.

6.7 Adding an exam or a section

There is no single registry, so a new exam touches both halves of the stack. In order:

1. `frontend/lib/exam.ts`: extend `ExamId` (or `SectionId`) and add the entry to `EXAMS` with a real `pace` string and `secondsPerQuestion`. If you add a section, update the hand-written check in

- `getExamPrefs`, which accepts only `'verbal'` and `'quant'`.
2. `frontend/lib/courses.ts`: add a `Course` per section, with the official domain names as `topics`.
 3. `frontend/lib/questionBank.ts`: add a bank array per section to `BANK`. The type is `Record<ExamId, Record<SectionId, ...>>`, so TypeScript will refuse to compile until you do. Include all three difficulties so both paces have something left after filtering.
 4. `frontend/app/mock/page.tsx`: `MODULE_QUESTIONS` and `SCORE_RANGE` are also keyed by `ExamId`. The rounding step and the source blurb test `exam === 'sat'`.
 5. `frontend/lib/profile.ts`: `SCORE_RANGES`, and the copy in `frontend/components/profile/ProfileFields.tsx`.
 6. `backend/src/services/question_sources.py`: add the entry to `EXAMS` with names suitable for search and an `option_count`.
 7. `backend/src/api/questions.py` and `backend/src/api/materials.py`: widen the `Literal` types. `backend/src/api/learn.py`: widen the regex. `backend/src/models/schemas.py`: widen `Profile.exam` and its target-score validator.
 8. SQL: the `CHECK` constraints in `add_materials.sql`, `add_profiles.sql` and `reset.sql` need a new migration that drops and recreates them.
 9. `backend/src/services/agent.py`: only SAT has an official source (`if exam == "sat"`). A new exam gets web and AI questions only, so it needs `OPENROUTER_API_KEY` set, or it falls through to the offline bank.

CHECK WITH GREP.

Before opening a pull request, search both trees for `'sat'`, `"gre"`, `Record<ExamId` and `Literal["sat"]`. Every hit is a place that enumerates exams.

6.8 Edge cases and tests

- **Private windows.** If `localStorage` throws, the getter returns defaults and the setter is a no-op, so the setup screen works but forgets every change.
- **Stale values.** A stored count of 15 from an older version falls back to 10 without touching the stored exam or section.
- **Two tabs.** There is no `storage` event listener. A tab that is already open keeps the preferences it read on mount.
- **Name drift.** The frontend says "Reading & Writing" and "Quantitative"; the backend says "Reading and Writing" and "Quantitative Reasoning". This is harmless but it means a label from one side should never be compared to the other.
- **Profile versus practice.** A user whose profile says GRE can practise SAT. Nothing reconciles the two.

There are no automated tests for this code. The frontend `package.json` has no test script and the backend has no test directory. The validation in `getExamPrefs` and the hand-kept parity between `SATQuestion` and `Question` are the first things worth pinning.

6.9 Study notes

READ IN THIS ORDER

1. `frontend/lib/exam.ts` , all 79 lines.
2. `frontend/lib/courses.ts` .
3. `frontend/app/games/[gameId]/select/page.tsx` , `updatePrefs` first.
4. `frontend/lib/api/questions.ts` , the `SATQuestion` interface.
5. `backend/src/models/schemas.py` , `Question` and `QuestionResponse` .
6. `backend/src/services/question_sources.py` , the top 56 lines.
7. `frontend/components/SquidGameContainer.tsx` , the two effects, to see preferences become game config.

CHECK YOURSELF

Where does the backend learn which exam a player is practising?

From the `exam` and `section` query parameters on each `GET /api/questions` call, which `fetchAIQuestions` fills from `localStorage` . Nothing is stored server side.

Why does the select page render a placeholder before showing the options?

The preferences are only readable in the browser. Loading them in an effect keeps the server render and the first client render identical.

A quick game and the deep game both run GRE Quant. How long does each give per question?

The quick game gives `QUICK_SECONDS_PER_QUESTION` , 35 seconds. Squid Game gives the section's `secondsPerQuestion` , 105 seconds.

Why can SAT topic statistics be matched to course topics without a mapping?

Official questions take their `topic` from the College Board's domain description, and `COURSES` uses those same domain names.

You add a third section. Which frontend function silently drops it if you forget it?

`getExamPrefs` , which only accepts `'verbal'` or `'quant'` and otherwise returns the default section.

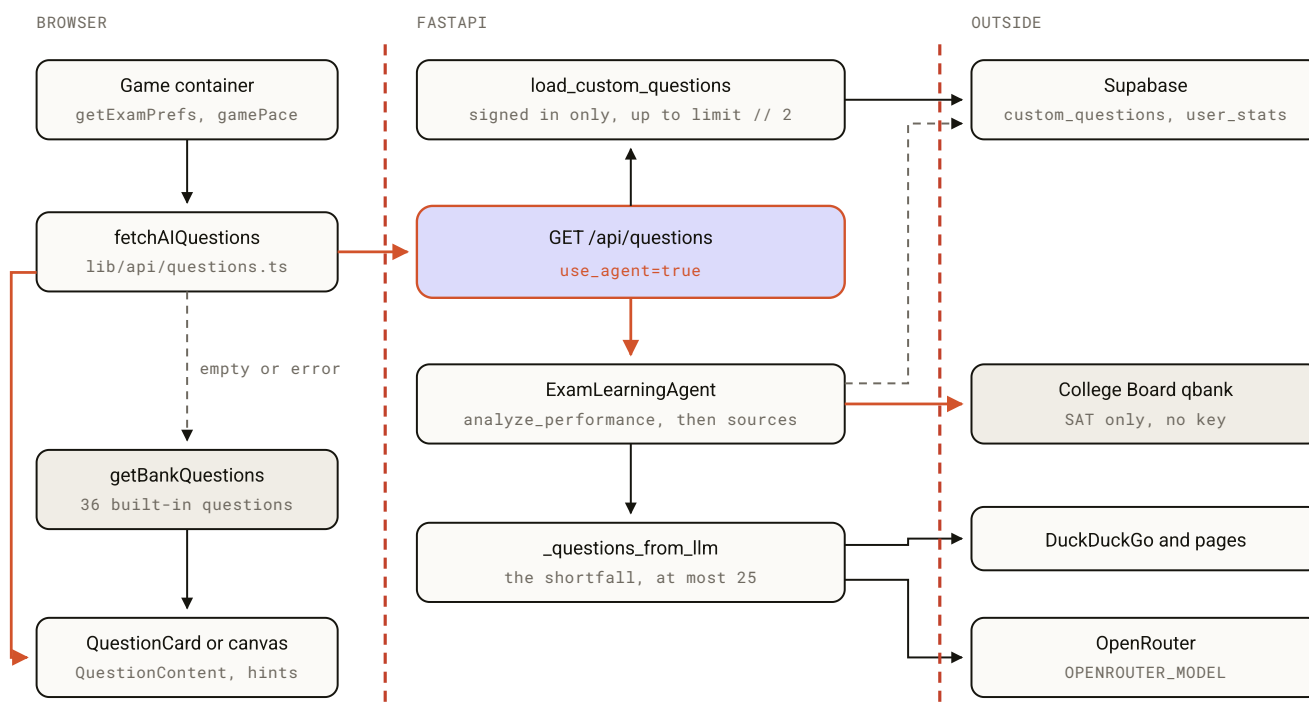
7 The question engine

One endpoint builds every question set the games play. It mixes in the user's own approved questions, pulls official SAT questions from the College Board, asks an LLM to copy questions from web pages or write new ones for whatever is still missing, and tags each question with where it came from. When any of that fails, the browser falls back to a small built-in bank, so a game always starts.

7.1 Intent

The project's promise is real exam questions inside arcade games. "Real" is the hard part. The College Board publishes its Educator Question Bank behind an open JSON API, so SAT questions can be official. ETS does not publish GRE questions for reuse, so for the GRE the engine searches the web for practice pages and asks a model to copy out questions that are actually on them, writing original ones only to fill the gap. Every question carries a `source` so the game can say honestly where it came from.

The engine is also personal. Before it picks anything it reads the user's weak topics from Supabase and leans the selection toward them. And it is game-aware: a fast arcade game asks for `pace=quick` and gets short questions; the slow game asks for `pace=deep` and gets passages and multi-step problems. The exam, section and pace model is chapter 6.



The hot path is a signed-in SAT request: custom questions, then official questions, then the model for anything missing. The dashed path on the left is the offline fallback in the browser. The dashed path on the right is the performance read that picks the weak topics.

7.2 Files and folders

backend/src/api/questions.py	GET /api/questions, GET /api/questions/topics
backend/src/services/agent.py	ExamLearningAgent, get_llm_client, extract_json, LLM_MODEL, MAX_LLM_QUESTIONS, PACE_REQUIREMENTS
backend/src/services/question_sources.py	CollegeBoardSource, WebQuestionSource, is_quick_question, _balance_difficulty, normalize_text
backend/src/services/custom_questions.py	load_custom_questions, to_game_question
backend/src/services/supabase_agent_ops.py	get_user_performance, get_topic_performance
backend/src/utils/html_utils.py	sanitize_html, html_to_text, has_figure
frontend/lib/api/questions.ts	SATQuestion, fetchAIQuestions, fetchQuestionsWithCache
frontend/lib/api/client.ts	apiClient.getAIQuestions
frontend/lib/questionBank.ts	the offline bank, getBankQuestions
frontend/components/QuestionContent.tsx	renders HTML/MathML or plain text
frontend/components/exam/QuestionCard.tsx	the in-game question panel, sourceLabel
frontend/lib/hints.ts	strategyHint, whyNot, insightFor, eliminationHint
frontend/components/exam/HintButton.tsx	
frontend/components/exam/useQuestionHints.ts	

7.3 Model

THE REQUEST

QUERY PARAM	DEFAULT	EFFECT
limit	10	1 to 100
use_agent	false	Must be true to get anything; the client always sends true
use_web_search	true	Lets the LLM step search the web first
exam , section	sat, quant	Literal types, anything else is a 422
pace	none	quick or deep
include_custom	true	Mix in the user's approved questions
topic , difficulty	none	Declared but never read

Auth is optional. `get_current_user_optional` uses `HTTPBearer(auto_error=False)` and returns `None` on a missing or bad token instead of failing, so the endpoint serves guests. A guest is analysed as user `00000000-0000-0000-0000-000000000000`, which has no stats, and gets no custom questions.

SOURCES AND IDS

SOURCE	MADE BY	ID
official	CollegeBoardSource._normalize	int(questionId, 16) & 0x7FFFFFFF
web , ai	_normalize_llm_question	zlib.crc32(stem) & 0x7FFFFFFF
custom	to_game_question	CUSTOM_ID_OFFSET (1,500,000,000) plus the row id
practice	getBankQuestions in the browser	9001 to 9309

Ids matter because game sessions store `questionId` per attempt. The offset is meant to keep custom ids apart from the others, but the masked hash and hex ids range up to 2,147,483,647, so a collision is possible, just unlikely.

7.4 The server side

THE ROUTE

`get_questions` in `backend/src/api/questions.py` does four things when `use_agent` is true. If the caller is signed in and `include_custom` is set, it loads up to `limit // 2` custom questions on a worker thread. It builds an `ExamLearningAgent` and asks it for `limit - len(custom)` more. It validates the combined list through the `Question` model and shuffles it. If the agent raises, it logs and returns the custom questions alone. Any other error becomes a 500.

USE_AGENT=FALSE RETURNS NOTHING.

The docstring and the fallback comment mention static questions, but there is no static bank on the server any more. Without `use_agent=true` the route returns an empty list. The offline bank lives only in the frontend.

The same router serves `GET /api/questions/topics`, which requires auth and returns the distinct `topic` values in `question_attempts`. It does not filter by user, so it lists topics from everyone's attempts. Nothing in the frontend calls it.

THE AGENT

`ExamLearningAgent.generate_questions` runs `analyze_performance` on a thread. That reads the `user_stats` row through `SupabaseAgentOps.get_user_performance` (total answered, accuracy, weak and strong topics) and aggregates `question_attempts` per topic through `get_topic_performance`. It sets `recommended_difficulty` to easy below 50 percent accuracy, hard above 75, medium otherwise. Both reads catch their own exceptions and return empty data, so a missing database degrades to "new user".

For the SAT it then calls `CollegeBoardSource.fetch`. Whatever is still missing, for any exam, goes to `_questions_from_llm`. The agent appends a summary to `context_memory`, a plain list on the instance; a new agent is built per request, so that memory never outlives the request.

COLLEGEBOARDSOURCE

The source talks to `BASE_URL`, the College Board's `qbank-api` host, in two steps. `_get_index` posts to `/get-questions` with `asmtEventId: 99`, the test number and the domain codes from `SECTIONS` (Math is test 2, domains `H, P, Q, S`; Reading and Writing is test 1, domains `INI, CAS, EOI, SEC`). It keeps only entries with an `external_id` and caches the index per section for `INDEX_TTL_SECONDS`, six hours, in a class attribute.

1. `_fits_pace` pre-filters on index metadata. Quick drops hard questions and, in verbal, keeps only the `QUICK_SKILLS` (Words in Context, Boundaries, Form, Structure, and Sense, Transitions). Deep drops easy quant questions and, in verbal, the quick skills.
2. `_pick_candidates` oversamples: five times the limit for quick, four otherwise, because many items are later dropped. It splits the target evenly across the three difficulty codes and fills about half of each band with items whose domain or skill matches a weak topic (`_matches_topic` is a case-insensitive substring test either way round).
3. Each candidate is fetched from `/get-question`, at most `MAX_CONCURRENT_REQUESTS` (8) at once, on one `httpx.AsyncClient` with a 20 second timeout. Failures are logged and skipped.
4. `_normalize` rejects anything that is not `type == "mcq"`, has fewer than two options, or contains a figure (`has_figure`). It finds the correct index from `keys`, or from the letter in `correct_answer`, and builds both the plain text and the sanitized HTML forms.
5. For quick pace, `is_quick_question` drops anything whose full text exceeds `QUICK_MAX_CHARS` (220 for quant, 420 for verbal) or that has an option over `QUICK_MAX_OPTION_CHARS` (60).
6. `_balance_difficulty` takes questions round-robin from the easy, medium and hard bands until it reaches the limit.

THE LLM STEP

`get_llm_client` returns an OpenAI SDK client pointed at `https://openrouter.ai/api/v1`, or `None` when `OPENROUTER_API_KEY` is not set. With no client, `_questions_from_llm` logs and returns an empty list; it does not even search, because a page is useless without a model to read it. The model is `LLM_MODEL`, taken from `OPENROUTER_MODEL` with the default `anthropic/claude-haiku-4.5`. The count is capped at `MAX_LLM_QUESTIONS`, 25, because one call cannot reliably return more.

With `use_web_search`, `WebQuestionSource.find_pages` runs one or two DuckDuckGo text searches through the `ddgs` package: `"{label} practice questions with answers and explanations"`, plus one for the first weak topic. `search` retries up to three times, sleeping 2 and then 4 seconds. Non-PDF result URLs are de-duplicated and the first eight are fetched with browser headers and a 12 second timeout. `_read_page` keeps a page only if its text has a line that starts like a lettered choice (`A.` or `(B)`); it keeps 9,000 characters starting 1,500 before that line. At most `MAX_PAGES`, four, are used.

The prompt is the student context from `build_agent_context` (accuracy, recommended difficulty, weak and strong topics, per-topic accuracy), then a task. With pages, the task is to copy questions from the numbered sources exactly, only where the source gives the answer, and then write original ones for the rest, with `sourceIndex` recording which. Without pages, it is to write them all. The requirements ask for the section's `option_count`, a mix of difficulties, no figures, the pace line from `PACE_REQUIREMENTS`, and passages in a separate field. The call uses temperature 0.4 and `max_tokens=12000`; `extract_json` parses from the first `[` to the last `]`.

`_normalize_llm_question` is where honesty is enforced. A question needs a stem, two to five options and an integer `correctAnswer` in range. It is tagged `web` only if `sourceIndex` points at a real page and the first 50 normalized characters of its stem or passage appear in that page's normalized text; the `sourceName` is then the page's host. Otherwise it is `ai`, "AI-written practice question", whatever the model claimed. Quick pace applies `is_quick_question` again.

SANITIZING HTML

Official questions arrive as HTML with MathML. `backend/src/utils/html_utils.py` parses it into a small tree with the standard library's `HTMLParser` and renders it two ways. `sanitize_html` keeps tags in `HTML_TAGS` and `MATH_TAGS`, keeps only the attributes in `KEPT_ATTRS`, removes `DROPPED_TAGS` (script, style, svg, img, iframe and others) with their content, removes `sr-only` elements, unwraps unknown tags, turns underline spans into `<u>`, and expands `<mfenced>`, which browsers no longer draw, into explicit `<mo>` brackets. `html_to_text` writes MathML out linearly, so a fraction becomes $(12x + 28)/4$ and a square root becomes $\sqrt{...}$, for canvas games and for the LLM. `has_figure` reports any svg, img or figure element.

CUSTOM QUESTIONS

`load_custom_questions` selects the user's `custom_questions` rows for the exam and section with `status = 'approved'`, looks up material names, converts each with `to_game_question` (topic defaults to "My material", source is `custom`, `sourceName` is the material's name), applies the quick filter for quick pace, shuffles and returns up to the limit. Errors return an empty list. How rows get there is chapter 12.

7.5 The client

FETCHING

`fetchAIQuestions(limit, fallbackQuestions, pace)` reads the exam and section from `getExamPrefs()` and calls `apiClient.getAIQuestions`, which always sends `use_agent=true` and `use_web_search=true` and adds the bearer token if the client has one. A non-empty list is returned as is. An empty list is turned into an error, and any error leads to `getBankQuestions(exam, section, pace)`, or to `fallbackQuestions` if the bank had nothing.

`fetchQuestionsWithCache(gameId, limit, fallbackQuestions)` wraps it with a five-minute cache in `sessionStorage` under `ai_questions_${gameId}_${exam}_${section}`, and derives the pace from `gamePace(gameId)`. Only Carnival and Whack-a-Mole use it; the other games call `fetchAIQuestions` directly and get a fresh set every run.

THE OFFLINE BANK

`frontend/lib/questionBank.ts` holds 36 hand-written questions: nine per exam and section, three of each difficulty, ids 9001 to 9309. GRE quant uses the four standard Quantitative Comparison choices from `QC_OPTIONS`; GRE verbal has five-choice items. `getBankQuestions` drops hard questions for quick pace and easy ones for deep pace (`PACE_EXCLUDES`), so a game gets six, prepends any passage to `question`, and tags each one `source: 'practice'`. The landing page's `HeroQuestion` also shows the first three SAT quant items.

RENDERING

`QuestionContent` takes an optional `html` and a required `text`. With HTML it runs a second sanitizer in the browser: it parses with `DOMParser`, removes any element whose tag is not in `ALLOWED_TAGS` (the same list as the backend), strips attributes not in `ALLOWED_ATTRS`, and renders the result with `dangerouslySetInnerHTML`. Without HTML it renders the text with `whitespace-pre-line`. Note the difference: the backend unwraps an unknown tag and keeps its text, the frontend removes it and its text.

`QuestionCard` is the overlay panel the React-rendered games share. It shows "Question n of m", the skill or topic and the difficulty, then the passage, the stem and the choices, each through `QuestionContent`. A choice is a button when the game passes `onPick` and plain text when the player answers inside the game world. After a wrong answer it shows either an "insight" (the player will see the question again, so the right answer stays hidden) or the explanation under "Why X is correct". `sourceLabel` turns `source` into a caption such as "Official College Board question", or "From" followed by the page host or material name; Zombie, Subway Surfers and Squid Game print it.

HINTS

`useQuestionHints(question)` resets when the question changes. The first request shows `strategyHint`: a tip chosen by matching the question's own wording against `PROMPT_TIPS`, then its skill and topic against `STRATEGY_TIPS`, then `DEFAULT_TIP`. Each later request calls `eliminationHint`, which rules out one wrong choice, preferring one the explanation addresses, and stops when only the right answer and one other remain. `whyNot` extracts the "Choice B is incorrect ..." sentence that College Board rationales contain, and rejects it if it is too short, uninformative, or names the correct letter. `insightFor` uses the same function after a wrong answer. `HintButton` is just the button, labelled "Hint" then "Another hint". Zombie, Subway Surfers, Squid Game and Pac-Man use the hook.

7.6 How it works

A signed-in player starts Subway Surfers on SAT Math with ten questions.

1. `SubwaySurfersGameContainer` calls `fetchAIQuestions(20, undefined, 'quick')`.
2. The browser sends `GET /api/questions/?use_agent=true&limit=20&use_web_search=true&exam=sat§ion=quant&pace=quick` with the bearer token.
3. The route resolves the user and loads up to 10 approved custom SAT Math questions. Say it finds 3.
4. The agent reads performance, finds weak topic "Advanced Math", and asks the College Board source for 17.
5. The source filters the cached index to non-hard items, picks 85 candidates (17 times 5) spread over easy and medium with half on Advanced Math, fetches them eight at a time, normalizes, drops long ones and balances. Say 17 survive.
6. Nothing is missing, so no LLM call is made. The route validates 20 questions, shuffles and returns them.
7. The game renders each with `QuestionCard`; the player can ask for hints.

WITH NO API KEYS

SITUATION	WHAT THE PLAYER GETS
SAT, no <code>OPENROUTER_API_KEY</code>	Official questions only. The College Board API needs no key. If it returns too few, the set is short.
GRE, no <code>OPENROUTER_API_KEY</code>	Only approved custom questions. With none, the route returns an empty list and the browser falls back to the bank.
No Supabase configuration	Performance and custom reads catch their errors, so the set is untargeted and has no custom questions.
Backend down or erroring	The bank: six questions per section after the pace filter.

7.7 Performance and cost

The College Board path makes one request per candidate. A 50-question quick request from Carnival is up to 250 detail calls, eight at a time. The index cache is per process, so on serverless hosting each cold instance pays for its own index fetch. The LLM path costs one model call of up to 12,000 output tokens per request, plus up to two searches and eight page reads; the search retries use `time.sleep`, but they run in a worker thread through `asyncio.to_thread`, so they do not block the event loop. The only client cache is the five-minute one in `fetchQuestionsWithCache`.

7.8 Edge cases and tests

- **Short sets are not topped up.** If the backend returns three questions, the game gets three. The bank is used only when the list is empty.
- **The mock exam uses no pace**, so it gets the whole index and no length filter (chapter 12).
- **GRE requests over 25** get at most 25 from the model, plus custom questions.
- **Agent failure hides official questions.** Any exception inside `generate_questions` returns only the custom questions, even if some official ones were already loaded.
- **Unused parameters.** `topic` and `difficulty` are accepted and ignored. `recommended_difficulty` reaches the prompt but not the College Board picker, which always spreads evenly.
- **Random elimination.** When no wrong choice has a usable explanation, `eliminationHint` picks one at random on every render; only the one picked at click time is shown.
- **Comments.** `hints.ts` has two stacked comments above `PROMPT_TIPS`, the first left over from an earlier version.

There are no automated tests. `html_to_text`, `_normalize_llm_question`, `_balance_difficulty`, `whyNot` and `getBankQuestions` are pure functions and the cheapest place to start.

7.9 Study notes

READ IN THIS ORDER

1. `frontend/lib/api/questions.ts` , the whole file.
2. `backend/src/api/questions.py` , `get_questions` .
3. `backend/src/services/agent.py` , `generate_questions` , then `_questions_from_llm` and `_normalize_llm_question` .
4. `backend/src/services/question_sources.py` , `CollegeBoardSource.fetch` and the helpers it calls, then `WebQuestionSource` .
5. `backend/src/utils/html_utils.py` , `sanitize_html` then `html_to_text` .
6. `backend/src/services/custom_questions.py` .
7. `frontend/components/QuestionContent.tsx` and `frontend/components/exam/QuestionCard.tsx` .
8. `frontend/lib/hints.ts` and `frontend/components/exam/useQuestionHints.ts` .

CHECK YOURSELF

The model says a question came from source 2. When is it tagged `web` ?

Only when page 2 exists and the first 50 normalized characters of the stem or passage appear in that page's text. Otherwise it is tagged `ai` .

Why does the College Board source fetch four or five times as many candidates as it needs?

Free-response items, items with figures and, for quick pace, long items are dropped after the detail fetch, when their content is known.

A fresh clone runs with no keys at all. What does a GRE game show?

The backend returns an empty list (or fails), and `fetchAIQuestions` falls back to the built-in bank: six of the nine questions for that section, after the pace filter.

Why is HTML sanitized in the browser when the backend already did it?

Defence in depth before `dangerouslySetInnerHTML` . The frontend allowlist is the same, so a well-behaved backend loses nothing.

How much of a set can custom questions take?

Up to `limit // 2` , and only for a signed-in user with approved questions for that exam and section.

8 The game framework

SATistics teaches through games, and every game is the same machine wearing a different costume: questions go in, a player answers them by doing something in a 3D scene, and an analytics object comes out. This chapter describes that machine. It covers how a game is registered, the three routes a player passes through, the container component that owns each game, the shared exam UI that every container borrows from, and how questions arrive and results leave. The one idea that shaped it is that the engine never touches React and React never touches the scene: they talk through a handful of callbacks and imperative methods.

This chapter describes the code on `main` at commit `9b9abf0`. There is no plugin system and no game interface file. A "framework" here is a set of conventions that six games happen to follow, plus a few shared components in `frontend/components/exam/` and `frontend/components/arcade/`. Where the games disagree with each other, this chapter says so, because the disagreements are where new contributors get caught.

8.1 Intent

The first intent is that a game is practice, not a reward for practice. Answering is the gameplay: the player shoots the zombie wearing the answer, runs through the gate of the answer, stands on the pad of the answer. A question is never a modal that interrupts the game; it is the reason the game is moving.

The second intent is that every game uses the same questions and reports the same numbers. All of them fetch from one backend endpoint through `lib/api/questions.ts`, all of them fall back to the same offline bank, and all of them hand a `GameAnalytics` object to `apiClient.saveScore`. That is what lets the statistics pages and the "Your games" panel treat a Pac-Man round and a Squid Game round as rows in one table.

The third intent is that the exam stays visible. The newer games render the full question, passage and choices in a shared `QuestionCard` beside the action, pace themselves from the real exam's seconds per question, offer hints that rule out choices, and refuse to show the answer after a first miss so the question can come back later.

8.2 Files and folders

<code>frontend/lib/games.ts</code>	the registry: id, name, description, status
<code>frontend/lib/gameMeta.ts</code>	GAME_META: catalogue title, genre, answerBy; gameTitle
<code>frontend/lib/arcade.ts</code>	pixel font, getHighScore, recordHighScore (localStorage)
<code>frontend/lib/exam.ts</code>	ExamPrefs, EXAMS pacing, QUESTION_COUNTS, gamePace
<code>frontend/lib/api/questions.ts</code>	SATQuestion, fetchAIQuestions, fetchQuestionsWithCache
<code>frontend/lib/api/client.ts</code>	apiClient.getAIQuestions, apiClient.saveScore
<code>frontend/lib/hints.ts</code>	strategyHint, eliminationHint, insightFor
<code>frontend/lib/questionBank.ts</code>	offline questions when the backend is unreachable

frontend/app/games/page.tsx	the catalogue (Explore)
frontend/app/games/[gameId]/select/page.tsx	setup: exam, section, question count
frontend/app/games/[gameId]/page.tsx	play: full screen, renders GameContainer
frontend/app/games/[gameId]/layout.tsx	a pass-through layout
frontend/components/GameContainer.tsx	dispatches a game id to its container
frontend/components/*GameContainer.tsx	one React container per game (six files)
frontend/components/GameOverModal.tsx	results screen, takes GameAnalytics
frontend/components/QuestionContent.tsx	renders sanitized question HTML or plain text
frontend/components/brand/GameCover.tsx	SVG cover art, keyed by game id
frontend/components/exam/	shared exam UI (see 8.6)
frontend/components/arcade/ArcadeFrame.tsx	neon cabinet, used by Zombie
frontend/components/arcade/GameHud.tsx	GameTopBar, HUD_PANEL, Pips
frontend/games/BaseGame.ts	abstract base: width, height, canvas, state, pause
frontend/games/GameRenderer.ts	generic canvas loop; only knows 'mario'
frontend/games/<id>/	one engine per game, plus its types
frontend/public/games/subway-surfers/	glTF map served at runtime
frontend/public/games/squid-game/	FBX characters, glTF models, textures, audio
frontend/public/*.fbx, *.mp3	Subway Surfers character and sound

The split to remember is `components/` versus `games/`. A file in `frontend/games/` is an engine: plain TypeScript that owns a Three.js scene and knows nothing about React. A file in `frontend/components/` is the React side: it fetches questions, constructs the engine, runs the animation frame loop and draws every piece of text on top of the canvas.

8.3 Model

THE REGISTRY

`lib/games.ts` exports `games: Game[]`. The `Game` interface lives, for historical reasons, in `frontend/games/subway-surfers/types/game.ts`: an `id`, a `name`, a `description` and a `status` of `'coming-soon'` or `'available'`. All six entries are `'available'`, and nothing in the app reads `status`. The array order is the catalogue order.

`lib/gameMeta.ts` holds what the catalogue actually shows. Its header comment says it is kept apart so the engine's `Game` type stays unchanged. Each entry in `GAME_META` has a display `title`, a `genre` (one of `'Action'` | `'Arcade'` | `'Runner'` | `'Survival'`, listed in `GENRES` for the filter chips) and an `answerBy` sentence. `gameTitle(id, fallback)` resolves an id to a title and special-cases `'mock-exam'`, which is not a game but saves sessions under that id.

ID	GAME_META TITLE	GENRE	CONTAINER	ENGINE CLASS
zombie	Zombie Apocalypse	Action	ZombieGameContainer (default export)	ZombieGame
whackamole	Whack-A-Mole	Arcade	WhackAMoleGameContainer	WhackAMoleGame
carnival	Balloon Pop	Arcade	CarnivalGameContainer	CarnivalGame
subway-surfers	Subway Surfers	Runner	SubwaySurfersGameContainer	SubwaySurfersGame
squid-game	Squid Game	Survival	SquidGameContainer	SquidGameGame
pac-man	Pac-Man	Arcade	PacManGameContainer	PacManGame

EXAM PREFERENCES

`lib/exam.ts` defines `ExamPrefs`: `exam` ('sat' | 'gre'), `section` ('quant' | 'verbal') and `questionCount`, which must be one of `QUESTION_COUNTS = [6, 10, 20]`. They live in `localStorage` under `exam_prefs`; `getExamPrefs()` validates every field and falls back to `{ exam: 'sat', section: 'quant', questionCount: 10 }`. `EXAMS` carries the real pacing per section, from 71 seconds per question for SAT Reading and Writing to 105 for GRE Quantitative. `QUICK_SECONDS_PER_QUESTION` is 35, the budget for the quick-fire games. `gamePace(id)` returns 'deep' for ids in `DEEP_GAMES`, which is only `['squid-game']`, and 'quick' for everything else; the backend uses it to choose short questions or passage questions.

QUESTIONS AND RESULTS

A question is `SATQuestion` from `lib/api/questions.ts`: an `id`, `question`, `options`, a zero-based `correctAnswer`, `topic`, `difficulty` and `explanation`, plus optional `passage`, `stem`, `skill`, sanitized HTML variants and source fields. Whack-A-Mole and Balloon Pop import an older, narrower `SATQuestion` from their own `types.ts`; it is structurally compatible, and their containers cast to the full type when they need the HTML fields.

A result is `GameAnalytics`, which is defined in `frontend/games/whackamole/types.ts` and imported from there by every other game: `gameId`, `score`, `accuracy` (a percentage), `correctAnswers`, `wrongAnswers`, `questionAttempts` (one `QuestionAttempt` per answer, with `timeSpent` in milliseconds), `topicPerformance`, `streakInfo.maxStreak` and `averageResponseTime`. The backend mirrors it as a Pydantic model in `backend/src/models/schemas.py`. The newer games also return a review list (`ZombieReviewItem`, `RunnerReviewItem`, `SquidReviewItem`, `PacManReviewItem`), all the same shape: `question`, `selected` (null on a timeout), `isCorrect`, `timeSpent`.

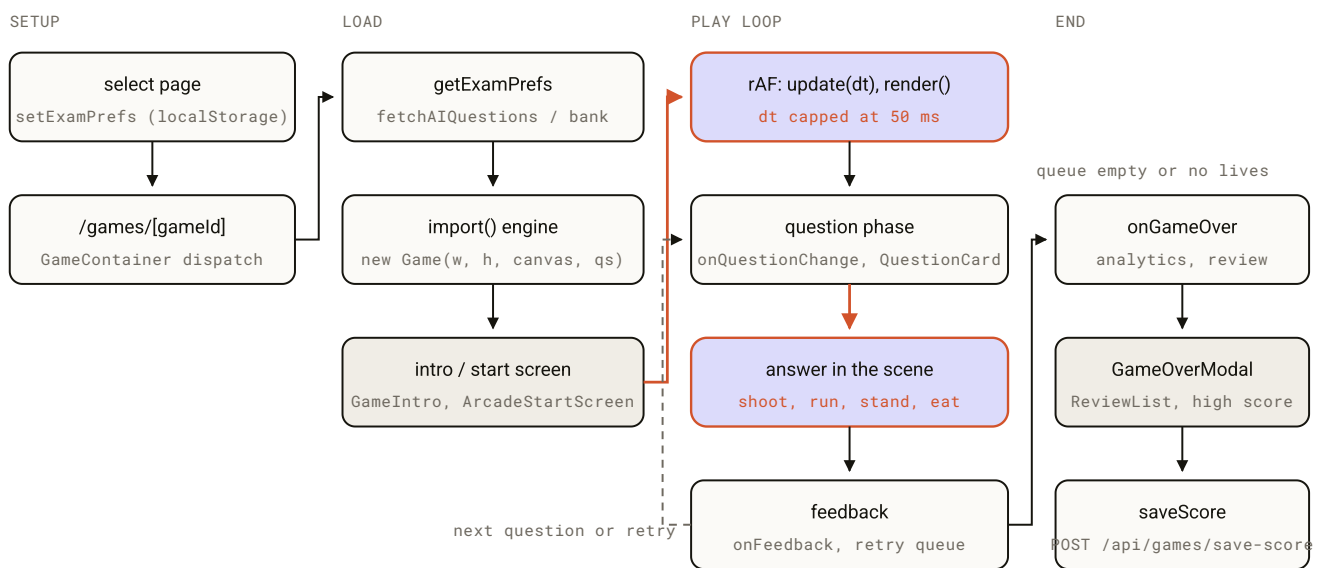
8.4 Routes

A player passes through three pages. `app/games/page.tsx` is the catalogue. It filters `games` by a search box and a genre chip, matching against the `GAME_META` title and `answerBy`, draws each card with `GameCover`, and links to `/games/[id]/select`. Its side panel, `YourGames`, calls `useRecentSessions(50)` from `lib/sessions.ts` and groups saved sessions by `game_id` to show plays and best score.

`app/games/[gameId]/select/page.tsx` is setup. It loads `getExamPrefs()` after mount, offers buttons for exam, section and question count, writes each change straight back with `setExamPrefs`, lists the section's topics from `courseFor`, and shows the real exam pace. The "Start game" button is a plain link to `/games/[id]`. Nothing is passed in the URL: the play page reads the same `localStorage` key.

`app/games/[gameId]/page.tsx` is a server component. It exports `generateStaticParams` over `games`, calls `notFound()` for any id not in the registry, and otherwise renders a fixed, black, full-screen div containing `<GameContainer game={game} />`. Everything below that is a client component.

8.5 How it works



The engine owns the scene and the rules; the container owns the DOM, the loop and the network.

The life of one round. Setup writes preferences to the browser, the container reads them back, loads questions and the engine, then runs the loop until the engine reports the game is over.

DISPATCH

`GameContainer` holds a constant `OWN_CONTAINER_GAMES` listing all six ids, then a chain of `if (game.id === ...)` early returns that render each game's own container. The early returns come after a `useEffect`, because React requires hooks to run in the same order on every render; the comment in the file says so. Only when the id matches none of them does the component render a bare full-screen `<canvas>`, and the effect constructs a `GameRenderer` on it.

`GameRenderer` and `BaseGame` are the original design: an abstract class with `init`, `update(deltaTime)`, `render(ctx)` and `handleInput(key)`, driven by a generic loop that also forwards keyboard events and draws on a 2D overlay canvas. Its `init()` switch knows exactly one id, `'mario'`, and `'mario'` is not in `games`, so the play page returns 404 before `GameContainer` ever sees it. In practice the canvas branch is unreachable. Every live game takes the container path.

THE CONTAINER PATTERN

`ZombieGameContainer` is the cleanest example, and the newer containers follow it closely. In order:

1. A mount effect calls `getExamPrefs()`, reads the local high score with `getHighScore`, and calls `fetchAIQuestions(count, undefined, gamePace(GAME_ID))`. A `cancelled` flag guards the `setQuestions` in case the page unmounts first.
2. A second effect waits for both the canvas ref and the questions, then dynamically imports the engine module with `await import('@games/zombie/ZombieGame')`. The type import at the top of the file is `import type`, so Three.js and the engine stay out of the page's initial bundle.
3. It constructs the engine with the window size, the canvas, the questions and a config of `{ questionCount, secondsPerQuestion }`, stores it in a ref, and assigns the callbacks: `onHudChange`, `onQuestionChange`, `onFeedback` and `onGameOver`. Most are passed React state setters directly.
4. It adds `resize` and `keydown` listeners and starts a `requestAnimationFrame` loop that computes `deltaTime`, caps it at 50 ms so a background tab does not cause a jump, and calls `game.update(deltaTime)` then `game.render()`.
5. The effect's cleanup sets `cancelled`, cancels the frame, removes the listeners and calls `game.cleanup()`, which disposes geometries, materials and the renderer.

Everything the player reads is DOM, not canvas: the score bar, the question card, the hint button, the calculator, the pause menu and the results modal. The engine pushes a snapshot of its state through `onHudChange`, throttled to one call per `HUD_UPDATE_INTERVAL_MS` (100 ms) in Zombie, Subway Surfers and Squid Game, and the container renders from that snapshot. Controls flow the other way through public methods on the engine held in `gameRef`: `start()`, `setPaused()`, `setMuted()`, `skipFeedback()`, and per-game verbs such as `shootAt`, `moveLane` or `lockIn`.

A CONSTRUCTOR THAT REPORTS TOO EARLY.

The newer engines call `emitHud(true)` from their constructor, before the container has assigned `onHudChange`. `ZombieGameContainer` works around it by calling `game.setMuted(false)` straight after wiring the callbacks, which re-emits the HUD. The engines that load assets asynchronously (Subway Surfers, Squid Game) emit again when loading finishes, so they do not need the trick.

GETTING QUESTIONS

`fetchAIQuestions(limit, fallbackQuestions, pace)` reads the exam and section from `getExamPrefs()` and calls `apiClient.getAIQuestions`, which issues `GET /api/questions/` with `use_agent=true`, `limit`, `use_web_search=true`, `exam`, `section` and, when given, `pace`. If the call throws or returns no questions, it falls back to `getBankQuestions(exam, section, pace)` from `lib/questionBank.ts`, then to `fallbackQuestions`. A game therefore always gets a list, though it may be short.

`fetchQuestionsWithCache(gameId, limit, fallback)` wraps that with a five-minute `sessionStorage` cache keyed `ai_questions_${gameId}_${exam}_${section}`. Only Whack-A-Mole and Balloon Pop use it, and both ask for 50 questions regardless of the chosen question count. The other four call `fetchAIQuestions` directly with a count derived from the preference: Zombie asks for `questionCount`, Subway Surfers and Squid Game ask for twice that so difficulty can adapt, and Pac-Man asks for a fixed 20.

TIMING AND PACING

Every engine that has a clock receives it from the container. Zombie and Subway Surfers get `QUICK_SECONDS_PER_QUESTION` ; Pac-Man gets it through `setQuestions(questions, QUICK_SECONDS_PER_QUESTION)` ; Whack-A-Mole imports the same constant inside the engine for its `popDuration` . Squid Game, the one deep game, gets `EXAMS[exam].sections[section].secondsPerQuestion` , the real exam's pace. Balloon Pop has no clock at all. Engines measure time from the `deltaTime` the loop gives them, so a paused game's clock stops; the exception is the 2-second `setTimeout` that Whack-A-Mole and Balloon Pop use between questions.

SAVING RESULTS

When an engine calls `onGameOver` , the container stores the result in state, which renders `GameOverModal` , and records the local high score with `recordHighScore(gameId, score)` (Zombie, Whack-A-Mole, Balloon Pop and Pac-Man do; Subway Surfers and Squid Game do not). It then imports `ApiClient` lazily and calls `ApiClient.saveScore(gameId, analytics)` , which posts `{ gameId, analytics }` to `/api/games/save-score` . The endpoint depends on `get_current_user` , so a signed-out player's save fails; every container catches the error and logs it, and the modal is unaffected.

8.6 Shared exam UI

FILE	EXPORTS	USED BY
<code>exam/GameIntro.tsx</code>	<code>GameIntro</code> , <code>GameLoading</code>	all but Zombie
<code>exam/QuestionCard.tsx</code>	<code>QuestionCard</code> , <code>sourceLabel</code> , <code>AnswerFeedback</code>	Zombie, Subway Surfers, Squid Game, Pac-Man
<code>exam/LaneChip.tsx</code>	<code>LaneChip</code>	QuestionCard, ReviewList, Balloon Pop
<code>exam/PauseMenu.tsx</code>	<code>PauseMenu</code>	all six
<code>exam/Calculator.tsx</code>	<code>Calculator</code> , <code>evaluate</code>	Zombie, Subway Surfers, Squid Game, the mock exam
<code>exam/HintButton.tsx</code>	<code>HintButton</code>	Zombie, Subway Surfers, Squid Game, Pac-Man
<code>exam/ReviewList.tsx</code>	<code>ReviewList</code>	Zombie, Subway Surfers, Squid Game, Pac-Man
<code>exam/useQuestionHints.ts</code>	<code>useQuestionHints</code>	Zombie, Subway Surfers, Squid Game, Pac-Man
<code>exam/useViewShift.ts</code>	<code>useViewShift</code> , <code>SIDE_PANEL_MIN_WIDTH</code>	Zombie, Subway Surfers, Squid Game, Pac-Man
<code>arcade/ArcadeFrame.tsx</code>	<code>ArcadeFrame</code> , <code>ArcadeTopBar</code> , <code>ArcadeStartScreen</code>	Zombie
<code>arcade/GameHud.tsx</code>	<code>GameTopBar</code> , <code>HUD_PANEL</code> , <code>Pips</code>	Whack-A-Mole, Balloon Pop, Pac-Man

`GameIntro` is the pre-game card: cover art, a kicker, a two-part title, a summary, a list of `{ label, text }` steps, a start button and a link back to `/games`. `GameLoading` shows the cover and a progress bar; pass `progress={null}` for an indeterminate pulse. `Zombie` uses the arcade equivalents instead: `ArcadeFrame` wraps the screen in the `Press_Start_2P` pixel font from `lib/arcade.ts`, sets `--arcade-color` and lays a CRT overlay on top; `ArcadeStartScreen` is the attract screen with "INSERT COIN".

`QuestionCard` renders the header (question number, skill, difficulty), the passage, the stem and the choices through `QuestionContent`, which sanitizes question HTML against an allowlist of tags including MathML. Choices are buttons only when `onPick` is passed; none of the current games passes it, because the answer is always picked in the scene. `activeOption` and `activeLabel` highlight a tentative pick (`Subway Surfers` passes the current lane and `"YOUR LANE"`). After an answer, `feedback` colours the choices, and if `insight` is set the card hides which choice was right and shows the nudge from `insightFor` instead, because the question will come back.

`useQuestionHints(question)` resets when the question changes. The first `requestHint()` adds a `strategyHint` and returns null; each later one adds an `eliminationHint` and returns the option it ruled out, which the container passes to the engine (`eliminateZombie`, `eliminateLane`, `eliminatePad`, `eliminateAnswer`) so the scene drops that target. `eliminationHint` always leaves the right answer and one other choice standing.

`useViewShift(panelRef, isActive, setViewShift)` measures the question panel and tells the engine how far to offset its camera so the action stays centred in the uncovered part of the screen: half the panel's width beside it at `SIDE_PANEL_MIN_WIDTH` (1024 px) and wider, half its height below it otherwise. A `ResizeObserver` re-measures as the panel grows with each question.

`PauseMenu` offers Resume, Restart (a full `window.location.reload()`), "Change exam or section" (back to the select page) and "Exit to games". `ReviewList` renders each review item in a `<details>`; for a missed question it shows the insight and a button to reveal the answer, so the review doubles as a second try.

`GameOverModal` takes `analytics`, `onRestart`, optional `title` and `subtitle`, and `children` rendered above its buttons, which is where containers put `ReviewList`. It accepts an `emoji` prop it no longer shows, kept so old callers compile.

8.7 Input and assets

KEYBOARD AND TOUCH

Input is wired by the container, not by a shared layer. Keyboard listeners go on `window` and ignore `e.repeat`; Space and the arrow keys call `preventDefault()` so the page does not scroll. P and Escape pause in the four games that listen for keys; Whack-A-Mole and Balloon Pop have no keyboard input and pause only from the menu button. Number keys 1 to 5 pick an answer in `Zombie` and `Subway Surfers`. Touch uses pointer events, never touch events: `Zombie` aims on `onPointerMove` and fires on `onPointerDown`; `Subway Surfers` and `Pac-Man` treat a pointer drag longer than `SWIPE_MIN_DISTANCE` (30 and 24 px) as a swipe; `Squid Game` and `Pac-Man` draw on-screen buttons, and `Squid's` `holdKey` helper maps pointer down and up to `handleInput` and `handleKeyRelease`, so a held button behaves like a held key. `Whack-A-Mole` and `Balloon Pop` listen for `click` on the canvas inside the engine and raycast against their meshes, which works for taps too. Canvases that take gestures carry `touch-none`.

ASSETS

Whack-A-Mole, Balloon Pop, Zombie and Pac-Man build their scenes entirely from Three.js primitives and `CanvasTexture` labels; they load no files. Subway Surfers loads a glTF map from `/games/subway-surfers/assets/subway_surfers_maps/scene.glTF`, the FBX character from `/Flying.fbx` with `/Falling.fbx` and `/Falling Flat Impact.fbx`, and three MP3s from the `public/` root. Squid Game loads three FBX animations and three glTF models from `/games/squid-game/assets`, two textures from its `squid-textures` folder, and audio from `/games/squid-game/audio`. Anything under `frontend/public/` is served from the site root; anything elsewhere is not.

8.8 Performance

The heavy cost is download, not frame time. `frontend/public/games` is about 129 MB, of which Squid Game is 124 MB: the `red_guy` guard model alone is 43 MB, and the `squid-textures` folder is 72 MB although the engine reads only two images from it. Engines are imported dynamically so none of this, nor Three.js, lands in the catalogue's bundle.

On the frame side, the newer engines are frame-rate independent: they scale motion by `dt` and smooth with an exponential `damp(rate, dt)`. Zombie, Subway Surfers and Squid Game cap the renderer's pixel ratio at 2; Pac-Man passes `window.devicePixelRatio` uncapped. HUD callbacks are throttled to 10 per second, because each one is a React state update. The older code does not follow these rules: Pac-Man moves Pac-Man and the ghosts by a fixed step per frame and calls `onHudChange` on every frame; Balloon Pop moves balloons per frame and runs its pop and particle animations in their own `requestAnimationFrame` chains, outside the game loop and outside pause.

8.9 Edge cases

- **No backend.** `fetchAIQuestions` falls back to the offline bank, so every game still starts. Saving then fails and is logged.
- **Signed out.** The game plays normally; `save-score` requires a user, so the session is not stored and does not appear in "Your games". The local high score still updates.
- **Restart.** Every restart is `window.location.reload()`. For the two cached games, a reload within five minutes reuses the same questions from `sessionStorage`.
- **Background tab.** The capped `deltaTime` stops a returning tab from skipping time, except in Whack-A-Mole, whose loop does not cap it, so one long frame can expire the question.
- **Five choices.** `LANE_COLORS` and `LANE_LETTERS` have five entries and the newer engines build one target per option. Whack-A-Mole and Balloon Pop always build four, so a fifth choice cannot be picked.
- **Fractional averages.** The backend's `GameAnalytics.averageResponseTime` is an `int` under Pydantic 2. Whack-A-Mole and Balloon Pop send an unrounded mean, which can fail validation; the other four round it.

There are no automated tests for the games or the shared exam components.

8.10 Study notes

READ IN THIS ORDER

1. `frontend/lib/games.ts` and `frontend/lib/gameMeta.ts` , the registry.
2. `frontend/lib/exam.ts` , preferences and pacing.
3. `frontend/app/games/[gameId]/select/page.tsx` and `page.tsx` , the two routes.
4. `frontend/components/GameContainer.tsx` , the dispatch.
5. `frontend/components/ZombieGameContainer.tsx` , the reference container.
6. `frontend/lib/api/questions.ts` , questions in.
7. `frontend/components/exam/QuestionCard.tsx` and `useQuestionHints.ts` , the exam UI.
8. `frontend/components/GameOverModal.tsx` , results out.

CHECK YOURSELF

Why does the "Start game" link carry no query string?

The setup page writes every choice to `localStorage` through `setExamPrefs` as it is made, and the container reads it back with `getExamPrefs()` . The URL only names the game.

Can `GameRenderer` run the Mario placeholder today?

No. `'mario'` is not in `games` , so `/games/mario` calls `notFound()` before `GameContainer` renders.

Why do containers import the engine with `import()` inside an effect?

To keep Three.js and the engine out of the page bundle and off the server. The top of the file uses `import type` , which is erased at compile time.

Where does a hint's elimination become visible in the scene?

`requestHint()` returns the ruled-out option and the container passes it to an engine method such as `eliminateLane` , which hides that gate. The hook only manages text and the list of eliminated options.

Why would a game's session be missing from the stats page after a good round?

The player was signed out, the backend was down, or the analytics failed validation; all three are caught and only logged in the console.

9 The games, and adding one

Six games are registered in `frontend/lib/games.ts`. They were written at different times and it shows: Zombie, Subway Surfers and Squid Game share a modern shape with phases, feedback, retries and a review list; Pac-Man sits in between; Whack-A-Mole and Balloon Pop are the originals and do the least. This chapter walks through each one as it is today, then gives the steps, the contract and a checklist for adding a seventh.

Read Chapter 8 first. It describes the container pattern, the shared exam UI and the question and save paths that every game here uses. This chapter only covers what each game does differently. Every game renders with Three.js on a WebGL canvas; none of them uses the 2D canvas path in `GameRenderer`.

9.1 The six at a glance

GAME	ANSWER BY	CLOCK	QUESTIONS FETCHED	ENDS WHEN	REVIEW LIST
Zombie	shoot a zombie	35 s quick	questionCount	queue empty or 4 hearts gone	yes
Whack-A-Mole	click a mole	35 s quick	50, cached	every question asked	no
Balloon Pop	shoot a balloon	none	50, cached	every question asked	no
Subway Surfers	fly through a gate	35 s quick, section clock	questionCount x 2	both modules done	yes
Squid Game	stand on a pad	real exam pace	questionCount x 2	finish line, no lives, or no questions	yes
Pac-Man	eat a lettered pellet	35 s quick	20	lives reach 0	yes

9.2 Zombie Apocalypse

```
frontend/games/zombie/ZombieGame.ts      engine, does not extend BaseGame
frontend/games/zombie/types.ts            ZombieConfig, ZombieHudState, ZombieFeedback, Zombi
frontend/components/ZombieGameContainer.tsx default export, arcade skin
```

A first-person shooter in a graveyard. One zombie walks toward the player for each answer choice, each carrying a sign texture with its letter in `LANE_COLORS`. The player aims with the pointer and clicks or taps the zombie wearing the right answer; keys 1 to 5 shoot by letter through `shootZombie(lane)`. `shootAt(x, y)` raycasts from the camera into each walking zombie's `hitMeshes`.

The horde's speed is $(ARRIVE_Z - SPAWN_Z) / secondsPerQuestion$, so the zombies reach the player exactly when the 35-second clock runs out, which resolves the question as a miss. A wrong shot or a timeout costs one of `MAX_HEALTH` (4) hearts; a wrong zombie only flinches and keeps walking. Points are

`DIFFICULTY_POINTS` (100, 150 or 200) plus up to 50 for time left plus 10 per streak step up to 5, multiplied by `HINT_POINTS_FACTOR` (0.6) for each zombie a hint removed. A first miss is pushed onto the back of the queue and `totalQuestions` grows by one, as long as the player still has health. The run ends when the queue is empty or health reaches zero, and `onGameOver` reports `survived` as `health > 0`.

This is the only game in the arcade skin: `ArcadeFrame`, `ArcadeTopBar` and `ArcadeStartScreen`. Sounds are synthesized with an `AudioContext` created in `start()`, so they begin after a user gesture. The scene is all primitives. Limits: the gun and view follow the pointer, which a touch player only moves by tapping.

9.3 Whack-A-Mole

```
frontend/games/whackamole/WhackAMoleGame.ts    extends BaseGame
frontend/games/whackamole/types.ts             SATQuestion (legacy), GameAnalytics, QuestionAtte
frontend/games/whackamole/questions.ts         satQuestions fallback set
frontend/components/WhackAMoleGameContainer.tsx
```

Four moles rise from a grass board, labelled A to D. The question and its four choices are shown in DOM panels above and below the board, colour-matched to the moles through `MOLE_COLORS`. The player clicks a mole; the engine raycasts in its own `click` listener and calls `submitAnswer(index)`. If nobody is hit within `popDuration` (`QUICK_SECONDS_PER_QUESTION * 1000`), it submits `-1`. A right answer scores `100 + streak * 10`. After every answer a 2-second `setTimeout` shows the next question, and the game ends when all questions have been asked.

Limits worth knowing. The game asks every question it was given, which is the 50 requested from `fetchQuestionsWithCache`, not the player's chosen count. It always builds four moles, so a fifth choice cannot be picked. There is no right-or-wrong feedback in the UI: the container keeps `selectedAnswer` and `showResult` state that nothing renders. The loop does not cap `deltaTime`. `cleanup()` removes listeners created with a fresh `.bind(this)`, which never matches the ones added, so they are not actually removed. The pending `setTimeout` is not cancelled on pause or unmount.

9.4 Balloon Pop

```
frontend/games/carnival/CarnivalGame.ts        extends BaseGame
frontend/games/carnival/types.ts               CarnivalGameState with bulletsRemaining
frontend/games/carnival/questions.ts           a near copy of whackamole/questions.ts
frontend/components/CarnivalGameContainer.tsx
```

The id is `carnival`; the catalogue calls it Balloon Pop. Four balloons bob in front of the player, labelled A to D. Each click fires a visible bullet from the camera and spends one of three darts; hitting a balloon pops it with a particle burst and submits that answer. Running out of darts without a hit submits `-1`. Scoring, the 2-second gap and the end condition are the same as Whack-A-Mole, and so are the four-choice limit, the 50-question fetch and the listener leak. The container shows a single `strategyHint` behind a Hint button and draws the dart count with `Pips`.

There is no clock, so a player can think as long as they like. Balloon motion and the pop and particle animations advance per frame rather than per second, and the latter two run in their own

`requestAnimationFrame` chains that ignore pause.

9.5 Subway Surfers

```
frontend/games/subway-surfers/SubwaySurfersGame.ts    extends BaseGame, about 1,300 lines
frontend/games/subway-surfers/types/game.ts           Game, LANE_COLORS, LANE_LETTERS, Runner* ty
frontend/components/SubwaySurfersGameContainer.tsx
frontend/public/games/subway-surfers/assets/subway_surfers_maps/scene.gltf
frontend/public/Flying.fbx, Falling.fbx, Falling Flat Impact.fbx
frontend/public/bg-music.mp3, falling.mp3, dying.mp3
```

A flying runner over an endless subway map. Each question spawns one gate per choice across the lanes. The player steers with the arrow keys, A and D, number keys or a swipe; the gates close in as the question's time runs out, and the answer is whatever lane the runner is in when it reaches the gate. Pressing up, W, Space or Enter calls `dive()`, which speeds the runner to `DIVE_SPEED` to commit early. Hints call `eliminateLane`, which hides that gate and moves the runner off it; `moveLane` skips eliminated lanes.

The run is shaped like the digital SAT. The constructor shuffles the pool and splits `questionCount` into two modules. Module 1 is an even mix of difficulties. If the player got at least `HARDER_MODULE_ACCURACY` (60%) of it right, module 2 prefers hard questions, otherwise easy ones, and `onModuleChange` tells the container which, so it can explain during the 4-second break. Each module has a section clock of `secondsPerQuestion` times its length; a single gate waits at most `QUESTION_TIME_CAP` (1.5) times the per-question budget. When the section clock runs out, the questions left are recorded as unanswered. A first miss is appended to the module and adds 35 seconds to its clock. A wrong answer plays the falling and impact animations before the next question.

The map is `MAP_TILE_COUNT` (3) copies of the glTF recycled ahead of the runner, with obstacles randomized per tile and trains moving on some tracks. Every `REBASE_DISTANCE` (3,000 units) the world is shifted back toward the origin to keep floating-point precision. Limits: it does not record a local high score, and `frontend/public/games/subway-surfers/jake_subway_surfers/` is shipped but not loaded by any code.

9.6 Squid Game

```
frontend/games/squid-game/SquidGameGame.ts    extends BaseGame, about 1,150 lines
frontend/games/squid-game/types.ts             SquidPhase, SquidOutcome, SquidHudState, ...
frontend/games/squid-game/yuka.d.ts            type stub for the unused yuka package
frontend/games/squid-game/questions.json       not imported anywhere
frontend/games/squid-game/assets/              a 72 MB glTF room, not served, not loaded
frontend/components/SquidGameContainer.tsx
frontend/public/games/squid-game/assets/       FBX animations, glTF guard, doll, tree, textures
frontend/public/games/squid-game/audio/        Cocoma.mp3, Redlight.mp3 (GreenLight.mp3 unused)
```

Red Light, Green Light, and the only deep game: `gamePace` returns `'deep'` for it, so the backend sends passage and multi-step questions, and the clock is the real exam's `secondsPerQuestion`. The phases are `loading`, `ready`, `green`, `turning`, `question`, `feedback` and `done`. On green the player runs with W A S D or the arrows (on-screen buttons on phones). When the doll turns, anyone moving faster than `CAUGHT_SPEED` is shot by the guards; the player loses a life. Then a question appears and one answer pad per

choice is laid on the sand ahead. Standing on a pad for `PAD_LOCK_SECONDS` (2.5) locks it in, or the player presses Space or the lock button for `lockIn()`.

A right answer earns a 4-second green light and steps difficulty up; a wrong one costs a life, earns 1.5 seconds and steps difficulty down. `beginQuestion` picks the unused question closest to the current difficulty, and returns a missed question after one other. The run speed is computed in the constructor so a player at `EXPECTED_ACCURACY` (70%) reaches the finish line about when the questions run out. Lives are `Math.max(3, Math.ceil(totalQuestions / 3))`. Outcomes are `victory` (crossed the finish, plus `FINISH_BONUS` 500 and 100 per life), `eliminated` and `short` (out of questions first). Twenty-four `NPC_COUNT` contestants are cloned skinned meshes that occasionally fail to stop. The calculator opens by default for quant.

Limits: the six models load in parallel through `track()`, and the 43 MB guard model dominates first-load time. Every hint calls `registerHint()`, including the first strategy tip, so Squid Game penalizes a tip that Zombie and Subway Surfers give for free. `GAME_META` says "five misses and you're out", which is only true for some question counts.

9.7 Pac-Man

```
frontend/games/pac-man/PacManGame.ts      extends BaseGame; exports PacManHudState, PacManRev
frontend/components/PacManGameContainer.tsx
```

A 15 by 15 maze from a hard-coded grid in `generateMaze`, with dots, power pellets and four ghosts, viewed from above. Every `DOTS_PER_QUESTION` (10) dots, outside a 5-second cooldown, a question appears: the ghosts freeze, and one lettered answer pellet per choice drops at least `MIN_ANSWER_DISTANCE` (6) from Pac-Man. Eating a pellet answers. A right answer scores 100 and turns the ghosts blue for 5 seconds; a wrong answer or a timeout costs one of three lives. Hints call `eliminateAnswer`. Clearing every dot calls `nextLevel`, which refills the maze and speeds up the ghosts.

Questions come from `setQuestions` rather than the constructor and repeat in order once the 20 fetched run out. The game only ends when lives reach zero, so the question count preference is ignored. There is no feedback after an answer and no retry queue; `QuestionCard` is given `feedback={null}` and `totalQuestions={0}`. Movement is a fixed `moveSpeed` of 0.1 per frame, so Pac-Man runs twice as fast on a 120 Hz display, and `emitHud()` fires every frame.

9.8 Mario, the one that is not there

`frontend/games/mario/MarioGame.ts` extends `BaseGame` and draws "Mario / Coming Soon..." on a 2D context; its methods are TODO comments and its `assets/` folder is empty. `GameRenderer` imports it and is the only thing that does. It is not in `games`, has no `GAME_META` entry, no cover and no container, and `/games/mario` returns 404. Treat `MarioGame`, `GameRenderer` and the canvas branch of `GameContainer` as dead code. `components/GameCard.tsx` is also unused.

9.9 Adding a new game

Copy the Zombie shape, not the Whack-A-Mole shape. The steps below use a hypothetical id `comet`; substitute your own, in kebab case, because it becomes the URL, the analytics `gameId`, the high score key

`arcade_hi_comet` and the `game_id` column of every saved session.

1. THE ENGINE

Create `frontend/games/comet/types.ts` with a phase union, a config of `{ questionCount, secondsPerQuestion }`, a HUD state, a feedback type and a review item (reuse the shape of `ZombieReviewItem` so `ReviewList` accepts it). Then create `frontend/games/comet/CometGame.ts`. Extending `BaseGame` is optional; `Zombie` does not. The skeleton the containers expect is:

```
export class CometGame {
  public onHudChange?: (state: CometHudState) => void
  public onQuestionChange?: (question: SATQuestion | null) => void
  public onFeedback?: (feedback: CometFeedback | null) => void
  public onGameOver?: (analytics: GameAnalytics, review: CometReviewItem[]) => void

  constructor(width: number, height: number, canvas: HTMLCanvasElement,
    questions: SATQuestion[], private config: CometConfig) { }

  start(): void { } // ready to first question
  update(deltaTime: number): void { } // milliseconds, already capped
  render(): void { }
  resize(width: number, height: number): void { }
  setViewShift(x: number, y = 0): void { }
  setPaused(paused: boolean): void { }
  setMuted(muted: boolean): void { }
  skipFeedback(): void { }
  eliminateOption(option: number): void { } // for hints
  cleanup(): void { } // dispose geometry, materials, renderer, audio
}
```

Import `SATQuestion` from `@/lib/api/questions`, not from a game folder, and `GameAnalytics` and `QuestionAttempt` from `@/games/whackamole/types`. Use `LANE_COLORS` and `LANE_LETTERS` so your targets match the chips in `QuestionCard`.

2. THE CONTAINER

Create `frontend/components/CometGameContainer.tsx` as a named export. Follow `ZombieGameContainer`: load prefs and questions on mount with a cancel flag; `await import('@/games/comet/CometGame')` in a second effect; assign the callbacks; run the loop with `Math.min(currentTime - lastTime, 50)`; clean up everything in the effect's return. Render `GameLoading`, then `GameIntro`, then a HUD (`GameTopBar` or `ArcadeTopBar`), a panel with `QuestionCard` and `HintButton` measured by `useViewShift`, a `Calculator` when `prefs.section === 'quant'`, `PauseMenu`, and finally `GameOverModal` with `ReviewList` as its child. In `onGameOver`, call `recordHighScore` and then `apiClient.saveScore('comet', analytics)` inside a try and catch.

3. REGISTRATION

- `frontend/lib/games.ts`: `append { id: 'comet', name, description, status: 'available' }`. This alone makes `/games/comet` resolve and adds the static param.

- `frontend/lib/gameMeta.ts` : add `comet: { title, genre, answerBy }`. The genre must be a member of `Genre`; if you add a new genre, add it to `GENRES` too or it gets no filter chip.
- `frontend/components/GameContainer.tsx` : import the container, add `'comet'` to `OWN_CONTAINER_GAMES`, and add an `if (game.id === 'comet')` return next to the others, after the hook.
- `frontend/components/brand/GameCover.tsx` : add a `comet` entry to `COVERS` with a `bg` and SVG `art` drawn on a 300 by 400 `viewBox` in the palette constants. Without it the catalogue shows the lime-circle `FALLBACK`.
- `frontend/lib/exam.ts` : add the id to `DEEP_GAMES` only if the game gives the player time for passage questions; pass the matching seconds per question.
- `frontend/lib/arcade.ts` needs no edit; its functions take the id.
- Assets go under `frontend/public/games/comet/` and are loaded by absolute path, for example `/games/comet/model.gltf`, with loading progress reported through the HUD.

The backend needs nothing. `/api/games/save-score` accepts any `gameId` string and the stats endpoints group by it.

THE CONTRACT

DIRECTION	WHAT	RULE
in	<code>SATQuestion[]</code>	Build one target per <code>options</code> entry, two to five. Never assume four.
in	<code>config</code>	<code>questionCount</code> from prefs; <code>secondsPerQuestion</code> from <code>QUICK_SECONDS_PER_QUESTION</code> or <code>EXAMS</code> .
in	<code>deltaTime</code>	Milliseconds. Scale all motion and clocks by it; no <code>setTimeout</code> for game time.
out	<code>onQuestionChange</code>	The question on screen, or null between questions.
out	<code>onFeedback</code>	<code>{ isCorrect, selected, correctAnswer, points, willRetry }</code> ; null to clear.
out	<code>onHudChange</code>	A fresh object, throttled to about 10 per second unless forced.
out	<code>onGameOver</code>	Called once. <code>gameId</code> equals the registry id; <code>averageResponseTime</code> and every <code>timeSpent</code> are integers.

INTEGERS ONLY.

The backend's `GameAnalytics` declares `timeSpent` and `averageResponseTime` as `int`. Wrap both in `Math.round`, as `Zombie` does, or the save can be rejected.

CHECKLIST

1. Engine in `frontend/games/<id>/`, no React imports, a `cleanup()` that disposes everything.
2. Container in `frontend/components/`, engine imported dynamically.

3. Entry in `games.ts` , `gameMeta.ts` , `GameContainer.tsx` (both the list and the return) and `GameCover.tsx` .
4. Variable option count handled, including five.
5. Keyboard and pointer input both work; canvas has `touch-none` if it takes gestures.
6. Pause stops every clock; P and Escape open `PauseMenu` .
7. Hints remove a target in the scene; a first miss returns later with `willRetry` .
8. `onGameOver` fires once, the review list is passed to `ReviewList` , the high score is recorded and `saveScore` is called.
9. Assets in `frontend/public/games/<id>/` , sized with the 129 MB already there in mind.
10. Played once signed in, then checked on the catalogue's "Your games" panel.

9.10 Study notes

READ IN THIS ORDER

1. `frontend/games/zombie/ZombieGame.ts` , the smallest modern engine.
2. `frontend/components/ZombieGameContainer.tsx` , its container.
3. `frontend/games/subway-surfers/SubwaySurfersGame.ts` , `beginModule` to `resolveAnswer` .
4. `frontend/games/squid-game/SquidGameGame.ts` , `beginQuestion` and `updatePads` .
5. `frontend/games/whackamole/WhackAMoleGame.ts` , to see what not to copy.

CHECK YOURSELF

Why does Subway Surfers fetch twice the question count?

Module 2 picks harder or easier questions depending on module 1, so the pool must hold more than the run will use. Squid Game does the same so its difficulty can step up and down.

A GRE question has five choices. Which games can a player answer it in?

Zombie, Subway Surfers, Squid Game and Pac-Man, which build one target per option. Whack-A-Mole and Balloon Pop always build four.

You registered a game in `games.ts` and `gameMeta.ts` only. What does the player see?

A fallback cover in the catalogue, a working setup page, and on play a blank black screen:

`GameContainer` falls through to the canvas branch and `GameRenderer` logs "Unknown game".

Why does a missed question not show its answer straight away?

It is queued to come back. The feedback carries `willRetry` , the container passes an `insight` to `QuestionCard` , and the card hides the right choice.

10 Learn: lessons and videos

The games test what you know; Learn teaches it. Every course topic on both exams has a short lesson: a summary, the key ideas, a worked example revealed one step at a time, the traps that cost points, and one quick check question. Beside it sits a column of YouTube lessons that the backend finds and ranks on demand. The lesson text is static TypeScript shipped with the frontend; the videos are the only part that touches the server.

10.1 Intent

A student who misses three Advanced Math questions in a game should be one click from the idea they are missing. Learn is built for that. The hub opens on the exam from the student's profile, puts their weakest topic first, and marks each card "Needs work" or "Strong" from the same `user_stats` lists the dashboard uses. A lesson ends with "Practice in a game", which sets the exam and section and drops the student into the games list.

Lessons are written content, not data. Keeping them in `frontend/lib/lessons.ts` means they are reviewed like code, typed, versioned and rendered without a request. Videos are the opposite: they change, there are thousands, and a good one for "Standard English Conventions" is a search away. So the backend searches, ranks toward lesson-length videos from established prep channels, and caches the result for a day.

10.2 Files and folders

<code>frontend/app/learn/page.tsx</code>	the hub: exam toggle, progress, featured lesson, co
<code>frontend/app/learn/[exam]/[topic]/page.tsx</code>	one lesson: LessonPage, LessonView, VideoPanel
<code>frontend/lib/lessons.ts</code>	Lesson type, LESSONS, topicSlug, lessonHref, findLe
<code>frontend/lib/courses.ts</code>	Course type, COURSES, courseFor
<code>frontend/lib/learnProgress.ts</code>	useLearned, lessonId (localStorage)
<code>frontend/components/learn/QuickCheck.tsx</code>	the one-question check
<code>frontend/components/learn/Videos.tsx</code>	useTopicVideos, VideoPlayer, VideoMeta, VideoRow, V
<code>frontend/lib/api/client.ts</code>	getTopicVideos, the Video type
<code>backend/src/api/learn.py</code>	GET /api/learn/videos
<code>backend/src/services/videos.py</code>	provider chain, ranking, cache
<code>backend/src/models/schemas.py</code>	Video, VideoResponse

10.3 Model

Courses. `frontend/lib/courses.ts` defines a `Course` as an `exam`, a `section` (`quant` or `verbal`), a title, a summary and a list of `topics`. The comment on `topics` matters: they are the official content domains and must match the topic names the question bank uses, because that is how weak topics from games line up with lessons. There are four courses:

COURSE	EXAM:SECTION	TOPICS
SAT Math	sat:quant	Algebra; Advanced Math; Problem-Solving and Data Analysis; Geometry and Trigonometry
SAT Reading & Writing	sat:verbal	Information and Ideas; Craft and Structure; Expression of Ideas; Standard English Conventions
GRE Quant	gre:quant	Arithmetic; Algebra; Geometry; Data Analysis
GRE Verbal	gre:verbal	Text Completion; Sentence Equivalence; Reading Comprehension

`courseFor(exam, section)` finds one, falling back to the first course.

Lessons. `frontend/lib/lessons.ts` defines the `Lesson` interface:

```
export interface Lesson {
  exam: ExamId
  section: SectionId
  topic: string
  summary: string
  ideas: { title: string; body: string }[]
  example: { prompt: string; steps: string[]; answer: string }
  traps: string[]
  check: { question: string; options: string[]; correctAnswer: number; explanation: string }
}
```

Content is written as `LessonContent`, which is `Lesson` without `exam` and `section`, in four arrays: `SAT_MATH`, `SAT_RW`, `GRE_QUANT` and `GRE_VERBAL`. `BY_COURSE` maps `'sat:quant'` and so on to those arrays, and `LESSONS` is built by walking `COURSES` in order and stamping each piece of content with its course's exam and section. The result is 15 lessons, one per topic, in course order. Each has three or four key ideas, two to four example steps, two or three traps, and a check with four options on the SAT and five on the GRE, matching each exam's `option_count` in the backend. `correctAnswer` is a zero-based index.

Slugs. `topicSlug` lowercases a topic and turns every run of non-alphanumerics into one hyphen, so "Problem-Solving and Data Analysis" becomes `problem-solving-and-data-analysis`. `lessonHref(exam, topic)` builds `/learn/<exam>/<slug>`, and `findLesson(exam, slug)` reverses it by slugging every lesson's topic and comparing. "Algebra" exists on both exams; the exam segment keeps them apart.

Progress. `frontend/lib/learnProgress.ts` stores the set of learned lessons as a JSON array under the `localStorage` key `learned_lessons`. Ids come from `lessonId(exam, topic)`, which is `"exam:topic"`, for example `"sat:Algebra"`. This progress is per browser, not per account; nothing is sent to the backend.

Videos. The `Video` shape is the same in `schemas.py` and `client.ts`: `id`, `title`, `channel`, `duration` (a clock string such as "12:04"), `url`, `thumbnail`, `published` (a date) and `trusted`. Nothing about videos is stored in the database.

10.4 The server side

`backend/src/api/learn.py` is mounted at `/api/learn` and has one route, `GET /videos`. It requires a signed-in user through `get_current_user` and validates its query: `exam` must match `^(sat|gre)$`, `section` must match `^(quant|verbal)$`, `topic` is 2 to 80 characters and `limit` is 1 to 12 (default 6). It awaits `find_topic_videos` and returns `VideoResponse`; any exception becomes a 502 "Video search failed: ...".

`backend/src/services/videos.py` does the work in four stages.

1. Cache. The key is `exam:section:topic` with the topic lowercased. `_cache` is a module-level dict of (timestamp, ranked list); a hit younger than `CACHE_TTL_SECONDS` (24 hours) is sliced to `limit` and returned. The limit is not part of the key, so the dashboard's request for three and the lesson page's request for six share one search.

2. Queries. The exam and section display names come from `EXAMS` in `question_sources.py`. First comes a dedicated GregMat search, `"GregMat GRE <topic>"` for the GRE or `"GregMat <topic>"` for the SAT; only results whose channel is GregMat and whose title mentions the topic are kept. Then up to three general queries run in order: `"<exam> <topic> lesson"`; for the SAT `"Khan Academy SAT <topic>"`, for the GRE `"GRE <topic> strategy"`; and `"<exam> <section> <topic> explained"`. The loop stops early once 15 non-GregMat videos are collected. Queries run one at a time on purpose, because parallel queries get rate limited. Results are deduplicated by video id.

3. Providers. `_search` builds a chain and returns the first provider's non-empty result:

ORDER	PROVIDER	ENABLED BY	NOTES
1	YouTube Data API	<code>YOUTUBE_API_KEY</code>	<code>search</code> with <code>videoEmbeddable=true</code> , 15 results, then one <code>videos</code> call for durations and view counts. ISO durations become clock strings with <code>_iso_to_clock</code> .
2	Serper	<code>SERPER_API_KEY</code>	POST <code>https://google.serper.dev/videos</code> , 15 results. No view counts, so views are 0.
3	DuckDuckGo	always	<code>DDGS().videos</code> through the <code>ddgs</code> package, run in a thread with <code>asyncio.to_thread</code> . Up to 2 attempts with a 2 second backoff, since rate limiting shows up as an error rather than a status.

An `httpx.HTTPError` from YouTube or Serper is logged and the chain moves on. Every result passes through `_video`, which keeps only URLs that match the YouTube id pattern (so every result can be embedded), builds the canonical watch URL and the `hqdefault.jpg` thumbnail, parses the duration into minutes and marks the video `trusted` if its channel matches `TRUSTED_CHANNELS`: Khan Academy, College Board, ETS, Official GRE, GregMat, Magoosh, Scalar Learning, Princeton Review, Kaplan, PrepScholar, Supertutor, 1600.io and Target Test Prep. Single-word names must match a whole word of the channel; multi-word names match as substrings.

4. Ranking. `_score` adds:

- 3 for a trusted channel;

- 2 for a length of 3 to 25 minutes (fits in a study session), 0.5 for anything else up to 60 minutes, and minus 2 for longer (multi-hour livestreams); no change if the duration is unknown;
- $\log_{10}(\text{views}) / 3$, so a million views is worth 2.

The list is sorted by score, then up to `GREGMAT_SLOTS` (3) GregMat videos that mention the topic are moved to the front. Only a non-empty result is cached, so a rate-limited search is retried on the next request rather than remembered as "nothing found".

10.5 The client

The hub. `frontend/app/learn/page.tsx` renders inside `DashboardLayout`. It sets the exam from the profile once the account loads (SAT if there is none), and the SAT and GRE chips switch it with `aria-pressed`. It fetches `getUserStats()` for weak and strong topics and reads `useLearned()`. A lime progress bar shows "*n* of *m* learned" for the current exam.

`FeaturedLesson` picks, in order, the first lesson whose topic is weak, the first lesson not yet learned, or the first lesson. It renders a cobalt card with "Recommended: your weakest topic" or "Recommended next", an "Open lesson" button and, on the right, the single best video for that topic from `useTopicVideos(..., 1)`. Below it, each course for the exam gets a heading and a grid of lesson cards with the summary, the number of key ideas and badges for "Needs work", "Strong" and "Learned".

The lesson. `LessonPage` resolves the lesson with `findLesson` from the route params and shows "That lesson doesn't exist." for an unknown slug. Otherwise it renders `LessonView`, keyed by the lesson id so all local state resets when you follow "Next". The article has six parts: a back link to the course, the title and summary; key ideas as glass cards; the worked example; the traps on coral-soft; the `QuickCheck` on cobalt-soft; and a footer with "Mark as learned", "Practice in a game" and a "Next: *topic*" link to the following lesson in the same course.

The worked example starts with one step visible. The button reads "Show next step" until all steps are shown, then "Show answer", and the answer replaces the button in a lime box. "Practice in a game" calls `setExamPrefs` with the lesson's exam and section, then pushes to `/games`.

`VideoPanel` is passed to the shell as its `aside`, and rendered again inline with `xl:hidden` so narrow screens still get it. It shows a skeleton while loading, `VideosUnavailable` if there is nothing, or the selected video with its title, `VideoMeta` and a list of the others as `VideoRow`s. Picking another video sets `started`, so the new one plays immediately. A "Wider"/"Narrow" toggle widens the aside from 320 px to 440 or 560 px; the choice is remembered in `localStorage` under `lesson_video_wide`.

Video components. `frontend/components/learn/Videos.tsx` holds `useTopicVideos(exam, section, topic, limit)`, which returns `{ videos, failed }` where `videos` is `null` while loading. It keeps a module-level `Map` from `exam:section:topic:limit` to the request promise, so revisiting a lesson in the same visit costs nothing, and deletes a failed promise so the next visit retries. `VideoPlayer` shows the thumbnail with a lime play button and swaps in a `youtube-nocookie.com` embed with `autoplay=1&rel=0` when clicked; no YouTube code loads until then. `VideoMeta` prints the channel, a "Prep channel" badge for trusted ones and the duration. `VideosUnavailable` says either "Video search is unavailable right now." or "No lesson videos found for this topic yet." and links to a YouTube search built by `youtubeSearchUrl`.

The quick check. `QuickCheck` takes a lesson's `check`. Options are lettered A to E. Once one is picked, all are disabled, the correct one turns lime with a check mark, a wrong pick turns coral, and the rest fade. "Correct." or "Not quite." and the explanation appear in a polite live region, with "Try again" to reset. The result is not recorded anywhere.

10.6 How it works

A student with SAT on their profile who missed most Advanced Math questions in their last game opens `/learn`. The hub sets the exam to SAT, the stats call returns `weak_topics` containing "Advanced Math", and `FeaturedLesson` features it as their weakest topic. Its video hook asks `/api/learn/videos?exam=sat§ion=quant&topic=Advanced%20Math&limit=1`.

On a cold cache the backend runs the `GregMat` query and up to three general queries through the provider chain, ranks every candidate it found, caches the list and returns the top one. The student opens the lesson. `VideoPanel` asks for six; the client has no entry for limit 6, so it calls the server, which answers from its cache instantly. The student steps through the example, answers the check, presses "Mark as learned" (now `"sat:Advanced Math"` is in `localStorage`) and then "Practice in a game", which leaves their exam prefs on SAT Math.

10.7 How to add or edit a lesson

1. **Editing text** is a change to the right array in `frontend/lib/lessons.ts`. Aim for four ideas and three short traps, keep the SAT at four options and the GRE at five, and make sure `check.correctAnswer` is the zero-based index of the right option. Use real typographic characters (the file already uses `-`, `x`, `²` and curly quotes) rather than markup; the fields render as plain text.
2. **Adding a topic to an existing course** takes two edits. Add the topic name to that course's `topics` in `frontend/lib/courses.ts`, spelled exactly as the question bank spells it, so weak-topic matching and the dashboard's focus topics work. Then add a `LessonContent` object with the same `topic` to the course's array in `lessons.ts`. Its position in the array is its position in the course and decides what "Next" points to.
3. **Check the slug.** Run the topic through `topicSlug` in your head: it must be unique within the exam. Two topics that differ only in punctuation would collide.
4. **Adding a course** means a new `COURSES` entry, a new content array and a new `BY_COURSE` key of the form `exam:section`. A course with no `BY_COURSE` entry produces no lessons but still shows its heading on the hub with an empty grid. A new section also needs the backend's `section` pattern in `learn.py` and an entry in `EXAMS` in `question_sources.py`.
5. **Check the videos.** Open the lesson and look at what the search returns. If the results are poor, the levers are the query templates and `TRUSTED_CHANNELS` in `videos.py`, not the lesson. Restart the backend to clear its cache.
6. **Run** `npm run type-check` in `frontend/`. The `Lesson` type catches missing fields; it does not catch a wrong `correctAnswer`, so read the check once more.

10.8 Edge cases and tests

- **No keys configured.** With neither `YOUTUBE_API_KEY` nor `SERPER_API_KEY`, every search goes to DuckDuckGo, which rate limits. A cold lesson can then take several seconds (up to four searches, each

with a possible 2 second backoff) and may return nothing; the panel says no videos were found and offers the YouTube search link. Nothing is cached, so the next visit tries again.

- **Serverless cache.** `_cache` lives in process memory with no size bound. On a serverless deployment each instance has its own and loses it on a cold start, so the one-day TTL is a ceiling, not a promise.
- **Learned is per browser.** Clearing site data or switching devices resets it, and two people on one browser share it.
- **Hub counts.** Each course heading shows `course.topics.length` while the grid shows the lessons that exist for that section. They agree today because every topic has a lesson; if they ever diverge, the count will be wrong.
- **Weak topics across exams.** Status badges match by topic name only. A weak "Algebra" from GRE games will also mark the SAT Algebra lesson as "Needs work".
- **Answer position.** Nine of the 15 quick checks have option A as the answer. A student who notices will stop reading the options. When you write or edit a check, vary the position.
- **Unknown slug.** `/learn/sat/nope` renders the not-found card inside the shell; it does not return a 404 status.

There are no automated tests. `topicSlug`, `findLesson`, `_score`, `_iso_to_clock`, `_duration_minutes` and `_mentions` are pure functions and the obvious first candidates.

10.9 Study notes

READ IN THIS ORDER

1. `frontend/lib/courses.ts` , the four courses and their topics.
2. `frontend/lib/lessons.ts` , the type, one lesson, then the bottom 20 lines.
3. `frontend/app/learn/[exam]/[topic]/page.tsx` , the lesson view and video panel.
4. `frontend/components/learn/Videos.tsx` and `QuickCheck.tsx` .
5. `frontend/app/learn/page.tsx` , the hub and `FeaturedLesson` .
6. `backend/src/api/learn.py` , then `backend/src/services/videos.py` from `find_topic_videos` upward.

CHECK YOURSELF

Why is the video cache key on the server free of `limit` while the client's includes it?

The server caches the whole ranked list and slices it per request, so every limit can share one search. The client caches request promises, and a promise for one video cannot answer a request for six.

Why are the general queries run one after another instead of with `asyncio.gather` ?

The free providers rate limit bursts. The comment in `find_topic_videos` says so, and serial queries also let the loop stop early once it has 15 non-GregMat videos.

What must be true of a new topic name for the dashboard to recommend its lesson?

It must appear in the course's `topics` and match, character for character, the topic the games report in `question_attempts` , since `weak_topics` comes from those reports.

A DuckDuckGo search hits a rate limit. What does the student see, and when is the search tried again?

The provider returns an empty list, the endpoint returns no videos, and the panel shows "No lesson videos found for this topic yet." with a YouTube search link. The empty result is not cached, so the next request searches again.

Where is "Mark as learned" stored, and what is its id for GRE Geometry?

In `localStorage` under `learned_lessons` , as `"gre:Geometry"` .

11 Dashboard, statistics and sessions

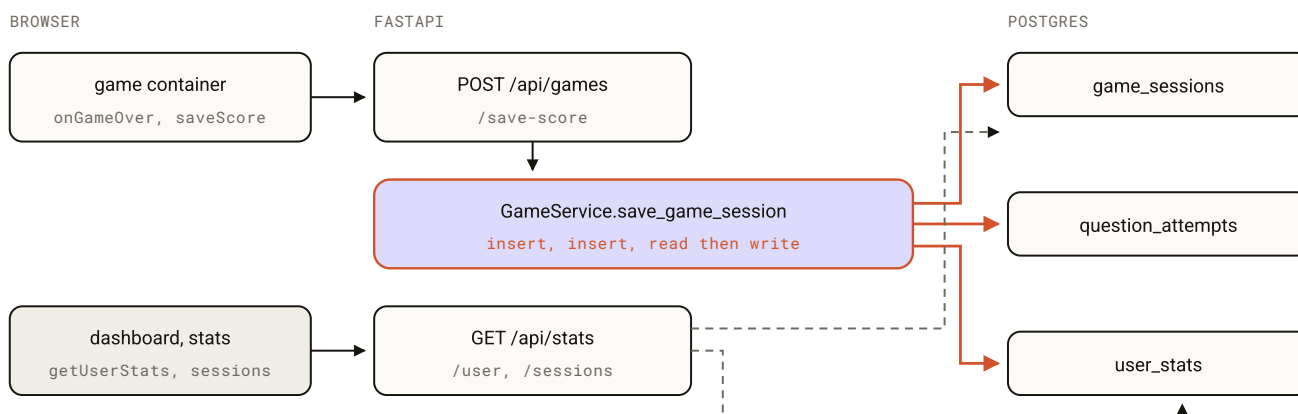
Every game and every mock exam ends the same way: the client posts one analytics object, the backend writes a session row, one row per question attempted, and an updated running total for the player. The dashboard and the statistics page are two views over those rows. This chapter follows a finished game into the database and back out onto the screen, including how the weak and strong topic lists are worked out and the places where the numbers are rougher than they look.

11.1 Intent

The dashboard answers one question when you sign in: what should I do next? It names a focus topic, offers a game to practice it and a lesson to study it, and shows videos on it. Everything else on the page (the numbers, the recent sessions, the courses, the games you have not tried) supports that one suggestion.

The statistics page answers a different question: how am I doing? It shows totals, a right and wrong bar for every answer so far, an accuracy trend, the weak and strong topics, and a record per game.

The storage design behind both is deliberately simple. Raw history lives in `game_sessions` and `question_attempts`; a single `user_stats` row per player holds running totals so neither page has to aggregate history on every load.



One write path and two read endpoints. The write is three separate statements, not a transaction; the reads never touch `question_attempts`.

11.2 Files and folders

```

frontend/app/dashboard/page.tsx
frontend/app/stats/page.tsx
frontend/lib/sessions.ts
frontend/lib/gameMeta.ts
frontend/lib/api/client.ts

```

```

greeting, Up next, numbers, videos, sessions, aside
totals, answer bar, AccuracyTrend, TopicList, by game, r
GameSession, useRecentSessions, recordsByGame, timeAgo
gameTitle, including 'mock-exam'
saveScore, getUserStats, getRecentSessions

```

```

frontend/components/*GameContainer.tsx  each game's onGameOver calls saveScore
frontend/app/mock/page.tsx             mock exams save as game 'mock-exam'

backend/src/api/games.py                POST /api/games/save-score
backend/src/api/stats.py                GET /api/stats/user, GET /api/stats/sessions
backend/src/services/game_service.py    GameService: session, attempts, user_stats
backend/src/services/supabase_agent_ops.py SupabaseAgentOps: reads for the question agent
backend/src/models/schemas.py          GameAnalytics, SaveScoreRequest, UserStatsResponse, Game
backend/database/schema.sql             the three tables, indexes, RLS
backend/database/add_user_id_to_attempts.sql question_attempts.user_id

```

11.3 Model

The analytics object. Every game builds the same shape, checked on the server by `GameAnalytics` in `schemas.py`:

FIELD	TYPE	MEANING
<code>gameId</code>	str	The game, repeated inside the analytics
<code>score</code>	int	Points for the round
<code>accuracy</code>	float	Percent, 0 to 100, as the game computed it (the server ignores it)
<code>correctAnswers</code> , <code>wrongAnswers</code>	int	Counts for the round
<code>questionAttempts</code>	list	<code>questionId</code> , <code>topic</code> , <code>difficulty</code> , <code>isCorrect</code> , <code>timeSpent</code> (ms)
<code>topicPerformance</code>	dict	Per topic: <code>correct</code> , <code>total</code> , <code>accuracy</code> (percent)
<code>streakInfo</code>	dict	Free-form; the server reads <code>maxStreak</code>
<code>averageResponseTime</code>	int	Milliseconds

`SaveScoreRequest` wraps it as `{ gameId, analytics }`. The outer `gameId` is the one stored.

Tables. `backend/database/schema.sql` creates three tables, all with row level security and all cascading from `auth.users`.

- `game_sessions`: a uuid `id`, `user_id`, `game_id`, `score`, `accuracy` as `DECIMAL(5, 4)` (a fraction from 0 to 1), `correct_answers`, `wrong_answers`, `max_streak`, `average_response_time` and `created_at`. Indexed on `user_id` and on `created_at` DESC.
- `question_attempts`: a uuid `id`, `session_id` (cascading from the session), `question_id` as an integer, `topic`, `difficulty`, `is_correct`, `time_spent` and `created_at`.
`add_user_id_to_attempts.sql` adds a denormalized, not-null `user_id`, two indexes (`user_id`, and `user_id, topic`) and rewrites the policies to use it, so per-user topic queries skip the join.
- `user_stats`: one row per user (`user_id` is unique) with `total_games_played`, `total_score`, `total_questions_answered`, `total_correct`, `total_wrong`, `overall_accuracy` (a fraction), `favorite_game`, `weak_topics` and `strong_topics` as `TEXT[]`, and `updated_at`.

On the client, `frontend/lib/sessions.ts` declares `GameSession` with the same fields as `GameSessionResponse`, and `GameRecord` (`gameId`, `plays`, `bestScore`, `lastPlayed`) for the per-game rollup.

11.4 The server side

Saving. `POST /api/games/save-score` in `backend/src/api/games.py` takes a `SaveScoreRequest`, resolves the user with `get_current_user` and calls `GameService(db).save_game_session(user_id, game_id, analytics)`. A failure is a 500 with the error text; success returns `{ success: true, sessionId }`.

`save_game_session` in `backend/src/services/game_service.py` does three things in order, each its own request to Supabase:

1. Insert the `game_sessions` row. `accuracy` is recomputed by `_accuracy` as `correct / (correct + wrong)`, or 0 when there were no answers, so the game's own percentage is never trusted. `max_streak` is `streakInfo.get("maxStreak", 0)`.
2. If there are attempts, insert all of them in one call with the new `session_id` and the `user_id`.
3. Call `_update_user_stats`.

Any exception is caught and returned as `{ success: False, error }`.

Weak and strong topics. `_update_user_stats` first classifies this round's topics from `topicPerformance`, working from the counts rather than the game's percentage (the comment notes that games report it as a percent). A topic with at least one question is **weak** if fewer than half were right, **strong** if at least 80 percent were, and neither in between.

If the player has no `user_stats` row yet, it inserts one with this round's totals and lists. Otherwise it adds this round to each total, recomputes `overall_accuracy` as `total_correct / total_questions_answered`, and merges the lists:

```
merged_weak   = (existing_weak   - this_round_strong) | this_round_weak
merged_strong = (existing_strong - this_round_weak)   | this_round_strong
```

So a topic moves from weak to strong (or back) as soon as one round says so, and a topic that lands in the middle band keeps whatever label it had. The lists are sets converted to Python lists, so their order is arbitrary.

FAVORITE GAME IS NEVER COMPUTED.

`favorite_game` exists in the table, the response model and the dashboard's types, but no code writes it, so it is always `null`. The dashboard falls back to the most recently played game (section 11.6). To make it real, compute the most-played `game_id` from `game_sessions` in `_update_user_stats` and write it with the other totals.

Reading. `backend/src/api/stats.py` has two routes. `GET /api/stats/user` returns the caller's `user_stats` row as `UserStatsResponse`, or an all-zero response with empty lists and an empty

`updated_at` if there is no row. `GET /api/stats/sessions?limit=` returns the newest `limit` sessions (default 10), newest first, as a list of `GameSessionResponse`. Neither reads `question_attempts`.

The agent's view. `SupabaseAgentOps` in `backend/src/services/supabase_agent_ops.py` is a set of static methods over the same tables for the AI question agent in `agent.py`. Two are in use:

`get_user_performance` reads `user_stats` and reports accuracy as a percent; `get_topic_performance` reads every `question_attempts` row for the user and aggregates `total`, `correct`, `accuracy` and `avg_time` per topic. That is the only place per-attempt history is read. The class also has `save_game_session`, `save_question_attempts` and `update_user_stats`, a second write path that expects a different, snake_case `game_data` dictionary and never updates topic lists. Their only caller, `ExamLearningAgent.update_performance`, is never called, so these three methods are dead code. Games save through `GameService` only.

11.5 The client

Saving. Each game container (`ZombieGameContainer`, `WhackAMoleGameContainer`, `CarnivalGameContainer`, `SubwaySurfersGameContainer`, `SquidGameContainer`, `PacManGameContainer`) registers an `onGameOver` handler that shows the results screen (the four arcade-style games also record a local high score) and then calls `apiClient.saveScore(gameId, analytics)`, logging any error to the console. The mock exam in `frontend/app/mock/page.tsx` builds the same object itself, with a score of 100 per correct answer and the game id `'mock-exam'`. `saveScore` posts `{ gameId, analytics }` to `/api/games/save-score`.

Reading. `getUserStats()` returns the stats object, or `null` on any error or an empty body. `getRecentSessions(limit)` returns an array, or `[]` on error. `useRecentSessions(limit)` in `sessions.ts` wraps the second in state that starts as `null` (loading).

`recordsByGame(sessions)` walks the sessions, which arrive newest first, and builds one `GameRecord` per game in a `Map`: plays counted, best score kept, and `lastPlayed` taken from the first (newest) session seen. Map insertion order makes the result most recently played first. `timeAgo(iso)` formats "just now", "*n* min ago", "*n* h ago", "yesterday" or "*n* days ago", rounding at each step.

11.6 How it works

The dashboard, `frontend/app/dashboard/page.tsx`, loads the account, the stats and the last 20 sessions, and derives everything else.

- **Greeting.** "Morning" before noon, "Afternoon" before 6 pm, else "Evening", plus the first word of the display name (or the email's local part). Under it, a test-day countdown if the profile has a future date, and the daily goal in minutes. An account without a profile gets a lime "Finish your profile" card.
- **The exam and its courses.** The exam comes from the profile, defaulting to SAT; `courses` are the two `COURSES` for that exam.
- **Focus topics.** `weak_topics` filtered to topics that belong to this exam's courses. `focusTopic` is the first of them, or the first topic of the first course (SAT Algebra, GRE Arithmetic) when there are none.
- **Next game.** `favorite_game` if it is a known game, else the game of the most recent session, else `'whackamole'`. Since `favorite_game` is always null today, this is in practice "the last thing you played".

- **Up next.** A cobalt-soft card names the focus topic, says whether it is your weakest topic or a good place to start, and offers "Play *game*" and "Study the lesson". Playing calls `practice(course)`, which stores the course's exam and section with `setExamPrefs` and pushes to `/games/<nextGameId>/select`. The game's cover fills the right side.
- **Numbers.** Games played, accuracy (the stored fraction times 100) and questions answered, each with a skeleton until stats arrive.
- **Watch.** `RecommendedVideos` asks for three videos on the focus topic through `useTopicVideos` and hides itself if none come back.
- **Recent sessions.** Up to eight `SessionCard`s, tinted lime-soft at 80 percent accuracy or more, white at 50 or more, coral-soft below, with right and wrong counts, best streak, an accuracy bar and the score. With no sessions, a dashed empty state links to the games.
- **The aside.** The exam's courses as tiles (each starts `practice`), up to four focus topics with a "Practice" button, and "Haven't tried yet": up to four games whose id is not among the last 20 sessions, drawn with `GameCover`.

The statistics page, `frontend/app/stats/page.tsx`, loads stats and the last 50 sessions. Until both arrive it shows four skeleton tiles; if `total_games_played` is 0 (or stats failed) it shows "Nothing to show yet". Otherwise:

- Four tiles: games played, accuracy, questions answered, and best streak, which is the maximum `max_streak` over the 50 loaded sessions.
- "Every answer so far": a lime segment over a coral bar, sized by `overall_accuracy`, labelled with the totals.
- `AccuracyTrend`, when there are at least two sessions: up to `TREND_LENGTH` (12) bars, oldest on the left, colored lime, cobalt or coral by the same 80 and 50 percent thresholds, each with a `title` naming the game.
- Two `TopicList`s, "Work on these" and "Going well". Their empty hints state the rules in plain words: under half right, and at least four in five.
- "By game" from `recordsByGame`, each linking to that game's select page, and the newest 15 sessions as a list.

11.7 Edge cases and tests

There are no automated tests for this area. These are the behaviors to know.

- **Mock exams are sessions too.** A mock exam is stored with `game_id` `'mock-exam'`. When picking "Up next", the dashboard skips any session whose id is not in `games`, so a mock exam never becomes the suggested game. The stats page's "By game" card still links mock exams to `/games/mock-exam/select`, which renders "That game doesn't exist"; it should link to `/mock` instead.
- **Topic labels are jumpy.** Labels come from single rounds, so one wrong answer in a topic marks it weak until a later round scores 80 percent or better on it. They are not computed from the attempt history, even though `get_topic_performance` could.
- **No transaction.** If the attempts insert or the stats update fails after the session insert, the route returns 500 but the session row stays, and the totals miss that round. The client does not retry, so nothing is duplicated, but nothing is repaired either.

- **Lost updates.** `_update_user_stats` reads the row, adds in Python and writes it back. Two games finishing at the same moment for one player (two tabs) can overwrite each other. A Postgres function doing the increment in one statement would fix it.
- **Silent failures.** `getUserStats` returns `null` on error. The stats page then shows "Nothing to show yet"; the dashboard's number tiles keep their skeletons, because they treat `null` as "still loading".
- **Windows, not history.** "Haven't tried yet" looks at the last 20 sessions and "Best streak" at the last 50, so a game played long ago can reappear as untried and an old streak can drop off.
- **Unbounded limit.** `/api/stats/sessions` accepts any `limit`; the clients ask for 20 and 50. Add an upper bound with `Query(..., le=...)` if you expose it further.
- **Question ids are integers.** `question_attempts.question_id` and `QuestionAttempt.questionId` are both integers; a question source with string ids must map them first or the whole save fails validation.
- **Leftovers.** `games.py` declares an unused `security = HTTPBearer()`, and `backend/src/api/games.ts`, `backend/src/services/gameScoreService.ts` and `backend/src/types/database.ts` are TypeScript from an earlier design that the FastAPI app never loads.

To check the path by hand, finish a short game, then compare the new `game_sessions` row, its `question_attempts` and the `user_stats` row in the Supabase table editor against the results screen. A good unit test would call `_update_user_stats` with a stub client and assert the merged lists for a topic that moves from weak to strong.

11.8 Study notes

READ IN THIS ORDER

1. `backend/database/schema.sql` , then `add_user_id_to_attempts.sql` .
2. `backend/src/models/schemas.py` , `GameAnalytics` through `GameSessionResponse` .
3. `backend/src/services/game_service.py` , all of it.
4. `backend/src/api/games.py` and `backend/src/api/stats.py` .
5. One game container's `onGameOver` , for example `frontend/components/WhackAMoleGameContainer.tsx` .
6. `frontend/lib/sessions.ts` .
7. `frontend/app/dashboard/page.tsx` , then `frontend/app/stats/page.tsx` .
8. `backend/src/services/supabase_agent_ops.py` , to see the read side the agent uses.

CHECK YOURSELF

A round has 3 Geometry questions and the player gets 2 right. Geometry was weak before. What is it after?

Still weak. Two out of three is about 67 percent, which is neither under 50 nor at least 80, so this round adds Geometry to neither list and the existing label stands.

Why does the server recompute accuracy instead of storing the game's value?

Games report accuracy as a percent while the columns hold a fraction from 0 to 1, and the counts are the source of truth. `_accuracy` derives it from `correctAnswers` and `wrongAnswers` .

The dashboard suggests the game you played last, not the one you play most. Why?

`favorite_game` is never written, so the fallback runs: the first record from `recordsByGame` , which is the most recent session's game.

Which endpoint reads `question_attempts` ?

None. The stats routes read `user_stats` and `game_sessions` only; the attempts are read by `SupabaseAgentOps.get_topic_performance` for the question agent.

What happens to a player's totals if `saveScore` fails because their token expired?

Nothing is written. The request fails with 401, the container logs the error, and the round exists only on the results screen and in the local arcade high score.

12 Mock exams and your own material

Two features sit beside the games. The mock exam is a timed section in test-day format: one clock, no feedback until the end, a score report after. "My material" lets a student upload a practice test or notes, review the questions found in it, and approve the ones that should appear in their games and mock exams. Both reuse the question engine of chapter 7 rather than adding their own.

12.1 Intent

Games are good for repetition, but a student also needs to feel the real exam's pace and to see an honest score afterwards. The mock exam gives them that with the same questions the games use. It lives entirely in the browser: one page, three screens (setup, exam, results), and a single call to save the result alongside game sessions so it feeds statistics and weak topics.

Students also have their own material: a teacher's handout, a practice test PDF, class notes. The materials feature turns that into questions in the standard game format. The design rule is that nothing from an upload reaches a game until the student has looked at it. Every question starts as `pending`, and a question cannot be approved without a correct answer.

12.2 Files and folders

<code>frontend/app/mock/page.tsx</code>	Setup, Exam, Results, MockExamPage
<code>frontend/components/exam/Calculator.tsx</code>	the on-screen calculator (quant only)
<code>frontend/app/materials/page.tsx</code>	upload form, list, QuestionReview
<code>frontend/lib/materials.ts</code>	Material, CustomQuestion
<code>frontend/lib/api/client.ts</code>	uploadMaterial, getMaterials, getMaterialQuestions, updateCustomQuestion, deleteCustomQuestion, approveAllQuestions, deleteMaterial, saveScore
<code>frontend/middleware.ts</code>	/mock and /materials need the auth_token cookie
<code>backend/src/api/materials.py</code>	the /api/materials routes
<code>backend/src/services/material_parser.py</code>	extract_text, parse_questions, questions_from_llm, merge_questions
<code>backend/src/services/custom_questions.py</code>	load_custom_questions, to_game_question
<code>backend/database/add_materials.sql</code>	materials, custom_questions, indexes, RLS

`add_materials.sql` is a separate migration. It is not part of `schema.sql` and `reset.sql` neither drops nor creates these tables, so a new database needs it run on its own.

12.3 Model

TWO TABLES

```
-- backend/database/add_materials.sql (abridged)
CREATE TABLE IF NOT EXISTS materials (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  exam TEXT NOT NULL CHECK (exam IN ('sat', 'gre')),
  section TEXT NOT NULL CHECK (section IN ('quant', 'verbal')),
  created_at TIMESTAMP WITH TIME ZONE ...
);

CREATE TABLE IF NOT EXISTS custom_questions (
  id BIGSERIAL PRIMARY KEY,
  material_id UUID NOT NULL REFERENCES materials(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  exam TEXT NOT NULL,
  section TEXT NOT NULL,
  question TEXT NOT NULL,
  passage TEXT NOT NULL DEFAULT '',
  options JSONB NOT NULL,
  correct_answer INTEGER,           -- NULL until found or picked
  topic TEXT NOT NULL DEFAULT 'My material',
  difficulty TEXT NOT NULL DEFAULT 'medium',
  explanation TEXT NOT NULL DEFAULT '',
  origin TEXT ... CHECK (origin IN ('extracted', 'generated')),
  status TEXT ... DEFAULT 'pending' CHECK (status IN ('pending', 'approved')),
  created_at ...
);
```

A material is a name, an owner, an exam and a section. It holds no text and no file: the upload is read once, turned into rows in `custom_questions`, and discarded. There is no storage bucket. If the parser misses a question, the only way to get it is to upload again.

Three indexes support the reads: by `materials.user_id`, by `custom_questions.material_id`, and a compound one on `(user_id, exam, section, status)`, which is exactly the filter `load_custom_questions` uses. Both tables have RLS enabled with an owner-only policy for all operations.

OWNERSHIP IS ENFORCED IN PYTHON.

The backend's `Database.get_client` prefers `SUPABASE_SERVICE_ROLE_KEY`, which bypasses RLS. Every query in `materials.py` therefore filters by `user_id` itself, and `_own_material` returns 404 for a material that is not the caller's. The RLS policies are a backstop. If the backend ever ran with only the anon key, the inserts would fail, because `auth.uid()` would be null.

CLIENT TYPES

`frontend/lib/materials.ts` mirrors the rows. `Material` adds two optional counts the list route computes, `question_count` and `approved_count`. `CustomQuestion` has `correct_answer: number | null`, `origin: 'extracted' | 'generated'` and `status: 'pending' | 'approved'`. It omits `topic` and `difficulty`, which the page never shows.

LIMITS

CONSTANT	FILE	VALUE	EFFECT
<code>MAX_UPLOAD_BYTES</code>	<code>materials.py</code>	4 MiB	413 above it; Vercel rejects bodies over 4.5 MB
<code>MAX_FILE_BYTES</code>	<code>materials/page.tsx</code>	4 MiB	The same check before sending
<code>MIN_TEXT_CHARS</code>	<code>materials.py</code>	40	400 "No readable text found"
<code>MAX_TEXT_CHARS</code>	<code>material_parser.py</code>	400,000	Truncates text read from a file
<code>LLM_CHUNK_CHARS</code>	<code>material_parser.py</code>	12,000	Text per model call
<code>MAX_LLM_CHUNKS</code>	<code>material_parser.py</code>	4	The model reads at most the first 48,000 characters

12.4 The server side

ROUTES

METHOD AND PATH	DOES
<code>POST /api/materials</code>	Multipart upload: read, parse, maybe call the model, insert material and questions
<code>GET /api/materials</code>	The caller's materials, newest first, with question and approved counts
<code>GET /api/materials/{material_id}/questions</code>	One material's questions, ordered by id
<code>PATCH /api/materials/questions/{question_id}</code>	Edit a question or change its status
<code>DELETE /api/materials/questions/{question_id}</code>	Delete one question
<code>POST /api/materials/{material_id}/approve-all</code>	Approve every question that has an answer
<code>DELETE /api/materials/{material_id}</code>	Delete a material and its questions

Every route depends on `get_current_user`, so all of them require a valid bearer token. The router is mounted at `/api/materials` in `backend/src/main.py`.

UPLOAD

`upload_material` takes form fields `exam` and `section` (both `Literal`), `name`, `text`, `generate` and an optional `file`. If a file is present it reads all of it, checks the size, and replaces `text` with `extract_text(filename, content)`; pasted text is ignored in that case. An empty name falls back to the file name, and later to "Untitled material". Then:

1. Text shorter than 40 characters after trimming is a 400 that suggests the PDF is scanned.
2. `parse_questions(text)` runs the rule-based parser.
3. If `generate` is set or the parser found nothing, `questions_from_llm` runs too.
4. `merge_questions` keeps every parsed question and adds model questions whose first 60 normalized characters are new.
5. No questions at all is a 422 that explains the numbered format.
6. One `materials` row is inserted, then all questions in one insert, each with the material's exam and section, topic defaulting to "My material" and difficulty to medium. The response is the material and the saved rows.

READING FILES

`extract_text` treats a file as PDF if its name ends in `.pdf` or its first five bytes are `%PDF-`, and reads it with `pypdf`'s `PdfReader`, joining each page's `extract_text()`. There is no OCR, so a scanned PDF yields little or no text. Anything else is decoded as UTF-8 with `errors="replace"`. The file input in the browser accepts `.pdf`, `.txt`, `.md`, `application/pdf` and `text/plain`, but the server does not check the type: a Word document would be decoded as noise and most likely fail with "No questions found".

THE PARSER

`parse_questions` reads line by line with five regular expressions.

PATTERN	MATCHES
<code>QUESTION_START</code>	<code>1.</code> , <code>12)</code> , <code>3:</code> or <code>Question 4.</code> followed by text; up to three digits
<code>OPTION_START</code>	<code>A)</code> , <code>b.</code> , <code>(C)</code> followed by text; letters A to E
<code>ANSWER_LINE</code>	<code>Answer: C</code> , <code>Correct answer is (B)</code>
<code>EXPLANATION_LINE</code>	<code>Explanation:</code> , <code>Solution:</code> , <code>Rationale:</code>
<code>ANSWER_KEY_HEADING</code>	A line that is just <code>Answer Key</code> or <code>Answers</code>

An answer key heading ends the questions: everything after it is scanned with `ANSWER_KEY_ENTRY` for pairs like `1. C` or `2-D`. Options must arrive in order (A, then B, then C), so a line that happens to start with a later letter, such as "C. Smith wrote", is not taken as a choice. A new numbered line starts a new question only once the current one has options, so a numbered list inside a stem stays part of it. Continuation lines join the stem, the last option or the explanation, depending on what came last. A question survives with two to five options. Its answer comes from its own answer line, else the key, else it is `None`. Parsed questions are `origin: "extracted"` and have no topic or difficulty.

THE MODEL

`questions_from_llm` imports the agent's `get_llm_client` lazily, so the parser still works where the agent's dependencies are missing, and returns an empty list when `OPENROUTER_API_KEY` is not set. It splits the text into 12,000-character chunks, keeps the first four, and sends them in parallel with temperature 0.3. The prompt asks the model to extract every multiple-choice question exactly, to give `correctAnswer` only when the material states it, and, with `generate`, to write up to eight original questions per chunk with `fromMaterial: false`.

`_normalize_llm_question` applies the same honesty test as the question engine: a question counts as extracted only if the model said so and the first 50 normalized characters of its stem appear in the chunk. A generated question without a valid answer is dropped, because it cannot be reviewed into shape. An extracted one without an answer is kept with `correctAnswer: None`.

REVIEW

`QuestionUpdate` accepts `question`, `options`, `correct_answer`, `explanation` and `status`. `update_question` merges the change over the stored row and rejects an answer outside the options (400) and an approval without an answer (400). `approve_all` approves the rows with an answer and returns `approved` and `needs_answer` counts. `delete_material` deletes the questions and then the material; the foreign key cascade would do the first step anyway.

12.5 The client

THE MATERIALS PAGE

`/materials` starts from the exam and section in `getExamPrefs()` and loads the list. The form has exam and section toggles, a file input, a textarea whose placeholder shows the expected format, a name, and a checkbox "Also write new questions from this material" that sets `generate`. `apiClient.uploadMaterial` builds a `FormData` and calls `fetch` directly, so the browser sets the multipart boundary; the generic `request` helper would force a JSON content type.

After an upload the new material opens in the review pane. Each question is a `QuestionReview` card with a "Needs review" or "In your games" chip, a "Written by AI" chip for generated questions, and its choices as buttons. Tapping a choice sets `correct_answer`. "Approve for games" is disabled until an answer exists and toggles to "Take out of games". "Approve all with answers" calls the bulk route. The page only edits the answer and status; editing text through the API is possible but has no UI.

THE MOCK EXAM PAGE

`/mock` holds three components behind one state machine in `MockExamPage`.

Setup picks exam, section and a length. `questionCount` turns the length into a number from `MODULE_QUESTIONS`, the real module sizes:

SECTION	QUICK CHECK	ONE MODULE	FULL SECTION
SAT Math	10	22	44
SAT Reading & Writing	10	27	54
GRE Quantitative	10	12	24
GRE Verbal	10	12	24

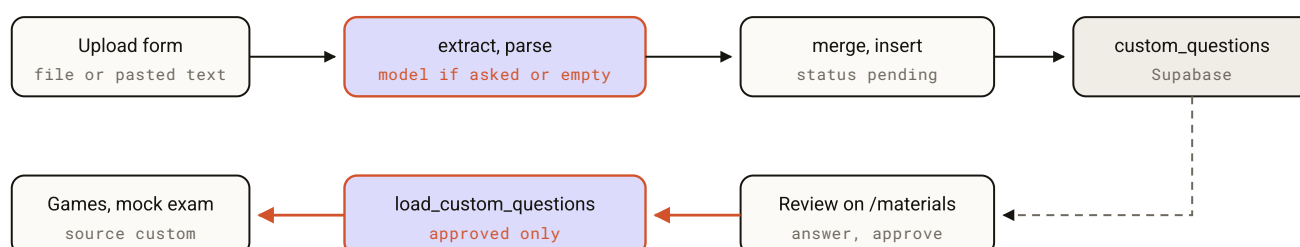
The time shown is the count times the section's `secondsPerQuestion` from chapter 6, in whole minutes. "Start the clock" saves the exam and section to the preferences and calls `fetchAIQuestions(count)` with no pace, so the engine applies no length filter and includes the user's approved custom questions.

Exam renders full screen, outside the dashboard layout. One clock, computed from `Date.now()` every 500 ms, counts down `questions.length * secondsPerQuestion`; at zero it finishes with whatever is answered, read through a ref so the timer sees the latest answers. The clock turns coral under five minutes or 15 percent of the total, whichever is smaller, and can be hidden. Questions with a passage use a two-column layout. The player can mark a question for review, jump through a navigator grid that marks answered, unanswered and flagged questions, open the `Calculator` on quant sections, and finish early through a confirmation that counts unanswered questions. Nothing is graded until the end.

Results shows correct count, accuracy, time used and an estimated score: a straight line across `SCORE_RANGE` (200 to 800 for an SAT section, 130 to 170 for a GRE section), rounded to 10 on the SAT and 1 on the GRE. The page says plainly that real scores are equated and this is only a marker. Below that are topics sorted weakest first and every question in a `<details>` element with the correct answer, the player's answer and the explanation.

On finish, `finish` also builds a `GameAnalytics` object and calls `apiClient.saveScore('mock-exam', ...)`: score is correct answers times 100, each attempt gets the average time per question in milliseconds, and the longest correct streak goes in `streakInfo`. Errors are logged, not shown. The type is imported from `frontend/games/whackamole/types`, a leftover of where the shape was first written.

12.6 How it works



Material becomes rows once, at upload. From then on only the review state changes, and only approved rows are read back into question sets.

1. A student pastes a ten-question worksheet with an "Answer Key" at the end and leaves the checkbox off.
2. `parse_questions` finds ten questions and takes their answers from the key. The model is not called.

3. Ten pending rows are saved. The page opens them; the student clicks "Approve all with answers".
4. The next time a signed-in game or mock exam for that exam and section asks for questions, `load_custom_questions` returns up to half the requested limit from these rows, each with `source: 'custom'` and `sourceName` set to the material's name, so the question panel shows "From" and the material name.

With no `OPENROUTER_API_KEY`, step 2 is all there is: the checkbox does nothing and unnumbered notes produce a 422. Custom questions still appear in the GRE when no other source is available, which makes them the main way to get real GRE questions into a keyless setup.

12.7 Edge cases and tests

- **Pasted text is not capped.** `MAX_TEXT_CHARS` applies only inside `extract_text`; a large paste goes to the parser in full.
- **The whole file is read before the size check.** On Vercel the platform limit stops oversized bodies first; elsewhere the server holds the bytes in memory.
- **Only the first 48,000 characters reach the model.** The parser sees up to 400,000.
- **Unhandled rejections.** On the materials page, delete question, approve all and delete material have no `try`; a failure shows nothing to the user.
- **The file input is not cleared.** After an upload `setFile(null)` resets state, but the uncontrolled input still shows the old file name.
- **Unreachable mock error.** The mock page shows an error when no questions load, but `fetchAIQuestions` always falls back to the bank, which is never empty. With the backend down, a 54-question full section silently becomes a 9-question exam, with the clock scaled to match.
- **The SAT blurb overstates.** Setup says SAT questions are official ones. Custom questions and, when the College Board falls short, model questions can be mixed in.
- **Mock exams change game settings.** Starting one writes its exam and section to `exam_prefs`.
- **No resume.** Exam state is React state; a reload loses the attempt.
- **Delete is idempotent.** `DELETE /api/materials/questions/{question_id}` returns success for an id that does not exist.

There are no automated tests. `parse_questions` is the obvious first target: it is a pure function with many formats to pin, including answer keys, out-of-order letters and multi-line options.

12.8 Study notes

READ IN THIS ORDER

1. `backend/database/add_materials.sql` .
2. `backend/src/api/materials.py` , `upload_material` first.
3. `backend/src/services/material_parser.py` , `parse_questions` with a sample worksheet beside it.
4. `backend/src/services/custom_questions.py` .
5. `frontend/app/materials/page.tsx` and `uploadMaterial` in `frontend/lib/api/client.ts` .
6. `frontend/app/mock/page.tsx` , `MockExamPage` first, then `Exam` , then `Results` .

CHECK YOURSELF

Where is the uploaded PDF stored?

Nowhere. It is read in memory, turned into `custom_questions` rows and discarded. The `materials` row holds only a name, owner, exam and section.

When is the model called during an upload?

When `generate` is true, or when the parser found no questions, and only if `OPENROUTER_API_KEY` is set.

Why can a generated question never be saved without an answer, while an extracted one can?

An extracted question's answer may be in the student's head or answer sheet, so review can supply it. A generated question with no answer has nothing to check against, so `_normalize_llm_question` drops it.

How does the mock exam's clock length follow from the exam?

It is the number of questions actually loaded times the section's `secondsPerQuestion` , so a shorter set gets a shorter clock.

The backend uses the service role key. What stops one user reading another's material?

The routes filter every query by the caller's `user_id` and `_own_material` 404s on foreign ids. RLS would only apply to a client using a user's own token or the anon key.

PART THREE

Under the hood

Every table and policy, every API route, how the two apps are deployed and operated, and how to contribute a change from first commit to merged pull request.

- 13. Data model and security
- 14. API reference
- 15. Deploying and operating
- 16. Contributing

13 Data model and security

Six tables in Supabase Postgres, every one keyed to `auth.users` and every one with Row Level Security switched on. None of that RLS is what protects the data in practice: the browser never talks to Supabase, and the FastAPI backend reaches the database with the service-role key, which bypasses every policy. What protects a user's rows is the backend's own `user_id` filters and the way it checks the bearer token. This chapter reads both honestly.

The first half is the schema: which SQL file is current, every column, index, trigger and policy, and how the tables relate. The second half is the security model: who writes each table, how a JWT becomes a user id in `auth_service.py`, what the browser keeps in storage, CORS, input validation, HTML sanitizing, and a checklist for contributors. Several weaknesses turn up along the way. They are stated plainly, because a contributor who does not know about them will build on them.

13.1 Where the schema lives

backend/database/	
— reset.sql	current: drops and recreates four tables, deletes auth users
— add_materials.sql	current: materials and custom_questions (not in reset.sql)
— schema.sql	historical: the first three tables, 2025
— add_user_id_to_attempts.sql	historical: migration that reset.sql has absorbed
— add_profiles.sql	historical: profiles, now also in reset.sql
backend/src/utils/database.py	the one Supabase client the backend uses
backend/src/services/auth_service.py	signup, login, token decoding
backend/src/models/schemas.py	Pydantic request and response models
backend/src/utils/html_utils.py	allowlist sanitizer for exam question HTML

There is no migration runner. The SQL files are pasted into the Supabase SQL editor by hand. The documented setup, in the README and in `backend/README.md`, is to run `backend/database/reset.sql` and then `backend/database/add_materials.sql` on a new project; together they create every table, trigger and policy. `reset.sql` creates `game_sessions`, `question_attempts`, `user_stats` and `profiles`. `add_materials.sql` creates `materials` and `custom_questions`, additively, with `CREATE TABLE IF NOT EXISTS`. The README's file tree still describes `reset.sql` alone as creating "the full schema"; the setup step below it is the accurate one.

FILE	STATUS	WHAT IT DOES
<code>reset.sql</code> 1	Current, run first	Drops <code>question_attempts</code> , <code>game_sessions</code> , <code>user_stats</code> , <code>profiles</code> and <code>set_updated_at()</code> , runs <code>DELETE FROM auth.users</code> , then recreates the four tables with indexes, two triggers and RLS.
<code>add_materials.sql</code> 1	Current, run second	Creates <code>materials</code> and <code>custom_questions</code> with <code>IF NOT EXISTS</code> , three indexes and two <code>FOR ALL</code> policies. Added with the material uploads feature. Without it every <code>/api/materials</code> route fails.
<code>schema.sql</code> 1	Historical	The original 2025 schema: the first three tables, without <code>question_attempts.user_id</code> , with an index on <code>question_attempts(topic)</code> and one on <code>user_stats(user_id)</code> that <code>reset.sql</code> does not create. Its <code>question_attempts</code> policies join through <code>game_sessions</code> . Only <code>QUICK_TEST.sh</code> still checks that it exists.
<code>add_user_id_to_attempts.sql</code> 1	Historical	Upgraded a <code>schema.sql</code> database: added and backfilled <code>question_attempts.user_id</code> , added the two user indexes, and replaced the join policies with direct <code>auth.uid() = user_id</code> ones. <code>reset.sql</code> has all of that built in.
<code>add_profiles.sql</code>	Historical	The <code>profiles</code> table and its three policies, identical to the copy in <code>reset.sql</code> except that it has no <code>updated_at</code> trigger.

TWO FILES, IN THIS ORDER, AND ONLY ON A NEW PROJECT

Run `reset.sql`, then `add_materials.sql`. Skipping the second leaves every `/api/materials` route failing. `reset.sql` neither creates nor drops the two material tables. Its `DELETE FROM auth.users` does empty them, through their `ON DELETE CASCADE` foreign keys, so running it on an existing project leaves the material tables in place but empty. And never run `reset.sql` against a project with real users: its header says it "wipes all app data AND all auth users".

13.2 The tables

The first four tables below are created by `reset.sql`, the last two by `add_materials.sql`. Every timestamp column defaults to `TIMEZONE('utc', NOW())`. Every `id` that is a UUID defaults to `gen_random_uuid()`. Every foreign key cascades on delete.

GAME_SESSIONS

One row per finished game or mock exam, written by `POST /api/games/save-score`.

COLUMN	TYPE	NOTES
<code>id</code>	UUID	Primary key
<code>user_id</code>	UUID NOT NULL	References <code>auth.users(id)</code>
<code>game_id</code>	TEXT NOT NULL	A game id from <code>frontend/lib/games.ts</code> (<code>zombie</code> , <code>whackamole</code> , <code>carnival</code> , <code>subway-surfers</code> , <code>squid-game</code> , <code>pac-man</code>) or <code>mock-exam</code> . Not constrained.
<code>score</code>	INTEGER NOT NULL	As reported by the client
<code>accuracy</code>	DECIMAL (5 , 4) NOT NULL	0 to 1, recomputed by the server from the two counts below
<code>correct_answers</code> , <code>wrong_answers</code>	INTEGER NOT NULL	
<code>max_streak</code>	INTEGER NOT NULL	From <code>analytics.streakInfo.maxStreak</code> , 0 if absent
<code>average_response_time</code>	INTEGER NOT NULL	Milliseconds
<code>created_at</code>	TIMESTAMPZ NOT NULL	

QUESTION_ATTEMPTS

One row per question answered inside a session. `user_id` duplicates the session's owner so the agent can query a user's history without a join.

COLUMN	TYPE	NOTES
id	UUID	Primary key
session_id	UUID NOT NULL	References game_sessions(id)
user_id	UUID NOT NULL	References auth.users(id)
question_id	INTEGER NOT NULL	Not a foreign key. Question ids come from many sources and are squeezed into 31 bits (13.6).
topic , difficulty	TEXT NOT NULL	Free text from the question
is_correct	BOOLEAN NOT NULL	
time_spent	INTEGER NOT NULL	Milliseconds
created_at	TIMESTAMPTZ NOT NULL	

USER_STATS

One running summary per user, read by the dashboard, profile, stats and learn pages and by the question agent.

COLUMN	TYPE	NOTES
id	UUID	Primary key
user_id	UUID NOT NULL UNIQUE	References auth.users(id)
total_games_played , total_score , total_questions_answered , total_correct , total_wrong	INTEGER DEFAULT 0	Running sums
overall_accuracy	DECIMAL(5,4)) DEFAULT 0	total_correct / total_questions_answered
favorite_game	TEXT	Never written by any code. Always null.
weak_topics , strong_topics	TEXT[] default empty	Topics under 50% and at or over 80% in the latest session, merged with the earlier lists by _update_user_stats
updated_at	TIMESTAMPTZ NOT NULL	Kept current by trigger

PROFILES

One row per user, created at signup or first login. The primary key is the user id itself.

COLUMN	TYPE	NOTES
user_id	UUID	Primary key, references auth.users(id)
display_name	TEXT NOT NULL	
username	TEXT NOT NULL UNIQUE	
exam	TEXT NOT NULL DEFAULT 'sat'	CHECK (exam IN ('sat', 'gre'))
target_score	INTEGER	Nullable
test_date	DATE	Nullable
daily_minutes	INTEGER NOT NULL DEFAULT 20	
avatar_color	TEXT NOT NULL DEFAULT '#3A33E8'	
created_at , updated_at	TIMESTAMPTZ NOT NULL	updated_at kept current by trigger

MATERIALS

One row per uploaded file or pasted text. Created by `add_materials.sql`.

COLUMN	TYPE	NOTES
id	UUID	Primary key
user_id	UUID NOT NULL	References auth.users(id)
name	TEXT NOT NULL	Defaults in code to the file name, then Untitled material
exam	TEXT NOT NULL	CHECK (exam IN ('sat', 'gre'))
section	TEXT NOT NULL	CHECK (section IN ('quant', 'verbal'))
created_at	TIMESTAMPTZ NOT NULL	

CUSTOM_QUESTIONS

Questions found in, or written from, a material. Only approved ones reach games. Created by `add_materials.sql`.

COLUMN	TYPE	NOTES
id	BIGSERIAL	Primary key. The only integer key in the schema.
material_id	UUID NOT NULL	References materials(id)
user_id	UUID NOT NULL	References auth.users(id)
exam , section	TEXT NOT NULL	Copied from the material. No check constraint here.
question	TEXT NOT NULL	The stem
passage	TEXT NOT NULL DEFAULT ''	
options	JSONB NOT NULL	Array of strings, 2 to 5 of them by the parser's rule
correct_answer	INTEGER	Index into options . Null until found in the material or set during review.
topic	TEXT NOT NULL DEFAULT 'My material'	
difficulty	TEXT NOT NULL DEFAULT 'medium'	
explanation	TEXT NOT NULL DEFAULT ''	
origin	TEXT NOT NULL DEFAULT 'extracted'	CHECK (origin IN ('extracted', 'generated'))
status	TEXT NOT NULL DEFAULT 'pending'	CHECK (status IN ('pending', 'approved'))
created_at	TIMESTAMPTZ NOT NULL	

13.3 Indexes, triggers and functions

INDEX	ON	SERVES
<code>idx_game_sessions_user_id</code>	<code>game_sessions(user_id)</code>	<code>GET /api/stats/sessions</code>
<code>idx_game_sessions_created_at</code>	<code>game_sessions(created_at DESC)</code>	The newest-first ordering of the same route
<code>idx_question_attempts_session_id</code>	<code>question_attempts(session_id)</code>	The cascade from <code>game_sessions</code>
<code>idx_question_attempts_user_id</code>	<code>question_attempts(user_id)</code>	<code>SupabaseAgentOps.get_topic_performance</code>
<code>idx_question_attempts_user_topic</code>	<code>question_attempts(user_id, topic)</code>	Per-topic history for one user
<code>idx_materials_user_id</code>	<code>materials(user_id)</code>	<code>GET /api/materials</code>
<code>idx_custom_questions_material_id</code>	<code>custom_questions(material_id)</code>	A material's review list, and its cascade
<code>idx_custom_questions_user_section</code>	<code>custom_questions(user_id, exam, section, status)</code>	<code>load_custom_questions</code> , which filters on exactly those four columns

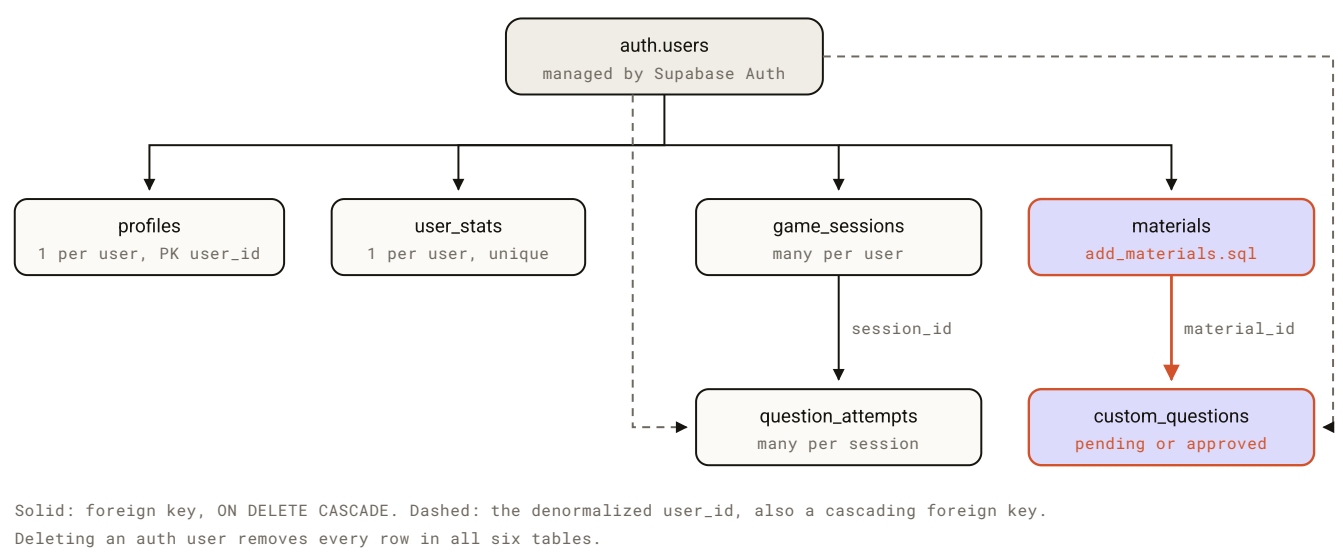
The unique constraints on `user_stats.user_id` and `profiles.username` create their own indexes. The first five indexes in the table come from `reset.sql`, the last three from `add_materials.sql`. `reset.sql` also defines one function and two triggers:

```
CREATE FUNCTION set_updated_at() RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER profiles_updated_at BEFORE UPDATE ON profiles
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
CREATE TRIGGER user_stats_updated_at BEFORE UPDATE ON user_stats
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

The backend also sets `updated_at` itself in both places that update these tables (`profile.py` and `game_service.py`), so the trigger is a backstop. There is no trigger that creates a profile when an auth user is created: the profile is written by `PUT /api/profile`, and a user who never completes the form has no row. `profile.py`'s upsert is an insert on first save, which the trigger does not touch, so `created_at` and `updated_at` start equal.

13.4 How the tables relate



The six tables. Arrows run from the referenced row to the rows that reference it.

Two tables carry a `user_id` they could have reached through a parent: `question_attempts` through `game_sessions`, and `custom_questions` through `materials`. The duplication is deliberate. It lets the agent and the game question loader filter by user with one indexed column, and it lets the policies in 13.5 be one comparison instead of a subquery. The cost is that nothing in the schema forces the copy to match the parent's owner. The backend always writes both from the same `current_user["id"]`, so they agree today.

13.5 Row Level Security, and why it does not apply

RLS is enabled on all six tables. These are every policy that exists. The first four rows come from `reset.sql`, the last two from `add_materials.sql`:

TABLE	SELECT	INSERT	UPDATE	DELETE
<code>game_sessions</code>	own rows	own rows	none	none
<code>question_attempts</code>	own rows	own rows	none	none
<code>user_stats</code>	own row	own row	own row (no <code>WITH CHECK</code>)	none
<code>profiles</code>	own row	own row	own row (no <code>WITH CHECK</code>)	none
<code>materials</code>	FOR ALL USING (<code>auth.uid() = user_id</code>) WITH CHECK (<code>auth.uid() = user_id</code>)			
<code>custom_questions</code>	FOR ALL USING (<code>auth.uid() = user_id</code>) WITH CHECK (<code>auth.uid() = user_id</code>)			

"Own" is `auth.uid() = user_id` in every case. An `UPDATE` policy without `WITH CHECK` reuses its `USING` expression for the new row, so a user could not move their stats or profile to another user id even through it.

Now the part that matters. `Database.get_client()` in `backend/src/utils/database.py` builds one process-wide client from `SUPABASE_URL` (or `NEXT_PUBLIC_SUPABASE_URL`) and the first key it finds among `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_SERVICE_KEY` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`. With a service-role key, Postgres sees the `service_role` role, which bypasses RLS entirely. Every route uses this client through `get_db()`. The frontend has `@supabase/supabase-js` and `@supabase/ssr` in `package.json` but never imports them; no browser code talks to Supabase. So in normal operation **no policy is ever evaluated**. The policies are a second line that only comes into play if someone reaches the database with the anon key and a user JWT, which this codebase never does.

Authorization therefore lives in Python. Every query on user data must carry `.eq("user_id", current_user["id"])` or check ownership first, as `_own_material()` in `materials.py` does. A route that forgets is a cross-user leak, and RLS will not catch it. One already exists: `GET /api/questions/topics` selects `topic` from all of `question_attempts` with no user filter and returns every topic any user has practised. It is low sensitivity, but it is the pattern to avoid.

IF THE SERVICE-ROLE KEY IS MISSING, THE BACKEND SILENTLY RUNS AS ANON

`get_client()` falls back to `NEXT_PUBLIC_SUPABASE_ANON_KEY`. With the anon key and no user session, `auth.uid()` is null, every policy fails, reads return empty lists and writes are rejected. The symptom is "stats are always zero" and "save-score returns 500", not a startup error.

SIGNING IN ON THE SHARED CLIENT MAY CHANGE WHO THE CLIENT IS

`AuthService.signup` and `login` call `auth.sign_up` and `auth.sign_in_with_password` on the same singleton client every route uses for data. In `supabase-py 2.x` the client listens for its own auth events, and on `SIGNED_IN` it replaces the `Authorization` header it sends to PostgREST with the new session's access token. If that is the behaviour of the installed version, then after the first login in a process every later query runs as that user, under RLS, instead of as the service role. A single developer testing with one account would never notice, because their own rows pass their own policies. A second user would see empty reads and failed writes, and everyone would fail once the first user's token expires. `Database.get_auth_client()` creates a separate client for auth and looks meant for this, but nothing calls it, and it too prefers the anon key and falls back to the service-role key. Doing auth calls on a fresh, separate client is the fix to make and test.

13.6 Who writes each table

TABLE	WRITTEN BY	READ BY
game_sessions	GameService.save_game_session (insert)	GET /api/stats/sessions ; GET /api/health/supabase (one id, as a probe)
question_attempts	GameService.save_game_session (bulk insert)	SupabaseAgentOps.get_topic_performance ; GET /api/questions/topics
user_stats	GameService._update_user_stats (select then insert or update)	GET /api/stats/user ; SupabaseAgentOps.get_user_performance
profiles	PUT /api/profile (upsert on user_id)	GET /api/profile ; the username check in PUT
materials	POST /api/materials (insert); DELETE /api/materials/{id}	GET /api/materials ; _own_material ; load_custom_questions
custom_questions	POST /api/materials (bulk insert); PATCH and DELETE /api/materials/questions/{id} ; approve-all ; material delete	The material routes; load_custom_questions in services/custom_questions.py

Every writer is the backend with the client from `database.py`. `SupabaseAgentOps` also has `save_game_session`, `save_question_attempts` and `update_user_stats`, reached only through `ExamLearningAgent.update_performance`, which nothing calls. They are dead code, and so is the TypeScript in `backend/src` (`index.ts`, `api/games.ts`, `services/gameScoreService.ts`, `config/supabase.ts`, `types/`): the deployed backend is Python only.

`user_stats` is updated by read, compute, write with no transaction or row lock. Two games saved at the same moment by one user can lose one of the increments. `save_game_session` is also three separate requests (session, attempts, stats); if the second fails, the session row stays and the route still reports failure.

`question_attempts.question_id` is `INTEGER`, so ids are kept below 2^{31} : College Board questions use `int(questionId, 16) & 0x7FFFFFFF`, web and AI questions use `zlib.crc32(stem) & 0x7FFFFFFF`, and custom questions use `CUSTOM_ID_OFFSET + row["id"]` with an offset of `1_500_000_000`. These ranges can collide in principle. The id is only used for analytics, so a collision mislabels an attempt rather than breaking anything.

13.7 How a token becomes a user

Login goes through the backend. `POST /api/auth/login` calls Supabase's `sign_in_with_password` and returns the session's `access_token`, a Supabase JWT. The frontend sends it back as `Authorization: Bearer ...`. Protected routes depend on `get_current_user` in `api/auth.py`, which uses `HTTPBearer(auto_error=False)`, returns 401 `Missing authorization token` when there is no header, and otherwise calls `AuthService.get_user(token)`:

```
# backend/src/services/auth_service.py, get_user (abridged)
decoded = jwt.decode(token, options={"verify_signature": False})
if decoded and "sub" in decoded:
    user_id = decoded["sub"]
    user_email = decoded.get("email", "")
    exp = decoded.get("exp", 0)
    if exp and exp < time.time():
        return None
    # For now, we'll trust the JWT payload since it's from Supabase
    return {"id": user_id, "email": user_email}
```

THE SIGNATURE IS NEVER VERIFIED

`get_user` decodes the token with `verify_signature: False` and checks only that `sub` exists and that `exp`, if present, is in the future. Anyone can build a token by hand with another user's id in `sub` and no `exp`, send it, and be treated as that user. Because every query then runs with the service-role key (13.5), that is full read and write access to the victim's profile, stats, sessions and materials. User ids are UUIDs and are not shown in the UI, which raises the bar but is not a control. The fix is to verify the token: either decode with the project's JWT secret or JWKS (HS256 or the project's asymmetric key, with `audience="authenticated"`), or call Supabase's `auth.get_user(token)` on a separate client. No JWT secret variable exists in the backend today, so this needs a new environment variable or a network call per request. `python-jose` is in `requirements.txt` but unused; `get_user` imports `jwt` (PyJWT), which `requirements.txt` does not list and which only arrives as a dependency of another package.

The other auth routes are thin. `POST /api/auth/logout` does nothing on the server; its comment says the token "will expire naturally". Login returns no refresh token to the browser (`TokenResponse` has no field for it, although `AuthService.login` reads one), so a session ends when the access token expires, which is the Supabase project's JWT expiry setting. `GET /api/questions/` uses `get_current_user_optional`, which returns `None` on a missing or bad token instead of raising.

13.8 What lives in the browser

KEY	STORAGE	WRITTEN BY	HOLDS
<code>auth_token</code>	<code>localStorage</code>	<code>ApiClient.setToken</code> in <code>lib/api/client.ts</code>	The Supabase access token
<code>auth_token</code>	cookie	Same	The same token, <code>path=/; max-age=604800; SameSite=Lax</code> . Not <code>HttpOnly</code> (set from JavaScript) and not <code>Secure</code> .
<code>pending_profile</code>	<code>localStorage</code>	<code>savePendingProfile</code> in <code>lib/profile.ts</code>	Email (lower-cased) and the signup form's profile, waiting for a first login when email confirmation is on
<code>exam_prefs</code>	<code>localStorage</code>	<code>setExamPrefs</code> in <code>lib/exam.ts</code>	<code>exam</code> , <code>section</code> , <code>questionCount</code>
<code>learned_lessons</code>	<code>localStorage</code>	<code>useLearned</code> in <code>lib/learnProgress.ts</code>	Lesson ids marked as learned
<code>lesson_video_wide</code>	<code>localStorage</code>	<code>app/learn/[exam]/[topic]/page.tsx</code>	<code>'1'</code> or <code>'0'</code>
<code>arcade_hi_{gameId}</code>	<code>localStorage</code>	<code>recordHighScore</code> in <code>lib/arcade.ts</code>	Best score per game on this device
<code>ai_questions_{gameId}_{exam}_{section}</code>	<code>sessionStorage</code>	<code>fetchQuestionsWithCache</code> in <code>lib/api/questions.ts</code>	A question set and timestamp, reused for 5 minutes. Only Whack-A-Mole calls it.

The cookie exists for `frontend/middleware.ts`, which redirects to `/login` when a protected path (`/dashboard`, `/games`, `/learn`, `/materials`, `/mock`, `/profile`, `/stats`) is requested without it. The middleware checks only that the cookie is present, not that it is valid, so it is a navigation convenience and not a security boundary. `app/robots.ts` disallows the same seven paths for crawlers. The cookie outlives the token: after the access token expires, the middleware still lets the user in and the API answers 401.

Because the token is readable from JavaScript in two places, any XSS on the site can steal it. That is why the HTML sanitizing in 13.11 matters. On logout `setToken(null)` removes the `localStorage` entry and expires the cookie.

`app/login/page.tsx` follows the `redirect` query parameter only when it is a same-site path (one leading `/`, not `//` or `/\`), so a crafted link cannot turn the login page into an open redirect.

13.9 CORS

`backend/src/main.py` adds Starlette's `CORSMiddleware` before any router. The allowed origins are the comma-separated `ALLOWED_ORIGINS` variable plus four fixed development origins: `http://localhost:3000`, `http://localhost:3001`, `http://127.0.0.1:3000` and `http://127.0.0.1:3001`. The rest of the configuration is permissive: `allow_credentials=True`, `methods` `GET POST PUT DELETE OPTIONS PATCH HEAD`, `allow_headers=["*"]`, `expose_headers=["*"]`, `max_age=3600`.

Since auth is a bearer header and not a cookie the backend reads, credentials mode adds nothing; the client sets `credentials: 'include'` anyway. `auth.py` also defines explicit `OPTIONS` handlers for `/signup` and `/login` that echo back any `Origin` with credentials allowed. The middleware answers real preflights before they reach a route, so these handlers only run for a bare `OPTIONS` request, but they reflect any origin and should be deleted rather than copied.

The global `@app.exception_handler(Exception)` returns `{"detail": "Internal server error"}` with status 500. In Starlette that handler runs in the outermost middleware, outside CORS, so the 500 has no `Access-Control-Allow-Origin` header and the browser reports a network failure rather than the error. Routes that raise `HTTPException` are not affected.

13.10 Input validation

FastAPI validates every body and query parameter against its type and returns 422 when it does not fit. The constraints that go beyond types are few, and listed here.

MODEL OR PARAMETER	CONSTRAINTS
<code>UserSignup</code> , <code>UserLogin</code>	<code>email: EmailStr</code> . <code>password: str</code> with no length rule; Supabase enforces its own minimum.
<code>Profile</code>	<code>display_name</code> 1 to 60 chars; <code>username</code> matches <code>^[a-z0-9_]{3,24}\$</code> ; <code>exam</code> is <code>sat</code> or <code>gre</code> ; <code>daily_minutes</code> 5 to 240; <code>avatar_color</code> matches <code>^#[0-9A-Fa-f]{6}\$</code> ; <code>test_date</code> a date; <code>target_score</code> 400 to 1600 for SAT, 260 to 340 for GRE, checked by <code>check_target_score</code>
<code>SaveScoreRequest</code> , <code>GameAnalytics</code> , <code>QuestionAttempt</code>	Types only. Scores, counts and times can be any integer, including negative. The server trusts what a game reports.
<code>QuestionUpdate</code> (in <code>materials.py</code>)	<code>status</code> is <code>pending</code> or <code>approved</code> . The route also rejects a <code>correct_answer</code> outside <code>options</code> and approval without an answer.
GET <code>/api/questions/</code>	<code>limit</code> 1 to 100; <code>exam</code> , <code>section</code> , <code>pace</code> are literals
GET <code>/api/learn/videos</code>	<code>exam</code> and <code>section</code> by regex; <code>topic</code> 2 to 80 chars; <code>limit</code> 1 to 12
POST <code>/api/materials</code>	<code>exam</code> , <code>section</code> literals; file at most <code>MAX_UPLOAD_BYTES</code> (4 MB), checked after the whole file is read; extracted text at least 40 chars and cut at 400,000
GET <code>/api/stats/sessions</code>	<code>limit: int = 10</code> , no bounds

The `username` uniqueness check in `PUT /api/profile` is a select before the upsert; two simultaneous saves of the same new name will make one of them hit the database's unique constraint and return a 500 rather than the friendly 409.

13.11 HTML sanitizing

Official SAT questions arrive as HTML with MathML. `CollegeBoardSource._normalize` in `question_sources.py` stores two forms: plain text from `html_to_text` for games that draw text themselves, and `questionHtml` , `passageHtml` , `optionsHtml` and `explanationHtml` from

`sanitize_html` for rich display. No other source produces HTML; web, AI, custom and built-in questions are text only.

`sanitize_html` in `backend/src/utils/html_utils.py` parses with the standard library's `HTMLParser` into its own tree and re-renders it from allowlists:

- `DROPPED_TAGS` (`script`, `style`, `svg`, `img`, `iframe`, `object`, `embed`, `form`, `button` and others) are removed with everything inside them, as is any element whose class contains `sr-only`.
- `HTML_TAGS` (paragraphs, inline formatting, lists, tables) and `MATH_TAGS` (the MathML presentation elements) are kept.
- Any other tag is unwrapped: the tag goes, its children stay.
- Only attributes in `KEPT_ATTRS` survive: `alttext`, `display`, `mathvariant`, `colspan`, `rowspan`, `columnalign`, `linethickness`, all HTML-escaped. No `href`, `src`, `style` or event handler can pass.
- Text is escaped. `mfenced` becomes an `mrow` with explicit `mo` brackets; a span with `underline` in its style becomes `u`.

The frontend does not trust this alone. `QuestionContent` in `frontend/components/QuestionContent.tsx` runs a second allowlist, with the same tags and attributes, through `DOMParser` before `dangerouslySetInnerHTML`. It removes disallowed elements outright, contents included. That component and the two JSON-LD `script` tags in `app/layout.tsx` and `app/developer/page.tsx` are the only uses of `dangerouslySetInnerHTML` in the app.

13.12 A security checklist for contributors

1. Every new route that touches user data depends on `get_current_user` and filters by `current_user["id"]`. RLS will not save you; the backend is the service role.
2. When you take an id from the path, prove ownership before acting on it, the way `_own_material` and the `.eq("user_id", ...)` on question updates do.
3. Never trust a JWT you have not verified. Until `get_user` verifies signatures, treat every user id as attacker-chosen when you judge the impact of a change.
4. Keep auth calls off the shared data client. Do not add another `db.auth.*` call to it.
5. New tables go in a new SQL file and in `reset.sql`, with RLS enabled and owner policies, even though the backend bypasses them. Update the README's setup step.
6. Constrain new inputs in the Pydantic model or with `Query(...)`: lengths, ranges, literals. Do not rely on the frontend.
7. Any HTML that reaches the DOM goes through `sanitize_html` on the server and `QuestionContent` on the client. Do not add a new `dangerouslySetInnerHTML`.
8. Do not return `str(e)` in a `detail`. Several routes do today (`signup`, `profile`, `stats`, `questions`, `/api/health/supabase`), which leaks database and library messages.
9. Never add a secret to a `NEXT_PUBLIC_` variable or to frontend code. The service-role key belongs only in the backend's environment as `SUPABASE_SERVICE_ROLE_KEY`.
10. Add a production origin to `ALLOWED_ORIGINS`; never widen CORS to `*` with credentials.

13.13 Study notes

READ IN THIS ORDER

1. `backend/database/reset.sql` , four of the six tables in 115 lines.
2. `backend/database/add_materials.sql` , the two tables `reset.sql` leaves out.
3. `backend/src/utils/database.py` , which key the backend uses and the fallback.
4. `backend/src/services/auth_service.py` , `get_user` in particular.
5. `backend/src/api/auth.py` , `get_current_user` and the OPTIONS handlers.
6. `backend/src/api/materials.py` , the clearest example of ownership checks.
7. `frontend/lib/api/client.ts` and `frontend/middleware.ts` , the token's life in the browser.
8. `backend/src/utils/html_utils.py` and `frontend/components/QuestionContent.tsx` .

CHECK YOURSELF

A new contributor adds `GET /api/materials/{id}` that selects the material by id only. RLS is on for `materials` . Is it safe?

No. The backend uses the service-role key, which bypasses RLS, so any signed-in user who knows the id can read it. It needs `_own_material(db, current_user["id"], material_id)` or an `.eq("user_id", ...)` .

You run `reset.sql` on a fresh project and uploads return 500. Why?

`reset.sql` does not create `materials` or `custom_questions` ; `add_materials.sql` does, and the documented setup runs it second.

What does an attacker need to act as another user today?

That user's id. `get_user` does not verify the JWT signature, so a hand-made token with that id in `sub` and no `exp` is accepted.

The dashboard loads (no redirect to login) but every stat is empty and the console shows 401s. What happened?

The access token expired. The `auth_token` cookie lasts 7 days and the middleware only checks that it exists; the backend rejects the expired token and there is no refresh token to renew it.

Why does `question_attempts` carry a `user_id` when `session_id` already leads to one?

So the agent's per-user queries and the owner policy are one indexed comparison instead of a join. `add_user_id_to_attempts.sql` added it; `reset.sql` builds it in.

Which column in the schema is never written?

`user_stats.favorite_game` .

14 API reference

Every HTTP route the FastAPI backend serves: twenty-four of them, counting two explicit `OPTIONS` handlers, across eight routers and the root. For each, the method and path, whether it needs a token, what it accepts, what it returns, how it fails, and which frontend function calls it. The last sections cover how the frontend client sends tokens and turns errors into exceptions.

The routers are mounted in `backend/src/main.py` with `app.include_router`, each under a prefix: `/api/health`, `/api/auth`, `/api/games`, `/api/stats`, `/api/questions`, `/api/profile`, `/api/learn` and `/api/materials`. The handlers are in `backend/src/api/*.py`. FastAPI also serves its generated documentation at `/docs` and the schema at `/openapi.json`, which are the quickest way to try a route against a local server on port 8000. The TypeScript file `backend/src/api/games.ts` is a leftover reference implementation and is not served.

14.1 Conventions

- **Auth.** "Required" means the route depends on `get_current_user` from `api/auth.py`: an `Authorization: Bearer <token>` header carrying the Supabase access token from login. No header gives 401 `Missing authorization token`; a token that cannot be decoded, has no `sub` or has expired gives 401 `Invalid or expired token`. The signature is not checked (13.7).
- **Errors.** Every error body is `{"detail": ...}`. For an `HTTPException`, `detail` is a string. For a request that fails validation, it is FastAPI's 422 list of `{loc, msg, type}` objects. An unhandled exception becomes 500 `{"detail": "Internal server error"}` from the global handler in `main.py`, without CORS headers.
- **Trailing slashes.** Two routes are declared as `"/"`, so their canonical paths end in a slash: `/api/health/`, `/api/questions/`. A request without the slash gets a 307 redirect. `/api/profile` and `/api/materials` are declared as `""` and have no slash.
- **Naming.** Request bodies for games use camelCase (`gameId`, `correctAnswers`); rows read from the database come back snake_case (`game_id`, `correct_answer`). Both styles are in use; follow the neighbouring route.

Examples use two placeholders: `$API` for the base URL and `$TOKEN` for an access token.

```
export API=http://localhost:8000
export TOKEN=<access token from /api/auth/login>
```

14.2 Root and health

ROUTE	AUTH	RETURNS	CALLER
GET /	None	{message, version, status}, with version "1.0.0" and status "running"	None in the frontend
GET /api/health/	None	{"status": "healthy", "service": "NYU Hacks Arcade API"}	None; for uptime checks
GET /api/health/supabase	None	Selects one id from game_sessions . On success {status: "healthy", supabase: "connected", database: "accessible"} ; on any exception, still status 200, with {status: "unhealthy", supabase: "disconnected", error}	None

The service name "NYU Hacks Arcade" is from the project's hackathon origin and survives in the FastAPI title. `/api/health/supabase` always answers 200, so a monitor must read the body. Its `error` field is the raw exception text, visible to anyone.

```
curl -s $API/api/health/supabase
```

14.3 Auth

ROUTE	AUTH	BODY	RETURNS	ERRORS	CALLER
POST <code>/api/auth/signup</code>	None	<code>UserSignup: email (EmailStr), password (str)</code>	<code>{success: true, user: {id, email}, message}</code> , plus <code>access_token</code> when Supabase returns a session (email confirmation off)	400 with Supabase's message, rewritten for "already exists" and "Email rate limit exceeded"; 500 Internal server error: ...	<code>apiClient.signup</code> , <code>app/signup/page.tsx</code>
POST <code>/api/auth/login</code>	None	<code>UserLogin: email, password</code>	<code>TokenResponse: {access_token, token_type: "bearer", user: {id, email}}</code>	401 Invalid email or password, or Supabase's message	<code>apiClient.login</code> , <code>app/login/page.tsx</code>
POST <code>/api/auth/logout</code>	Optional	None	<code>{success: true, message: "Logged out successfully"}</code>	None in practice	<code>apiClient.logout</code> , <code>components/AuthButton.tsx</code> , <code>components/DashboardLayout.tsx</code>
GET <code>/api/auth/me</code>	Required	None	<code>{id, email}</code> decoded from the token	401	<code>apiClient.getCurrentUser</code> , <code>lib/useAccount.ts</code> , <code>AuthButton.tsx</code>
OPTIONS <code>/api/auth/signup</code> , OPTIONS <code>/api/auth/login</code>	None	None	200 with CORS headers echoing the request's <code>Origin</code>	None	None; CORS middleware answers real preflights first

Signup and login run on the backend because the frontend has no Supabase client. Login returns the access token only; `AuthService.login` also reads the refresh token, but `TokenResponse` drops it, so the browser cannot renew a session. Logout changes nothing on the server. `/me` never touches the database: it returns whatever the token's payload says.

```
curl -s -X POST $API/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email": "student@example.com", "password": "<password>"}'

curl -s $API/api/auth/me -H "Authorization: Bearer $TOKEN"
```

14.4 Profile

ROUTE	AUTH	BODY	RETURNS	ERRORS	CALLER
GET /api/profile	Required	None	ProfileResponse : {profile: Profile null}; null when the user has no row	401; 500 with the exception text	apiClient.getProfile, lib/useAccount.ts
PUT /api/profile	Required	Profile (below)	{profile: Profile} as saved	401; 409 The username @name is taken. Try another one. ; 422 on validation, including the target score range; 500	apiClient.saveProfile, app/signup/page.tsx, app/login/page.tsx (pending profile), app/profile/page.tsx

PROFILE FIELD	TYPE	CONSTRAINT
display_name	str	1 to 60 characters
username	str	^[a-z0-9_]{3,24}\$, unique across users
exam	"sat" "gre"	Default "sat"
target_score	int or null	400 to 1600 for SAT, 260 to 340 for GRE
test_date	date or null	ISO yyyy-mm-dd
daily_minutes	int	5 to 240, default 20
avatar_color	str	^#[0-9A-Fa-f]{6}\$, default #3A33E8

PUT is an upsert on user_id, so the same call creates and updates. The whole profile is sent every time; there is no partial update.

```
curl -s -X PUT $API/api/profile \  
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \  
  -d '{"display_name": "Ada", "username": "ada_1", "exam": "sat",  
    "target_score": 1450, "test_date": "2026-12-05",  
    "daily_minutes": 30, "avatar_color": "#3A33E8"}'
```

14.5 Games and stats

ROUTE	AUTH	INPUT	RETURNS	ERRORS	CALLER
POST <code>/api/games/save-score</code>	Required	<code>SaveScoreRequest</code> : <code>gameId</code> (str), <code>analytics</code> (<code>GameAnalytics</code>)	<code>SaveScoreResponse</code> : { <code>success</code> : true, <code>sessionId</code> }	401; 422; 500 with the exception text from <code>GameService</code>	<code>apiClient.saveScore</code> in each <code>components/*GameContainer.tsx</code> and <code>app/mock/page.tsx</code> (as <code>mock-exam</code>)
GET <code>/api/stats/user</code>	Required	None	<code>UserStatsResponse</code> ; all zeros, empty lists and <code>updated_at</code> : "" if the user has no row	401; 500	<code>apiClient.getUserStats</code> : <code>app/dashboard</code> , <code>app/stats</code> , <code>app/profile</code> , <code>app/learn</code> pages
GET <code>/api/stats/sessions</code>	Required	Query <code>limit</code> : int, default 10, no bounds	List of <code>GameSessionResponse</code> : { <code>id</code> , <code>game_id</code> , <code>score</code> , <code>accuracy</code> , <code>correct_answers</code> , <code>wrong_answers</code> , <code>max_streak</code> , <code>created_at</code> } , newest first	401; 500	<code>apiClient.getRecentSessions</code> via <code>useRecentSessions</code> in <code>lib/sessions.ts</code> (limit 30)

GAMEANALYTICS FIELD	TYPE	USED BY THE SERVER
<code>gameId</code>	str	No; the outer <code>gameId</code> is stored
<code>score</code>	int	Stored and added to <code>total_score</code>
<code>accuracy</code>	float	No; recomputed from the counts
<code>correctAnswers</code> , <code>wrongAnswers</code>	int	Stored and summed
<code>questionAttempts</code>	list of { <code>questionId</code> : int, <code>topic</code> , <code>difficulty</code> , <code>isCorrect</code> : bool, <code>timeSpent</code> : int}	One <code>question_attempts</code> row each
<code>topicPerformance</code>	map of topic to { <code>correct</code> , <code>total</code> , <code>accuracy</code> }	<code>correct / total</code> decides weak (under 0.5) and strong (0.8 or more); <code>accuracy</code> ignored
<code>streakInfo</code>	dict	Only <code>maxStreak</code>
<code>averageResponseTime</code>	int	Stored, in milliseconds

Nothing bounds the numbers. A client can post any score, which is acceptable for a practice tool but means scores are not evidence of anything. The client's `getUserStats` and `getRecentSessions` swallow errors and return `null` and `[]` , so a 401 on these pages looks like a new account.

```
curl -s -X POST $API/api/games/save-score \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"gameId": "zombie", "analytics": {"gameId": "zombie", "score": 300,
  "accuracy": 75, "correctAnswers": 3, "wrongAnswers": 1,
  "questionAttempts": [{"questionId": 42, "topic": "Algebra",
    "difficulty": "easy", "isCorrect": true, "timeSpent": 5200}],
  "topicPerformance": {"Algebra": {"correct": 3, "total": 4, "accuracy": 75}},
  "streakInfo": {"maxStreak": 2}, "averageResponseTime": 6100}}'

curl -s "$API/api/stats/sessions?limit=5" -H "Authorization: Bearer $TOKEN"
```

14.6 Questions

ROUTE	AUTH	INPUT	RETURNS	ERRORS	CALLER
GET /api/questions/	Optional (<code>get_current_user_option</code>)	Query, below	<code>QuestionResponse</code> : { <code>questions</code> : <code>Question[]</code> , <code>total</code> }	422; 500 with the exception text	<code>apiClient.getAIQuestions</code> via <code>fetchAIQuestions</code> and <code>fetchQuestionsWithCache</code> in <code>lib/api/questions.ts</code>
GET /api/questions/topics	Required	None	{ <code>topics</code> : <code>string[]</code> }, distinct topics across <i>all</i> users' attempts	401; 500	<code>apiClient.getTopics</code> , which nothing calls

QUERY PARAMETER	TYPE	DEFAULT	EFFECT
<code>use_agent</code>	bool	<code>false</code>	When false, the route returns an empty list. Every real caller sends <code>true</code> .
<code>limit</code>	int, 1 to 100	10	Total questions wanted
<code>exam</code>	"sat" "gre"	sat	
<code>section</code>	"quant" "verbal"	quant	
<code>pace</code>	"quick" "deep" or absent	absent	Quick keeps only short questions; deep is passed through to the sources
<code>use_web_search</code>	bool	<code>true</code>	Lets the agent search the web for GRE (and fallback SAT) questions
<code>include_custom</code>	bool	<code>true</code>	For a signed-in user, up to <code>limit // 2</code> of their approved custom questions are mixed in
<code>topic</code> , <code>difficulty</code>	str	absent	Accepted and ignored

With `use_agent=true` the route loads custom questions (if signed in), then asks `ExamLearningAgent.generate_questions` for the rest, using the all-zero UUID as the user id for guests, and

shuffles the result. If the agent throws, the route still answers 200 with just the custom questions, which may be none. The frontend's `fetchAIQuestions` treats an empty list as a failure and falls back to the built-in bank in `lib/questionBank.ts`. A `Question` has `id`, `question`, `options`, `correctAnswer`, `topic`, `difficulty`, `explanation`, and optional `exam`, `section`, `skill`, `passage`, `stem`, the four `*Html` fields, `source`, `sourceName` and `sourceUrl`. `source` is `official`, `web`, `ai` or `custom` from this route.

This is the slowest route in the API. It fetches from College Board, may search the web and calls an LLM through OpenRouter, so it can take many seconds. `apiClient.getQuestions`, which calls it without `use_agent`, is unused and would always get an empty list.

```
curl -s "$API/api/questions/?use_agent=true&limit=10&exam=sat&section=verbal&pace=quick" \
-H "Authorization: Bearer $TOKEN"
```

14.7 Learn

ROUTE	AUTH	QUERY	RETURNS	ERRORS	CALLER
GET /api/learn/videos	Required	exam (^(sat gre)\$), section (^(quant verbal)\$), topic (2 to 80 chars), all required; limit 1 to 12, default 6	VideoResponse : {videos: [{id, title, channel, duration, url, thumbnail, published, trusted}]}	401; 422; 502 Video search failed: ...	apiClient.getTopicVideos, components/learn/Videos.tsx

`find_topic_videos` in `services/videos.py` searches with the YouTube Data API when `YOUTUBE_API_KEY` is set, Serper when `SERPER_API_KEY` is, and DuckDuckGo otherwise. Results are cached in process memory per exam, section and topic.

```
curl -s "$API/api/learn/videos?exam=gre&section=quant&topic=Geometry&limit=3" \
-H "Authorization: Bearer $TOKEN"
```

14.8 Materials

All seven routes need a token. Each one scopes its query to the caller, either by `_own_material` (404 `Material not found` if the material is not theirs) or by an `.eq("user_id", ...)` on the question.

ROUTE	INPUT	RETURNS	ERRORS	CALLER
POST <code>/api/materials</code>	Multipart form: <code>exam</code> , <code>section</code> (required literals), <code>name</code> , <code>text</code> , <code>generate</code> (bool, default false), <code>file</code> (optional PDF or text)	<code>{material, questions}</code> : the new <code>materials</code> row and the saved <code>custom_questions</code> rows, all <code>pending</code>	413 over 4 MB; 400 unreadable file; 400 fewer than 40 readable characters; 422 no questions found	<code>apiClient.uploadMaterial</code>
GET <code>/api/materials</code>	None	<code>{materials}</code> , newest first, each with <code>question_count</code> and <code>approved_count</code>	401	<code>apiClient.getMaterials</code>
GET <code>/api/materials/{material_id}/questions</code>	Path: material UUID	<code>{questions}</code> ordered by <code>id</code>	404	<code>apiClient.getMaterialQuestions</code>
PATCH <code>/api/materials/questions/{question_id}</code>	Path: int. Body <code>QuestionUpdate</code> : any of <code>question</code> , <code>options</code> , <code>correct_answer</code> , <code>explanation</code> , <code>status</code> (<code>pending</code> or <code>approved</code>)	<code>{question}</code> , the updated row	404; 400 The answer must be one of the choices. ; 400 Pick the correct answer before approving this question.	<code>apiClient.updateCustomQuestion</code> (sends only <code>correct_answer</code> and <code>status</code>)
DELETE <code>/api/materials/questions/{question_id}</code>	Path: int	<code>{success: true}</code> , even if nothing matched	401	<code>apiClient.deleteCustomQuestion</code>
POST <code>/api/materials/{material_id}/approve-all</code>	Path: material UUID	<code>{approved, needs_answer}</code> counts	404	<code>apiClient.approveAllQuestions</code>
DELETE <code>/api/materials/{material_id}</code>	Path: material UUID	<code>{success: true}</code>	404	<code>apiClient.deleteMaterial</code>

Every caller is in `app/materials/page.tsx`. Uploading runs the regex parser in `services/material_parser.py`, and calls the LLM too when `generate` is true or the parser found nothing. The material row is inserted only once at least one question exists. These routes do not catch database errors, so a missing table or a Supabase outage surfaces as the global 500, which the browser sees as a CORS failure. A material id that is not a valid UUID makes PostgREST reject the query, with the same result.

```
curl -s -X POST $API/api/materials \
  -H "Authorization: Bearer $TOKEN" \
  -F exam=sat -F section=quant -F name='Unit 3 quiz' -F generate=false \
  -F file=@quiz.pdf

curl -s -X PATCH $API/api/materials/questions/17 \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"correct_answer": 2, "status": "approved"}'
```

14.9 The frontend client

`frontend/lib/api/client.ts` exports one instance, `apiClient`, of the class `ApiClient`. Its base URL is `NEXT_PUBLIC_API_URL`, defaulting to `http://localhost:8000`; `next.config.js` repeats the same default. The browser calls the backend directly on that origin. `frontend/vercel.json` has an `/api/:path*` rewrite to a placeholder Railway URL, but the client never uses relative `/api` paths, so that rewrite does nothing.

TOKENS

The constructor reads `auth_token` from `localStorage`, or from the `auth_token` cookie if storage is empty, copying it back to storage. `setToken(token)` writes both, the cookie with `max-age=604800` and `SameSite=Lax`; `setToken(null)` clears both. `login` and `signup` call `setToken` when the response has an `access_token`; `logout` calls `setToken(null)` in a `finally`, so the token is cleared even if the request fails. There is no refresh: when the token expires, calls return 401 until the user signs in again.

REQUESTS AND ERRORS

The private `request<T>(endpoint, options)` sets `Content-Type: application/json`, adds `Authorization: Bearer` when a token is held, and fetches with `mode: 'cors'` and `credentials: 'include'`. On a non-2xx response it parses the body (falling back to `{detail: statusText}`) and throws `new Error(error.detail || 'HTTP error! status: N')`. On success it returns parsed JSON when the content type is JSON, else `{}`. Every failure is logged with `console.error('API request failed:', ...)` and rethrown.

Three consequences follow. The status code is lost: callers see only a message, so a 401 and a 500 look alike, and nothing signs the user out on 401. A 422 has a list in `detail`, and `new Error(list)` produces the message `[object Object]`; pages that show `err.message` will show that. And a CORS-less 500 surfaces as a `TypeError` from `fetch`, not as the server's message.

`uploadMaterial` is the one method that bypasses `request`: it builds a `FormData` and calls `fetch` itself so the browser sets the multipart boundary, adds the bearer header by hand, and throws `body.detail` the same way. `getUserStats` and `getRecentSessions` catch everything and return `null` and `[]`. `getQuestions` and `getTopics` have no callers.

14.10 Adding a route

1. Put the handler in the router file for its area under `backend/src/api/`, or a new file registered in `main.py` with a prefix and tag.

2. Depend on `get_current_user` and `get_db`; filter every query by `current_user["id"]`.
3. Define request and response models in `backend/src/models/schemas.py` with real constraints, and set `response_model`.
4. Raise `HTTPException` with a sentence a user can read; do not put `str(e)` in it.
5. Add a typed method to `ApiClient` and call it from the page, not `fetch`.
6. Add the route to this chapter.

14.11 Study notes

READ IN THIS ORDER

1. `backend/src/main.py`, the prefixes, CORS and the exception handler.
2. `backend/src/api/auth.py`, for `get_current_user`, which most routes depend on.
3. `backend/src/models/schemas.py`, every request and response shape in one place.
4. `backend/src/api/profile.py` and `stats.py`, the simplest authenticated routes.
5. `backend/src/api/questions.py`, the optional-auth route and the agent call.
6. `backend/src/api/materials.py`, multipart input and ownership checks.
7. `frontend/lib/api/client.ts`, the other end of every row in this chapter.

CHECK YOURSELF

A game calls `GET /api/questions/?limit=10` and gets `{"questions": [], "total": 0}`. Why?
`use_agent` defaults to false, and without it the route builds no questions. Send `use_agent=true`, as `getAIQuestions` does.

The profile page shows the error "[object Object]". What status did the server return?

Almost certainly 422. Validation errors put a list in `detail`, and the client passes it straight to `new Error`.

Which material route reports success even when the id does not belong to the caller?

`DELETE /api/materials/questions/{id}`. It deletes nothing (the user filter matches no row) but always returns `{success: true}`.

Why does `uploadMaterial` not use `request()`?

`request()` forces `Content-Type: application/json`. A multipart body needs the browser to set the content type with its boundary.

Which fields of `GameAnalytics` does the server ignore?

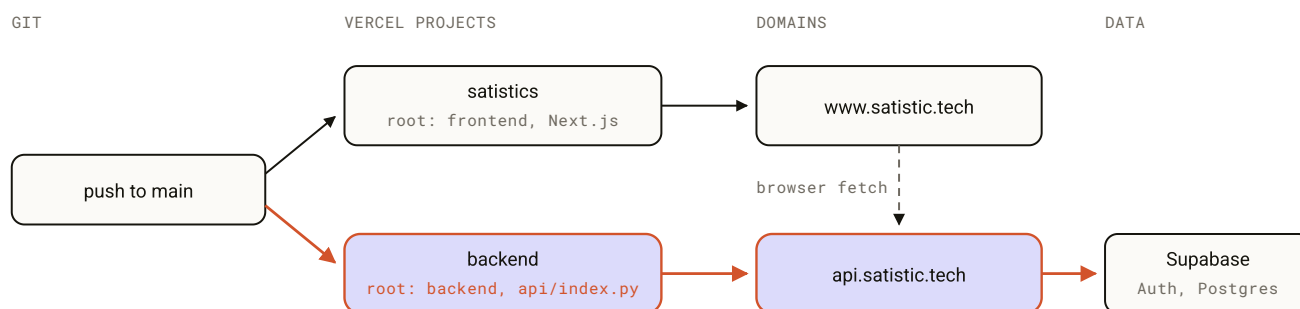
The inner `gameId`, `accuracy`, each topic's `accuracy`, and everything in `streakInfo` except `maxStreak`.

15 Deploying and operating

One repository becomes two Vercel projects. The project called `statistics` builds `frontend/` and serves `https://www.satistic.tech`; the project called `backend` builds `backend/` as a single Python function and serves `https://api.satistic.tech`. Both deploy on every push to `main`. The database is a Supabase project that neither deploy touches: schema changes are SQL you run by hand.

This chapter describes how each project is configured, which variables each one needs, why the API lives on a custom domain, how CORS and search-engine files are set up, how to change the schema and roll back a bad deploy, and what the other deploy files in `backend/` are. It ends with a checklist to run after any deploy that touches configuration. Setting up your own copy uses the same steps with your own domains and keys.

15.1 Two projects, one repository



The two builds are independent. Neither waits for the other, and neither touches the database schema.

Production. Pages come from one project, API calls go to the other, and only the API holds Supabase credentials.

VERCEL PROJECT	ROOT DIRECTORY	DOMAIN	CONFIG FILES
<code>statistics</code>	<code>frontend</code>	<code>www.satistic.tech</code>	<code>frontend/vercel.json</code> , <code>frontend/next.config.js</code> , <code>frontend/.vercelignore</code>
<code>backend</code>	<code>backend</code>	<code>api.satistic.tech</code>	<code>backend/vercel.json</code> , <code>backend/api/index.py</code> , <code>backend/.vercelignore</code>

The root directory is set in each project's settings, not in a file. There used to be a `vercel.json` at the repository root with a `rootDirectory` field; Vercel rejected the field and every backend build failed until it was removed (commit "Fix backend Vercel deployment"). Do not add one back. Each project reads only the `vercel.json` inside its own root.

Both projects use Vercel's Git integration. A push to `main` becomes production in both; a push to any other branch makes preview deployments. GitHub Actions (`.github/workflows/ci.yml`) runs lint, typecheck,

build and a backend import check on the same push, but nothing makes Vercel wait for CI. A red CI run does not stop a deploy, which is why `CONTRIBUTING.md` says nobody commits to `main` directly.

15.2 The backend on Vercel

The backend is one Python serverless function. Vercel's Python runtime installs `backend/requirements.txt`, finds `backend/api/index.py`, and serves the ASGI object named `app` that the file exposes. The file is four lines of real code:

```
# backend/api/index.py
backend_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if backend_dir not in sys.path:
    sys.path.insert(0, backend_dir)

from src.main import app # noqa: E402,F401
```

It puts `backend/` on the import path so `from src.api import ...` resolves exactly as it does locally, then re-exports the FastAPI app. There is no adapter such as Mangum; the runtime speaks ASGI directly.

`backend/vercel.json` sends every path to that one function:

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/api/index" }
  ]
}
```

The rewrite changes which function runs, not the path FastAPI sees, so `/api/health/` still matches the health router. The file sets no `maxDuration` or memory, so the platform defaults apply. Question generation that calls the College Board bank, a web search and a model in sequence is the slowest request; if it starts timing out, a `functions` block in this file is the place to raise the limit.

`backend/.vercelignore` keeps the upload small and secret-free. It excludes `.env` and `.env.*`, `venv/`, caches, the alternative deploy files (`Dockerfile`, `fly.toml`, `railway.json`, `render.yaml`), every `*.sh` script, and the whole `database/` folder. The SQL never ships, because nothing at runtime reads it.

Serverless has two consequences worth knowing. The in-memory caches (the 24-hour video cache in `src/services/videos.py` and the College Board index cache in `src/services/question_sources.py`) live per warm instance and vanish on a cold start. And `Database.get_client()` keeps one Supabase client per instance, shared by every request that instance serves.

15.3 The frontend on Vercel

`frontend/vercel.json` declares the Next.js framework with `npm install`, `npm run build` and an output directory of `.next`. It also contains a rewrite that is left over from an earlier hosting plan:

```
"rewrites": [  
  {  
    "source": "/api/:path*",  
    "destination": "https://your-backend-url.railway.app/api/:path*"  
  }  
]
```

The destination is a placeholder that does not resolve. Nothing depends on it, because the browser calls the API by its full URL from `NEXT_PUBLIC_API_URL`, never through a relative `/api` path, and `frontend/middleware.ts` skips `/api` paths. A request to `www.satistic.tech/api/...` fails. Removing the block, or pointing it at `https://api.satistic.tech`, is a safe cleanup.

`frontend/next.config.js` sets `reactStrictMode`, allows remote images from one host for the developer page, sets `output: 'standalone'` (which Vercel ignores; it matters only for self-hosting in a container), and copies `NEXT_PUBLIC_API_URL` into the build's `env` with a default of `http://localhost:8000`.

THE API URL IS BAKED INTO THE BUILD

`NEXT_PUBLIC_API_URL` is inlined into the JavaScript bundle at build time. Changing it in the Vercel dashboard does nothing until the frontend is redeployed. If it is missing from the project, the build succeeds and production quietly calls `http://localhost:8000`, which shows up as every API call failing in users' browsers.

`frontend/.vercelignore` excludes `.env.local`, `.env.*.local`, `node_modules/`, `.next/`, `.vercel/` and log files. `frontend/deploy.sh` builds and runs `npx vercel --prod` by hand; with the Git integration you do not need it, and its reminder to set Supabase variables on the frontend is out of date (see 15.4).

15.4 Environment variables per project

Set these in each project's Settings, Environment Variables, for the Production environment. Section 3.4 explains each one and where it is read.

PROJECT	VARIABLE	PRODUCTION VALUE
backend	SUPABASE_URL	The production Supabase project URL
backend	SUPABASE_SERVICE_ROLE_KEY	That project's service-role key. Mark it sensitive
backend	NEXT_PUBLIC_SUPABASE_ANON_KEY	That project's anon key (fallback only)
backend	ALLOWED_ORIGINS	https://www.satistic.tech, https://satistic.tech or your own origins
backend	OPENROUTER_API_KEY , OPENROUTER_MODEL	Optional; AI questions and material reading
backend	YOUTUBE_API_KEY , SERPER_API_KEY	Optional, but in production DuckDuckGo rate limits quickly, so set at least one
satistic s	NEXT_PUBLIC_API_URL	https://api.satistic.tech

The service-role key must be under one of the two names the backend reads, `SUPABASE_SERVICE_ROLE_KEY` or `SUPABASE_SERVICE_KEY`. If neither is set, the client silently falls back to the anon key, Row Level Security rejects the backend's writes, and saving a score fails. That exact failure is why `SUPABASE_SERVICE_KEY` was added as a fallback name. Backend variables take effect on the next deployment, so redeploy after changing one.

15.5 Domains, CORS and deployment protection

Both domains are added in each project's Settings, Domains, with DNS records at the registrar pointing at Vercel. The site's canonical host is `www.satistic.tech`; it is `SITE_URL` in `frontend/lib/site.ts`.

Why the API has its own domain. Vercel's deployment protection puts a Vercel login in front of generated deployment URLs (the `*.vercel.app` hosts). A browser `fetch` from the site to such a URL gets a 401 and a login page instead of JSON, and the CORS preflight fails with it, so the site looks broken while the API is fine. A custom production domain is not covered by standard protection, so `api.satistic.tech` is reachable by every browser while the generated URLs stay private. Point `NEXT_PUBLIC_API_URL` at the custom domain, never at a `vercel.app` URL.

CORS. `backend/src/main.py` builds the allow list once, at import:

```
allowed_origins = os.getenv("ALLOWED_ORIGINS", "").split(",") if os.getenv("ALLOWED_ORIGINS") else
default_origins = [
    "http://localhost:3000",
    "http://localhost:3001",
    "http://127.0.0.1:3000",
    "http://127.0.0.1:3001",
]
allowed_origins = [origin for origin in allowed_origins + default_origins if origin]
```

The middleware allows credentials, the methods `GET`, `POST`, `PUT`, `DELETE`, `OPTIONS`, `PATCH` and `HEAD`, any header, and caches a preflight for 3600 seconds. Matching is exact: `https://satisfic.tech` and `https://www.satisfic.tech` are different origins, and the split does not trim spaces, so write the list with commas only. A frontend preview deployment has its own origin, which is not on the list, so previews can load pages but their API calls to production are blocked. Test API changes locally, not on previews.

15.6 Changing the database

There is no migration runner. Schema lives in SQL files under `backend/database/`, and a change reaches production when someone pastes it into the Supabase SQL Editor for the production project.

`CONTRIBUTING.md` sets the rule: a schema change is a new SQL file, and the same change is folded into `reset.sql` so a fresh project gets the whole schema in one run. Every new table gets RLS enabled and policies, even though the backend uses the service-role key.

Order matters because the two sides deploy separately:

1. Write the change additively: `CREATE TABLE IF NOT EXISTS`, `ADD COLUMN` with a default, and `DROP POLICY IF EXISTS` before `CREATE POLICY`, so the file can be rerun.
2. Run it on a development project and exercise the code against it.
3. Run it on production *before* merging code that reads the new column or table.
4. Merge. The new backend deploys against a schema that already fits it.
5. Remove old columns only in a later change, after no deployed code reads them.

A new project gets the schema from two files, run in order: `reset.sql`, then `add_materials.sql`, which creates `materials` and `custom_questions`. Never run `reset.sql` on production. It drops the core tables and deletes every user in `auth.users`. Chapter 13 lists the tables and policies.

15.7 Search engines

Three files in the frontend describe the site to crawlers, and all of them build URLs from `SITE_URL` in `frontend/lib/site.ts`:

- `frontend/app/sitemap.ts` lists four public URLs: the home page, `/developer`, `/signup` and `/login`, with `lastModified` set to build time.
- `frontend/app/robots.ts` allows `/`, disallows `/dashboard`, `/games`, `/learn`, `/materials`, `/mock`, `/profile` and `/stats`, and points at `/sitemap.xml`.
- `frontend/app/layout.tsx` sets `metadataBase`, the title template `%s | SATistics`, the description and application name from `lib/site.ts`, and renders JSON-LD through the `jsonLd` helper, which escapes `<` so the data cannot close the script tag.

The robots disallow list matches `protectedPrefixes` in `frontend/middleware.ts` route for route. Keep the two lists in step when you add a signed-in route. If you run your own copy on another domain, `SITE_URL` is the one value to change.

15.8 Rolling back

Every deployment Vercel has built stays available. To undo a bad release, open the project's Deployments tab, find the last good production deployment, and use Instant Rollback (or "Promote to Production"). The CLI does the same with `vercel rollback` from a directory linked to the project. The switch is immediate because nothing is rebuilt.

Three things to keep in mind:

- **The projects roll back separately.** If a change touched both the API and the client, decide whether the old frontend works with the new backend before rolling back only one.
- **An instant rollback keeps the old build's inlined values.** A frontend deployment carries the `NEXT_PUBLIC_API_URL` it was built with.
- **The database does not roll back.** Nothing reverses a SQL change. This is why schema changes are additive and run first.

After a rollback, fix forward on a branch: `git revert` the bad commit in a pull request, so `main` matches what is serving again and the next push does not reintroduce the bug.

15.9 The other deploy files

`backend/` carries four configurations for hosts other than Vercel. None is used by the live site; they are alternatives for self-hosting and leftovers from before the move to Vercel. `.vercelignore` keeps them out of the Vercel upload.

FILE	WHAT IT DOES	NOTES
<code>Dockerfile</code>	<code>python:3.11-slim</code> , installs <code>gcc</code> and the requirements, copies <code>src/</code> and <code>database/</code> , runs <code>uvicorn src.main:app</code> on port 8000	The most portable option. The port is fixed at 8000 and does not read <code>PORT</code> .
<code>fly.toml</code>	App <code>nyuhacks-arcade-api</code> in region <code>iad</code> , uvicorn on 8080, machines stop when idle	Uses the hackathon app name.
<code>railway.json</code>	Nixpacks build, <code>uvicorn src.main:app --host 0.0.0.0 --port \$PORT</code> , restart on failure up to 10 times	The placeholder rewrite in <code>frontend/vercel.json</code> dates from this setup.
<code>render.yaml</code>	Free web service <code>nyuhacks-arcade-api</code> in <code>oregon</code> , same start command, declares <code>SUPABASE_URL</code> and <code>SUPABASE_SERVICE_ROLE_KEY</code>	Lists only two of the variables; add the rest by hand.

On any of them, set the same backend variables as in 15.4 and point `NEXT_PUBLIC_API_URL` at the new host.

15.10 Operations checklist

Run this after a deploy that changes configuration, domains, dependencies or schema.

1. `curl https://api.satistic.tech/api/health/` returns `"status":"healthy"` .

2. `curl https://api.satistic.tech/api/health/supabase` returns `"supabase": "connected"`. If it says unhealthy, read the `error` field: a missing table means SQL was not run, an auth error means a wrong key.
3. A CORS preflight from the site origin succeeds:

```
curl -si -X OPTIONS https://api.satistic.tech/api/auth/login \  
-H "Origin: https://www.satistic.tech" \  
-H "Access-Control-Request-Method: POST" | grep -i access-control-allow-origin
```

4. Load `https://www.satistic.tech`, log in, and check in the browser's network tab that requests go to `api.satistic.tech`, not `localhost`.
5. Play one round and confirm it appears on `/stats`.
6. Open one lesson and confirm videos load.
7. `https://www.satistic.tech/sitemap.xml` and `/robots.txt` show the production host.
8. In the backend project's runtime logs, look for `Unhandled exception` lines, which `global_exception_handler` prints with a traceback before returning a generic 500.

WHAT A GREEN DEPLOYMENT DOES NOT TELL YOU

Vercel marks a deployment ready when it builds. It does not check that the environment variables are present, that the API URL was right at build time, or that the database has the tables the code queries. Each of those fails only at request time, in a user's browser. The checklist above is how you find out first.

15.11 Study notes

READ IN THIS ORDER

1. `backend/api/index.py` and `backend/vercel.json` , the whole backend deploy.
2. `backend/.vercelignore` , for what never ships.
3. `backend/src/main.py` , the CORS block and the exception handler.
4. `frontend/next.config.js` and `frontend/vercel.json` , including the stale rewrite.
5. `frontend/lib/site.ts` , `app/sitemap.ts` , `app/robots.ts` and the metadata in `app/layout.tsx` .
6. `.github/workflows/ci.yml` , for what runs beside each deploy.
7. `backend/Dockerfile` , if you plan to host the API elsewhere.

CHECK YOURSELF

You change `NEXT_PUBLIC_API_URL` in the dashboard. Why does production still call the old API?

The value is inlined at build time. The frontend has to be redeployed.

Why does the site call `api.satistic.tech` and not the backend's `vercel.app` URL?

Deployment protection puts a Vercel login in front of generated URLs, so browser calls get a 401 page. The custom production domain is not protected.

Saving scores fails in production with a row-level security error. What is the likely cause?

Neither `SUPABASE_SERVICE_ROLE_KEY` nor `SUPABASE_SERVICE_KEY` is set, so the client fell back to the anon key.

A frontend preview deployment loads but every API call fails. Is the backend down?

Probably not. The preview's origin is not in `ALLOWED_ORIGINS` , so CORS blocks it.

You rolled back the backend after a release that added a column. What did not roll back?

The database. SQL changes are not reversed, which is why they must be additive and applied before the code that needs them.

16 Contributing

A change to SATistics travels a short road: an issue, a branch, a few focused commits, a pull request that passes CI, and a merge to `main` that Vercel deploys within minutes. The road is short because the codebase is split by area, so most changes touch one folder of the frontend, one router of the backend, or one chapter of this book. This chapter is the map and the rules of the road.

`CONTRIBUTING.md` at the repository root is the short version and the one GitHub shows to new contributors. This chapter agrees with it and adds the detail: which files belong to which area, what the checks prove and what they do not, the conventions you will see in the code, and how to change this handbook.

`CODE_OF_CONDUCT.md` sets the expectations for how we treat each other, and `SECURITY.md` explains how to report a vulnerability privately instead of in a public issue.

16.1 Picking something to work on

Start from the area you care about. Each row names the chapter that explains it and the files you will open first.

AREA	CHAPTER	MAIN FILES
Look and feel, design tokens	4	<code>frontend/tailwind.config.js</code> , <code>frontend/app/globals.css</code> , <code>frontend/components/brand/</code>
Accounts and profiles	5	<code>backend/src/api/auth.py</code> , <code>backend/src/services/auth_service.py</code> , <code>backend/src/api/profile.py</code> , <code>frontend/app/signup/</code> , <code>frontend/middleware.ts</code>
Questions and the AI agent	6, 7	<code>backend/src/services/agent.py</code> , <code>backend/src/services/question_sources.py</code> , <code>frontend/lib/api/questions.ts</code> , <code>frontend/lib/questionBank.ts</code>
Uploaded material	12	<code>backend/src/api/materials.py</code> , <code>backend/src/services/material_parser.py</code> , <code>backend/src/services/custom_questions.py</code> , <code>frontend/app/materials/</code>
Games	8, 9	<code>frontend/games/</code> , <code>frontend/components/*GameContainer.tsx</code> , <code>frontend/components/exam/</code> , <code>frontend/lib/games.ts</code> , <code>frontend/lib/gameMeta.ts</code>
Lessons and videos	10	<code>frontend/lib/courses.ts</code> , <code>frontend/lib/lessons.ts</code> , <code>frontend/app/learn/</code> , <code>backend/src/services/videos.py</code>
Dashboard and stats	11	<code>frontend/app/dashboard/</code> , <code>frontend/app/stats/</code> , <code>backend/src/api/stats.py</code> , <code>backend/src/services/game_service.py</code>
Mock exams	12	<code>frontend/app/mock/</code> , <code>frontend/components/exam/Calculator.tsx</code>
Tables and policies	13	<code>backend/database/</code>
API routes	14	<code>backend/src/main.py</code> , <code>backend/src/api/</code> , <code>backend/src/models/schemas.py</code> , <code>frontend/lib/api/client.ts</code>
Deploying	15	<code>backend/vercel.json</code> , <code>backend/api/index.py</code> , <code>frontend/next.config.js</code>

For a first change, `CONTRIBUTING.md` suggests work that needs no deep knowledge of the rest of the system: a lesson in `frontend/lib/lessons.ts` , more questions in the offline bank in `frontend/lib/questionBank.ts` , tests for pure functions, accessibility in game overlays, or an issue labelled `good first issue` .

For anything bigger than a fix, open an issue first. The repository has three issue templates in `.github/ISSUE_TEMPLATE/` : a bug report, a feature request, and a new game proposal that asks how the player chooses an answer and which assets (with licenses) the game will use. Agreeing on an idea is cheaper than reworking a finished pull request. A new question type, topic or exam section always starts as an issue, because SAT and GRE content must follow the official domains listed in `frontend/lib/courses.ts` .

16.2 Branches and commits

Branch from `main` for every change and open a pull request. Never push to `main` directly: both Vercel projects deploy it to production on every push (15.1), and nothing makes the deploy wait for CI.

```
git switch main
git pull
git switch -c add-pac-man-hint-button
# ...edit, check, commit...
git push -u origin add-pac-man-hint-button
```

Commits follow the history that is already there:

- **Small and focused.** One logical change per commit. A refactor and a feature are separate commits, and usually separate pull requests.
- **Imperative subject lines** that say what changed for the user: "Add hints, material uploads, arcade skin and touch support to the games", "Fix backend Vercel deployment". Not "updated files". A scope prefix such as `fix(learn):` or `feat(games):` is welcome but not required.
- **A body when the why is not obvious.** Recent commits use a short bulleted body listing each user-visible change. Explain causes, not just effects: "Remove root vercel.json: its invalid rootDirectory field failed every backend build".
- **No AI co-author trailers.** If a tool helped, you are still the author and the one responsible for the change.
- **No secrets, ever.** Not in code, docs, fixtures or screenshots. CI runs a gitleaks scan on every pull request, but it is a net, not a plan.

16.3 Checks before a pull request

Run what CI runs. `.github/workflows/ci.yml` has three jobs; run the first two yourself before you push.

```
cd frontend
npm run lint           # next lint with next/core-web-vitals
npx tsc --noEmit       # the whole project under strict mode
npm run build          # catches server/client boundary and route errors
```

```
cd backend
pip install -r requirements.txt
uvicorn src.main:app --reload --port 8000
curl http://localhost:8000/api/health/
```

CI JOB	STEPS	WHAT IT PROVES
Frontend	Node 22, <code>npm ci</code> , <code>npm run lint</code> , <code>npx tsc --noEmit</code> , <code>npm run build</code> with <code>NEXT_PUBLIC_API_URL=http://localhost:8000</code>	The frontend lints, type-checks and builds
Backend	Python 3.12, <code>pip install -r requirements.txt</code> , <code>python -c "from src.main import app"</code>	Every module imports and every router registers
Secret scan	<code>gitleaks/gitleaks-action</code> over the full history, pull requests only	No key-shaped string entered the branch

Note what the backend job does not prove. Importing the app does not call a route, touch Supabase or run a query. A typo inside a handler passes CI. So for any backend change, start the server, call the route you changed (the interactive docs at `/docs` are the quickest way), and then click through the screens that use it, on a desktop window and at phone width.

16.4 There are no tests yet

SATistics has no automated test suite. There is no test runner in `frontend/package.json`, no test dependency in `backend/requirements.txt`, and no test files. Lint, the type checker, the build and the import check are the only automated guards. Say so in your pull request instead of implying coverage that does not exist, and describe how you verified the change by hand.

Adding tests is one of the most useful contributions you can make. The best first targets are pure functions with no network or DOM, where a test is short and a regression would be silent:

- **Backend parsing and ranking.** `parse_questions` and `merge_questions` in `backend/src/services/material_parser.py`; `is_quick_question` and `normalize_text` in `question_sources.py`; `extract_json` in `agent.py`; the duration parser, `_mentions` and `_score` in `videos.py`. `pytest` with plain function calls is enough.
- **Frontend logic.** `frontend/lib/exam.ts`, `frontend/lib/sessions.ts` and `getBankQuestions` in `frontend/lib/questionBank.ts`. A small Vitest setup would cover these without a browser.
- **API contracts.** FastAPI's `TestClient` against the routers with `get_db` and `get_current_user` overridden by dependency injection would test validation and auth checks without a real Supabase project.

A pull request that adds a test runner should also add it to `ci.yml`, so the tests keep running after the first one is merged.

16.5 The pull request

`.github/PULL_REQUEST_TEMPLATE.md` asks for three things: what changed, why (link the issue with `Closes #123`), and how you verified it. Fill in all three. Before you ask for review, walk the checklist:

- One change per pull request.
- `npm run lint`, `npx tsc --noEmit` and `npm run build` pass in `frontend/`.
- For backend changes, the server starts and `/api/health/` responds, and you called the changed route.

- You clicked through the affected screens on desktop and at phone width. Anything visual has a screenshot or a short recording.
- No secrets, keys or personal data in the diff.
- A new environment variable is read on the backend and documented with a placeholder in `backend/.env.example` (or `frontend/.env.example`, if it truly belongs in the browser, which should be rare).
- A schema change is a new SQL file under `backend/database/` and is folded into `reset.sql`, with RLS enabled and policies on every new table.
- `CHANGELOG.md` has a line under "Unreleased" if a user would notice the change.
- The handbook chapter for the area is updated if behaviour changed (16.8).

16.6 Common changes, at a glance

ADDING A GAME

A game is a class in its own folder under `frontend/games/`, a container component that mounts it, and three registrations. The steps, and the contract every game shares, are in chapter 8 and chapter 9. In outline:

1. Open a "New game proposal" issue.
2. Create `frontend/games/<id>/`. Canvas games extend `BaseGame` in `frontend/games/BaseGame.ts`, which declares `init`, `update`, `render` and `handleInput`; the 3D games use `three`.
3. Add a container in `frontend/components/` and a branch for it in `GameContainer.tsx`.
4. Register the game in `frontend/lib/games.ts` and its title, genre and `answerBy` line in `GAME_META` in `frontend/lib/gameMeta.ts`.
5. Get questions through `frontend/lib/api/questions.ts`, show them with the components in `frontend/components/exam/`, and save results with `apiClient.saveScore`. A game that builds its own question pipeline is hard to maintain.

ADDING OR IMPROVING A LESSON

Lessons are static data. A topic must exist in a course's `topics` list in `frontend/lib/courses.ts`, and its lesson is an entry in `frontend/lib/lessons.ts` with a `summary`, `ideas`, a worked `example`, `traps` and a quick `check`, matching the `Lesson` interface at the top of the file. The topic name must match the topic names the question bank uses, or the lesson's practice link and videos will not line up. Chapter 10 covers the rest.

ADDING AN API ROUTE

1. Add a handler to the router in `backend/src/api/` that fits, or a new module with `router = APIRouter()` registered in `backend/src/main.py` with `app.include_router(..., prefix="/api/<name>", tags=[...])`.
2. Declare request and response models in `backend/src/models/schemas.py`.
3. Require a user with `Depends(get_current_user)` from `src/api/auth.py`, and take the database with `Depends(get_db)`. Filter every query by `current_user["id"]`: the service-role client bypasses RLS, so the handler is the only thing keeping one user out of another's rows.

4. Keep logic that is more than a query in `backend/src/services/`.
5. Add a typed method to `ApiClient` in `frontend/lib/api/client.ts`.

Chapter 14 lists every existing route and its auth rule.

16.7 Code style

There is no formatter configuration in the repository, so match the file you are editing. These are the conventions the code follows.

TYPESCRIPT AND REACT

- **Strict mode.** `frontend/tsconfig.json` sets `"strict": true`. Write real types; a handful of files still use `any` (for example the auth responses in `lib/api/client.ts`), and replacing them is welcome, adding more is not.
- **Path aliases.** Import with `@/lib/...`, `@/components/...`, `@/games/...` and `@/types/...`, as configured in `tsconfig.json`, not with long relative paths.
- **App Router.** Each page is `frontend/app/<route>/page.tsx`. Interactive pages and components start with `'use client'`; keep the directive as low in the tree as you can. A new signed-in route also goes into `protectedPrefixes` in `frontend/middleware.ts` and the disallow list in `frontend/app/robots.ts`.
- **Components.** Named function components with a `Props` interface above them, one main component per file, grouped by feature under `components/` (`exam/`, `learn/`, `brand/`, `arcade/`). Shared data and helpers live in `lib/`, one concern per file.
- **Network.** All API calls go through the single `apiClient` in `lib/api/client.ts`. Do not call `fetch` against the backend from a component.
- **Lint.** ESLint extends `next/core-web-vitals` (`frontend/.eslintrc.json`).

STYLING

- **Theme tokens.** Use the colours in `frontend/tailwind.config.js`: `paper`, `ink`, `mist`, and `cobalt`, `lime` and `coral` with their `soft` variants. Use the shadows `shadow-brutal`, `shadow-brutal-sm`, `shadow-brutal-lg` and `shadow-glass`, and the `font-display` and `font-serif` families. Do not add raw hex values to components.
- **Component classes.** Reuse the classes in `frontend/app/globals.css`: `.btn` with `.btn-cobalt`, `.btn-ink`, `.btn-lime` or `.btn-paper`, plus `.brutal`, `.glass`, `.field` and `.chip`.
- **Copy.** Sentence case, plain verbs, no emoji in the interface. Chapter 4 has the full rules.

PYTHON

- **Layers.** Routers in `src/api/` stay thin: validate, authorise, call a service, shape the response. Services in `src/services/` hold the logic. Pydantic models live in `src/models/schemas.py`, and their field names are camelCase (`gameId`) because they mirror the TypeScript types.
- **Imports** are absolute from the `src` package: `from src.services.videos import find_topic_videos`.
- **Type hints** use `typing` (`List`, `Dict`, `Optional`, `Literal`). Keep code runnable on Python 3.9: no `X` | `None` unions and no `match` statements.

- **Docstrings** open every module and route, in one plain sentence.
- **Blocking work** such as a Supabase query or a DuckDuckGo search inside an async route runs through `asyncio.to_thread`, as `questions.py` and `videos.py` do.
- **Configuration** comes from `os.getenv` with a documented default, and every new name goes into `backend/.env.example`.

DEAD CODE TO LEAVE ALONE OR REMOVE DELIBERATELY

A few files are left over from the hackathon and are not used: the TypeScript files under `backend/src/` (`index.ts`, `config/supabase.ts`, `api/games.ts`, `services/gameScoreService.ts`, `types/`), the `frontend/games/mario/` folder, which no entry in `lib/games.ts` points to, and the `backend/setup.sh` and `start.sh` scripts, which call files that no longer exist. Do not build on them. Removing them is a fine pull request on its own.

16.8 Updating this handbook

The handbook is part of the code. If your change alters behaviour a chapter describes, update the chapter in the same pull request. The source is in `docs/handbook/src/`:

<code>docs/handbook/</code>	
<code>build.py</code>	assembles <code>index.html</code> from <code>src/</code> , generates the contents
<code>make-book.sh</code>	runs <code>build.py</code> , <code>render-pdf.sh</code> , then <code>render-epub.sh</code>
<code>render-pdf.sh</code>	typesets the PDF in two passes for real page numbers
<code>render-epub.sh</code>	packs the Kindle edition with <code>pack-epub.py</code>
<code>src/WRITING.md</code>	the rules every chapter follows
<code>src/chapters/</code>	one HTML fragment per chapter, <code>ch01.html</code> to <code>ch16.html</code> , <code>chA</code> , <code>chB</code>
<code>src/book.css</code>	the stylesheet

Read `src/WRITING.md` before you edit. The rules that trip people up: every name, path and number must exist in the repository; no em dashes; no secrets or personal data; each chapter is well-formed XML with one root `<section>`, because the same fragment goes into the EPUB; and every id starts with the chapter stem. Check a chapter, then build:

```
python3 -c "import xml.dom.minidom,sys; xml.dom.minidom.parseString('<div>'+open(sys.argv[1]).read())"
python3 docs/handbook/build.py          # warns on em dashes and unresolved links
./docs/handbook/make-book.sh            # index.html, PDF and EPUB
```

A new `h3` appears in the contents automatically. A new chapter file also needs its stem added to the right part in the `PARTS` list at the top of `build.py`.

16.9 Study notes

READ IN THIS ORDER

1. `CONTRIBUTING.md` , the short form of this chapter.
2. `.github/workflows/ci.yml` and `.github/PULL_REQUEST_TEMPLATE.md` , what runs and what you are asked.
3. `frontend/tsconfig.json` , `frontend/.eslintrc.json` and `frontend/tailwind.config.js` , the rules the compiler and the design enforce.
4. `backend/src/main.py` and one small router, `backend/src/api/learn.py` , as the pattern for a new route.
5. `frontend/lib/gameMeta.ts` and `frontend/components/GameContainer.tsx` , where a new game is registered.
6. `docs/handbook/src/WRITING.md` , before you touch a chapter.

CHECK YOURSELF

CI is green on your backend pull request. What has it not checked?

Any route's behaviour. The backend job only imports `src.main` , so it proves modules load and routers register, nothing more. You have to start the server and call the route.

Why must a new backend query filter by `current_user["id"]` when every table has RLS?

The backend's client uses the service-role key, which bypasses RLS. The filter in the handler is the only boundary between users.

You add a signed-in page at `/review` . Which two lists change besides the page itself?

`protectedPrefixes` in `frontend/middleware.ts` and the disallow list in `frontend/app/robots.ts` .

Where does a schema change go?

A new SQL file under `backend/database/` , the same change folded into `reset.sql` , RLS and policies on any new table, and a note in the pull request that it must be run on production before merge.

Your chapter edit builds in the browser but stops `build.py` . What is the usual cause?

It is not well-formed XML: an unclosed tag, an unescaped `&` or `<` , or a void element that is not self-closed.

APPENDICES

Reference

A glossary with a reading order for studying the code, and the decisions that shaped it.

- A. Glossary and reading order
- B. Decisions

A Glossary and reading order

The words the code and the product use, defined the way the code uses them and pointed at the file where each one lives, followed by four routes through the repository: one for frontend work, one for games, one for the backend, and one for questions and AI.

The glossary has three parts: the exam vocabulary the product is built on, the project's own terms, and the infrastructure terms that show up in the code. Paths are relative to the repository root. Where a term means something narrower here than it does in general, the definition says so.

A.1 Exams and study

TERM	MEANING HERE	WHERE
SAT	The digital SAT. Exam id <code>sat</code> . Sections are Math (<code>quant</code> , 95 seconds per question) and Reading & Writing (<code>verbal</code> , 71 seconds). Four answer choices. Target scores 400 to 1600.	<code>EXAMS</code> in <code>frontend/lib/exam.ts</code> and <code>backend/src/services/question_sources.py</code> ; <code>Profile</code> in <code>schemas.py</code>
GRE	Exam id <code>gre</code> . Sections are Quantitative (<code>quant</code> , 105 seconds) and Verbal (<code>verbal</code> , 90 seconds). Five answer choices in the backend's settings. Target scores 260 to 340.	Same files
Exam	Which test a user studies for: <code>sat</code> or <code>gre</code> . Stored on the profile, on each material, and in the browser's <code>exam_prefs</code> .	<code>ExamId</code> in <code>lib/exam.ts</code>
Section	The half of an exam a question belongs to. Always one of two ids: <code>quant</code> or <code>verbal</code> , whatever the exam calls them on screen.	<code>SectionId</code> in <code>lib/exam.ts</code>
Quant, verbal	The two section ids. <code>quant</code> is SAT Math or GRE Quantitative; <code>verbal</code> is SAT Reading & Writing or GRE Verbal.	<code>lib/exam.ts</code> ; <code>materials.section</code> check constraint
Exam prefs	The exam, section and question count (6, 10 or 20) a player picked. Kept in <code>localStorage</code> as <code>exam_prefs</code> .	<code>getExamPrefs</code> , <code>setExamPrefs</code> , <code>QUESTION_COUNTS</code>
Course	One exam section presented as a course: a title, a summary and its topics. There are four: SAT Math, SAT Reading & Writing, GRE Quant, GRE Verbal.	<code>COURSES</code> in <code>frontend/lib/courses.ts</code>
Topic	A content area inside a course, matching the official content domains (for SAT Math: Algebra, Advanced Math, Problem-Solving and Data Analysis, Geometry and Trigonometry). Every question carries one, and stats are kept per topic.	<code>Course.topics</code> ; <code>question_attempts.topic</code>
Domain	The College Board's name for what the app calls a topic. The code comment calls topics "official content domains".	<code>lib/courses.ts</code>
Skill	A finer label under a topic on official questions, such as Words in Context or Boundaries. Used for quick-pace filtering and for hints.	<code>Question.skill</code> ; <code>QUICK_SKILLS</code> in <code>question_sources.py</code> ; <code>lib/hints.ts</code>
Difficulty	<code>easy</code> , <code>medium</code> or <code>hard</code> . The agent recommends one from the user's accuracy (under 50% easy, over 75% hard).	<code>DIFFICULTIES</code> ; <code>analyze_performance</code> in <code>agent.py</code>
Pace: quick, deep	What kind of question a game needs. <code>quick</code> games want short questions answerable in seconds; <code>deep</code> games take passages and multi-step problems. Squid Game is the only deep game.	<code>Pace</code> , <code>gamePace</code> , <code>DEEP_GAMES</code> in <code>lib/exam.ts</code> ; <code>is_quick_question</code>

TERM	MEANING HERE	WHERE
Weak topic, strong topic	A topic answered under 50% correct in a session is weak; 80% or more is strong. The lists in <code>user_stats</code> are merged session by session, and a topic moves between them. The agent asks for more questions in weak topics.	<code>_update_user_stats</code> in <code>game_service.py</code> ; <code>CollegeBoardSource._pick_candidates</code>
Mock exam	A timed, game-free test at the real exam's pace, in three lengths: <code>quick</code> (10 questions), <code>module</code> (22 or 27 SAT, 12 GRE) and <code>section</code> (two modules). It ends with a rough scaled-score estimate and is saved as a session with game id <code>mock-exam</code> .	<code>frontend/app/mock/page.tsx</code>
Target score, test date, daily minutes	The goal fields on a profile. Daily minutes is 5 to 240 on the server; the form offers 10, 20, 30 or 45.	<code>Profile</code> in <code>schemas.py</code> ; <code>DAILY_MINUTES</code> in <code>lib/profile.ts</code>

A.2 Project terms

TERM	MEANING HERE	WHERE
Game	One of six playable games, by id: <code>zombie</code> , <code>whackamole</code> , <code>carnival</code> (shown as Balloon Pop), <code>subway-surfers</code> , <code>squid-game</code> , <code>pac-man</code> . Each answers questions as part of play. A <code>mario</code> game exists in <code>frontend/games/mario/</code> and <code>GameRenderer.ts</code> but is not in the catalogue.	<code>frontend/lib/games.ts</code> , <code>lib/gameMeta.ts</code> , <code>frontend/games/</code>
Game container	The React component that loads questions, mounts a game's engine and saves the score at the end.	<code>frontend/components/*GameContainer.tsx</code>
Session	One finished game or mock exam: a row in <code>game_sessions</code> . Not a login session.	<code>GameService.save_game_session</code> ; <code>lib/sessions.ts</code>
Attempt	One question answered within a session: a row in <code>question_attempts</code> , sent as <code>questionAttempts</code> in the analytics.	<code>QuestionAttempt</code> in <code>schemas.py</code>
Analytics	The summary a game posts when it ends: score, counts, attempts, per-topic performance, streak and average response time.	<code>GameAnalytics</code> in <code>schemas.py</code>
Stats	A user's running totals and weak and strong topics, one <code>user_stats</code> row.	<code>GET /api/stats/user</code>
Question source	Where a question came from, in <code>Question.source</code> : <code>official</code> (College Board's question bank), <code>web</code> (extracted from a practice page found by search), <code>ai</code> (written by the LLM), <code>custom</code> (the user's own approved material) or <code>practice</code> (the built-in offline bank).	<code>question_sources.py</code> , <code>agent.py</code> , <code>custom_questions.py</code> , <code>lib/questionBank.ts</code>
Agent	<code>ExamLearningAgent</code> : reads the user's stats, then assembles a question set from the sources above, weighted toward weak topics.	<code>backend/src/services/agent.py</code>
Built-in bank	A fixed set of questions per exam and section shipped with the frontend, used when the backend returns nothing.	<code>getBankQuestions</code> in <code>frontend/lib/questionBank.ts</code>
Hint	A strategy tip, an elimination of one wrong choice, or a "why not" note for a chosen answer, built from the question's wording, skill and topic.	<code>frontend/lib/hints.ts</code> ; <code>components/exam/HintButton.tsx</code>
Lesson	The study page for one course topic: summary, core ideas, one worked example, traps, and a quick check, with lesson videos alongside.	<code>Lesson</code> , <code>LESSONS</code> in <code>frontend/lib/lessons.ts</code> ; <code>app/learn/[exam]/[topic]/page.tsx</code>
Quick check	The one multiple-choice question at the end of a lesson, answered in place with no score saved.	<code>Lesson.check</code> ; <code>components/learn/QuickCheck.tsx</code>
Learned	A lesson the viewer marked as learned, kept only in this browser as <code>learned_lessons</code> .	<code>useLearned</code> in <code>lib/learnProgress.ts</code>

TERM	MEANING HERE	WHERE
Lesson videos, trusted	YouTube videos found for a topic and ranked. <code>trusted</code> marks a video from a channel in <code>TRUSTED_CHANNELS</code> .	<code>backend/src/services/videos.py</code> ; <code>components/learn/Videos.tsx</code>
Material	A file or pasted text a user uploaded to practise from: a row in <code>materials</code> .	<code>backend/src/api/materials.py</code> ; <code>app/materials/page.tsx</code>
Custom question	A question from a material: a row in <code>custom_questions</code> . Its <code>origin</code> is <code>extracted</code> (copied from the material) or <code>generated</code> (written by the LLM from it); its <code>status</code> is <code>pending</code> until approved. Only approved ones, with an answer, appear in games, as up to half of a set.	<code>services/custom_questions.py</code> ; <code>services/material_parser.py</code>
Cover	The illustrated SVG art for a game, drawn in the site palette, shown on cards and loading screens.	<code>GameCover</code> in <code>components/brand/GameCover.tsx</code>
Arcade frame	The full-screen cabinet the arcade-style games render inside: pixel font (Press Start 2P), a neon accent colour and a CRT overlay.	<code>ArcadeFrame</code> in <code>components/arcade/ArcadeFrame.tsx</code> ; <code>lib/arcade.ts</code>
HUD	In-game panels and chips drawn over the canvas.	<code>components/arcade/GameHud.tsx</code>
High score	The best score for a game on this device, in <code>localStorage</code> as <code>arcade_hi_{gameId}</code> . Not sent to the server.	<code>getHighScore</code> , <code>recordHighScore</code> in <code>lib/arcade.ts</code>
Lane	An answer choice's position in Subway Surfers, reused as a coloured letter chip elsewhere.	<code>components/exam/LaneChip.tsx</code>
Profile, username	The user's display name, unique lower-case handle, exam goals and avatar colour: one <code>profiles</code> row.	<code>backend/src/api/profile.py</code> ; <code>lib/profile.ts</code>
Pending profile	A profile filled in at signup when no session came back (email confirmation on), held in <code>localStorage</code> and saved at first login with the same email.	<code>savePendingProfile</code> , <code>takePendingProfile</code>
Account	The signed-in user plus their profile, cached across pages.	<code>useAccount</code> in <code>lib/useAccount.ts</code>

A.3 Infrastructure terms

TERM	MEANING HERE	WHERE
Supabase	Hosted Postgres plus Auth. The backend uses both through one Python client; the frontend does not use it at all.	<code>backend/src/utils/database.py</code>
Service-role key	The Supabase key that bypasses Row Level Security. The backend reads it from <code>SUPABASE_SERVICE_ROLE_KEY</code> (or <code>SUPABASE_SERVICE_KEY</code>) and uses it for every query.	<code>Database.get_client</code>
Anon key	<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> . Subject to RLS. Only a fallback in the backend.	<code>database.py</code> , <code>config.py</code>
RLS	Row Level Security: Postgres policies that limit rows to <code>auth.uid() = user_id</code> . Enabled on all six tables, bypassed by the service role, so the backend's own filters are the real access control.	<code>backend/database/*.sql</code> ; 13.5
Access token, <code>auth_token</code>	The Supabase JWT returned by login and sent as a bearer header. The browser keeps it in <code>localStorage</code> and in a cookie both named <code>auth_token</code> .	<code>frontend/lib/api/client.ts</code> ; <code>frontend/middleware.ts</code>
<code>get_current_user</code>	The FastAPI dependency that turns a bearer token into <code>{id, email}</code> or a 401.	<code>backend/src/api/auth.py</code>
Guest user id	The all-zero UUID the question route passes to the agent when no one is signed in.	<code>backend/src/api/questions.py</code>
Sanitized HTML	Official question HTML and MathML re-rendered from an allowlist of tags and attributes, in the <code>*Html</code> fields.	<code>backend/src/utils/html_utils.py</code> ; <code>components/QuestionContent.tsx</code>
OpenRouter	The LLM gateway the agent and the material parser call, through the OpenAI SDK, with <code>OPENROUTER_API_KEY</code> and model <code>OPENROUTER_MODEL</code> (default <code>anthropic/claude-haiku-4.5</code>).	<code>backend/src/services/agent.py</code> ; <code>backend/src/config.py</code>
<code>CUSTOM_ID_OF_FSET</code>	1,500,000,000, added to a custom question's database id so it does not clash with ids from other sources.	<code>services/custom_questions.py</code>

A.4 Reading order: frontend contributor

- `frontend/app/layout.tsx` and `frontend/app/page.tsx`, the shell and the public landing page.
- `frontend/middleware.ts`, which paths need a login and how that is checked.
- `frontend/lib/api/client.ts`, every backend call and the token's storage.
- `frontend/lib/useAccount.ts` and `frontend/lib/profile.ts`, the signed-in user and their profile.
- `frontend/app/login/page.tsx` and `frontend/app/signup/page.tsx`, with `components/auth/AuthShell.tsx`.
- `frontend/components/DashboardLayout.tsx` and `frontend/app/dashboard/page.tsx`.

7. `frontend/lib/exam.ts` and `frontend/lib/courses.ts` , the exam vocabulary every page shares.
8. `frontend/app/learn/page.tsx` , `app/learn/[exam]/[topic]/page.tsx` and `frontend/lib/lessons.ts` .
9. `frontend/app/stats/page.tsx` and `frontend/lib/sessions.ts` .
10. `frontend/app/materials/page.tsx` with `frontend/lib/materials.ts` .
11. `frontend/app/mock/page.tsx` , the largest page, a self-contained timed exam.
12. `frontend/tailwind.config.js` and `frontend/app/globals.css` , for the palette and the brutalist borders and shadows.

A.5 Reading order: game developer

1. `frontend/lib/games.ts` and `frontend/lib/gameMeta.ts` , the catalogue.
2. `frontend/app/games/page.tsx` and `app/games/[gameId]/select/page.tsx` , how a player picks a game and an exam section.
3. `frontend/app/games/[gameId]/page.tsx` and `layout.tsx` , which container mounts.
4. `frontend/lib/api/questions.ts` , `SATQuestion` , `fetchAIQuestions` and the fallback to `lib/questionBank.ts` .
5. `frontend/games/BaseGame.ts` and `frontend/games/GameRenderer.ts` , the engine contract.
6. One simple game end to end: `components/WhackAMoleGameContainer.tsx` with `games/whackamole/` .
7. One arcade-style game: `components/PacManGameContainer.tsx` with `games/pac-man/` , `components/arcade/ArcadeFrame.tsx` and `GameHud.tsx` .
8. `frontend/components/exam/` : `QuestionCard` , `HintButton` , `useQuestionHints` , `PauseMenu` , `ReviewList` , `GameIntro` .
9. `frontend/lib/hints.ts` and `frontend/lib/arcade.ts` .
10. `backend/src/models/schemas.py` (`GameAnalytics`) and `backend/src/services/game_service.py` , what happens to the score you post.

A.6 Reading order: backend contributor

1. `backend/src/main.py` , routers, CORS and the exception handler.
2. `backend/database/reset.sql` and `backend/database/add_materials.sql` , the schema.
3. `backend/src/utils/database.py` and `backend/src/config.py` , keys and environment.
4. `backend/src/api/auth.py` and `backend/src/services/auth_service.py` ; read 13.7 alongside.
5. `backend/src/models/schemas.py` .
6. `backend/src/api/profile.py` , `stats.py` , `games.py` with `services/game_service.py` .
7. `backend/src/api/materials.py` , the most complete router.
8. `backend/api/index.py` , `backend/vercel.json` and `backend/requirements.txt` , how it is deployed.

A.7 Reading order: questions and AI

1. `backend/src/api/questions.py` , the one entry point and its parameters.

2. `backend/src/services/agent.py` , `generate_questions` first, then `_questions_from_llm` and `_normalize_llm_question` .
3. `backend/src/services/supabase_agent_ops.py` , the two read functions the agent uses (the write functions are unused).
4. `backend/src/services/question_sources.py` : `CollegeBoardSource` , `is_quick_question` , `_balance_difficulty` , then `WebQuestionSource` .
5. `backend/src/utlils/html_utils.py` , sanitized HTML and plain text from the same tree.
6. `backend/src/services/custom_questions.py` and `backend/src/services/material_parser.py` , the user's own questions.
7. `backend/src/services/videos.py` , the other search-driven service.
8. `frontend/lib/api/questions.ts` , `frontend/lib/questionBank.ts` and `frontend/components/QuestionContent.tsx` , the receiving end.

A.8 Study notes

READ IN THIS ORDER

1. `frontend/lib/exam.ts` , where most of the exam vocabulary above is defined.
2. `frontend/lib/courses.ts` and `frontend/lib/gameMeta.ts` .
3. `backend/src/models/schemas.py` , where the project terms become types.

CHECK YOURSELF

What is the difference between a session and a login?

A session is one finished game or mock exam, a row in `game_sessions` . A login produces an access token; the code never calls that a session.

Which question sources can produce sanitized HTML?

Only `official` . Web, AI, custom and practice questions are plain text.

Which game is deep pace, and what does that change?

Squid Game. It is sent `pace=deep` , so the sources may return passages and multi-step problems; quick games get only short questions.

B Decisions

This is a log of the design and engineering decisions that can be read off the code. Each entry gives the decision, the reason (from a comment in the code where there is one), and the consequence a contributor lives with. None of them is sacred. If you change one, change the entry too.

B.1 A separate FastAPI backend

Decision. All data access and every outside call runs in a Python FastAPI app in `backend/`, deployed as its own Vercel project. The Next.js app has no API routes.

Reason. The question pipeline is Python: the `openai` client for OpenRouter, `ddgs` for DuckDuckGo, `pypdf` for uploads, and `httpx` for the College Board and YouTube. Keeping secrets and slow work off the browser and out of the page server keeps the frontend a static-ish client.

Consequence. Two deploys, two sets of environment variables, and CORS: `ALLOWED_ORIGINS` must name the frontend's origin. The frontend must be built with `NEXT_PUBLIC_API_URL` pointing at the backend. A change that spans both needs both deployed.

B.2 Supabase for auth and data, behind the backend

Decision. Supabase provides Auth and Postgres. Only the backend talks to it, through one client built by `Database.getClient`, which prefers `SUPABASE_SERVICE_ROLE_KEY`.

Reason. One client with full rights is simple: no per-request client, no RLS to reason about in each query. The comment in `utils/database.py` says it plainly: the "service role key bypasses RLS".

Consequence. The RLS policies in `backend/database/*.sql` are defence in depth only. Isolation between users depends on every query filtering on `user_id`. The frontend's `@supabase/ssr` and `@supabase/supabase-js` dependencies are unused.

B.3 A bearer token in localStorage and a cookie

Decision. The Supabase access token is returned by `/api/auth/login`, stored by `ApiClient.setToken` under `auth_token` in `localStorage` and a seven-day cookie, and sent as `Authorization: Bearer`. `middleware.ts` checks only that the cookie exists. `AuthService.getUser` decodes the JWT without verifying its signature and checks `exp`.

Reason. The cookie exists so the edge middleware can redirect signed-out visitors before a page renders; the comment in `setToken` says "Also set cookie for middleware auth check". Decoding without a network call keeps every request fast.

Consequence. No refresh: when the token expires, calls fail with 401 until the student logs in again. And because the signature is not checked while the backend holds the service-role key, a forged token is trusted. Verifying the JWT is the most important open fix in the codebase.

B.4 Official questions first, then the LLM, then an offline bank

Decision. `ExamLearningAgent.generate_questions` takes SAT questions from the College Board Educator Question Bank (`CollegeBoardSource`). Any shortfall, and every GRE question, comes from one LLM call that is given up to four web pages of practice questions (`WebQuestionSource.find_pages`) and told to extract real questions first and write the rest. If the backend returns nothing, `fetchAIQuestions` falls back to `getBankQuestions` in `frontend/lib/questionBank.ts`.

Reason. The landing page promises questions "Drafted in the real format" and "Checked against the web". An LLM alone invents plausible but wrong questions; real sources fix that where they exist. The offline bank means a game always starts, even with no API key.

Consequence. A question is labelled `web` only if the first 50 normalized characters of its stem or passage appear on the cited page (`_normalize_llm_question`); otherwise it is `ai`. Every question carries `source`, `sourceName` and `sourceUrl` so the UI can say where it came from. One LLM call is capped at `MAX_LLM_QUESTIONS` (25). The offline bank is small: nine questions per section, 36 in all.

B.5 Question length matched to the game

Decision. Games have a pace. `gamePace` marks `squid-game` as `deep` and every other game as `quick`, and the backend filters with `is_quick_question` and `_fits_pace`.

Reason. The comment in `question_sources.py`: "Fast games need questions that can be read and answered in seconds; slow games take the long passages and multi-step problems."

Consequence. Quick games never see reading passages, so verbal practice in them is limited to short skills (`QUICK_SKILLS`). The College Board fetch oversamples five times for quick games because so many candidates are dropped.

B.6 Weak topics decided per game, by fixed thresholds

Decision. `GameService._update_user_stats` marks a topic weak below 50 percent accuracy in the game just played and strong at 80 percent or more, and moves topics between the two lists.

Reason. The comment: "A topic moves between the lists as the player improves or slips."

Consequence. One bad game makes a topic weak, and one good game clears it. Topics in neither range keep whatever label they had. The dashboard and the agent both read these lists.

B.7 Lessons as static content

Decision. The fifteen lessons are TypeScript constants in `frontend/lib/lessons.ts`, one per topic in `COURSES`, each with ideas, a worked example, traps and a quick check.

Reason. Lessons must be right, and they change rarely. Static content is reviewed in a pull request, costs nothing to serve, and needs no backend.

Consequence. Editing a lesson is a code change and a deploy. A new topic in `COURSES` needs a lesson in the matching array, or it has none.

B.8 A video provider chain with a one-day cache

Decision. `videos.py` tries the YouTube Data API (`YOUTUBE_API_KEY`), then Serper (`SERPER_API_KEY`), then DuckDuckGo; the first provider with results wins. Results are cached in memory for `CACHE_TTL_SECONDS` (24 hours), ranked toward trusted channels and 3 to 25 minute lengths, with up to `GREGMAT_SLOTS` (3) GregMat videos first.

Reason. DuckDuckGo needs no key but rate limits; the paid providers are reliable but optional. The module docstring explains the ranking: lesson-length videos from established test-prep channels.

Consequence. With no keys set, videos still work, slowly and sometimes not at all. Only non-empty results are cached, "so a rate-limited search is retried next time". The cache is per process, so each Vercel instance warms its own.

B.9 Profiles in a table, with a pending-profile flow

Decision. Profiles live in `profiles`, keyed by `user_id`, saved by an upsert in `PUT /api/profile`. When signup returns no session (email confirmation on), the profile is parked in `localStorage` as `pending_profile` and saved on first login.

Reason. The comment in `frontend/lib/profile.ts`: "When Supabase requires email confirmation, signup returns no session, so the profile can't be saved yet."

Consequence. A student who confirms and logs in on another device has no profile and is asked again. Usernames are unique; a clash is a 409, and signup or login sends the student to `/profile?setup=1` rather than failing.

B.10 Lesson progress in localStorage

Decision. "Mark as learned" writes to the `learned_lessons` key in `frontend/lib/learnProgress.ts`. So do exam preferences (`exam_prefs`) and high scores (`arcade_hi_<gameId>`).

Reason. The file's comment: "Kept in this browser only." It needs no table, no endpoint and no sign-in.

Consequence. Progress does not follow the student between browsers. Moving it to a table is a small, self-contained change.

B.11 Mock exams saved as game sessions

Decision. `app/mock/page.tsx` saves results with `saveScore( 'mock-exam', ... )`, in the same `GameAnalytics` shape as a game.

Reason. The comment: "Save it alongside game sessions so it feeds statistics and weak topics." No new table or endpoint.

Consequence. `gameTitle` special-cases `mock-exam`. Statistics mix mock exams with games.

B.12 Uploaded questions need approval

Decision. Questions from uploads go into `custom_questions` as `pending`. Only `approved` ones are served, and they make up at most half of a game's set (`limit // 2` in `get_questions`).

Reason. Parsed and LLM-extracted questions can have the wrong or missing answer (`correct_answer` is nullable). The student is the only one who can confirm them.

Consequence. Uploading has a review step. The rest of every set still comes from the agent, so the student's own material never crowds out real exam questions.

B.13 A neo-brutalist and glass design system

Decision. The site's look is thick ink outlines with hard offset shadows (`brutal`, `brutal-sm`, `brutal-lg` in `tailwind.config.js`) combined with frosted `glass` panels, over a paper background, in a five-colour palette: `paper`, `ink`, `cobalt`, `lime`, `coral`.

Reason. The comment in `app/globals.css`: "SATistics theme: brutalist outlines with frosted glass panels". It reads as a game, not a textbook.

Consequence. Use the component classes in `globals.css` (`.brutal`, `.glass`, `.btn`) and the Tailwind tokens rather than raw colours. The in-game screens use a separate arcade skin with the Press Start 2P font from `lib/arcade.ts`.

B.14 Game covers drawn as inline SVG

Decision. `components/brand/GameCover.tsx` draws each game's cover as SVG in the site palette, with a `FALLBACK` for unknown ids.

Reason. The file's comment: "Illustrated cover art for each game, drawn as SVG in the site palette." No image assets to license, load or keep in sync with the palette.

Consequence. A new game needs a cover entry in `COVERS`, written as SVG.

B.15 Schema as hand-run SQL files

Decision. The schema lives in `backend/database/*.sql`, run by hand in the Supabase SQL editor. There is no migration tool.

Reason. Few tables, one environment, one maintainer.

Consequence. Order matters and nothing records what has been applied. `reset.sql` recreates the four core tables but not `materials` or `custom_questions`, so after a reset run `add_materials.sql` again.

B.16 The backend as one Vercel Python function

Decision. `backend/vercel.json` rewrites every path to `/api/index`, and `backend/api/index.py` exposes the whole FastAPI app.

Reason. One deploy target for both apps, no server to run.

Consequence. In-memory caches are per instance and lost on cold starts. Long question builds (web search plus an LLM call) run inside a single function invocation. The Docker, Fly, Railway and Render configs in `backend/` are left over and not used.

SATistics Engineering Handbook, Revision A, 2 October 2026. Written by Shiv Vyas and contributors from the code on `main`. Source of record: the code. Built by `docs/handbook/build.py`; page numbers by `render-pdf.sh`; the Kindle edition by `render-epub.sh`.